

國立臺北商業大學

資訊管理系

114資訊系統專案設計

系統手冊



組 別：第114401組

題 目：PingPro桌訓系統

指導老師：陳信宏老師

組 長：11146035 張禎益

組 員：11146021 曾順暉

11146027 高偉倫

11146036 陳頤謙

中華民國114年12月31日

目錄

第一章 前言

1-1 背景介紹.....	12
1-2 動機.....	12
1-3 系統目的與目標.....	12
1-4 預期成果	12

第二章 營運計畫

2-1 可行性分析.....	13
2-2 商業模式—Business model	13
2-3 市場分析—STP	14
2-4 競爭力分析SWOT-TOWS.....	15

第三章 系統規格

3-1 系統架構.....	17
3-2 系統軟、硬體需求與技術平台.....	18
3-3 使用標準與工具.....	19

第四章 專案時程與組織分工

4-1 專案時程：甘特圖.....	21
4-2 專案組織與分工.....	22
4-3 上傳GitHub紀錄.....	25

第五章 需求模型

5-1 使用者需求.....	27
5-2 使用個案圖(Use case diagram).....	31
5-3 使用個案描述：使用活動圖.....	34
5-4 分析類別圖(Analysis class diagram).....	64

第六章 設計模型

6-1 循序圖(Sequential diagram).....	65
6-2 設計類別圖(Design class diagram).....	84

第七章 實作模型

7-1 佈署圖(Deployment diagram).....	85
7-2 套件圖(Package diagram).....	87
7-3 元件圖(Component diagram).....	88
7-4 狀態機(State machine).....	90

第八章 資料庫設計

8-1 資料庫關聯表.....	112
8-2 表格及其Meta data.....	113

第9章 程式

9-1 元件清單及其規格描述.....	116
9-2 其他附屬之各種元件.....	121
9-3 程式碼.....	122

第10章 測試模型

10-1 測試計畫.....	138
10-2 測試個案與測試結果資料.....	147

第11章 操作手冊..... 148

第12章 使用手冊..... 149

第13章 感想..... 170

第14章 參考資料..... 172

附錄

附錄-1 評審建議修改事項.....	173
附錄-2 系統架設部署設定.....	174

圖目錄

圖3-1-1	系統架構.....	17
圖3-1-2	系統架構.....	17
圖4-3-1	11146021 曾順暉上傳github紀錄.....	25
圖4-3-2	11146035 張禎益上傳github紀錄.....	25
圖4-3-3	11146027 高偉倫上傳github紀錄.....	26
圖4-3-4	11146036 陳頤謙上傳github紀錄.....	26
圖5-2-1	使用個案圖-Use case diagram-選手個案圖.....	31
圖5-2-2	使用個案圖-Use case diagram- 教練個案圖.....	32
圖5-2-3	使用個案圖-Use case diagram- 管理者個案圖.....	33
圖5-3-1	使用活動圖-Activity diagram- 選手註冊登入、教練登入.....	34
圖5-3-2	使用活動圖-Activity diagram- 選手勾查看及選訓練紀錄.....	35
圖5-3-3	使用活動圖-Activity diagram- 選手查看公告.....	36
圖5-3-4	使用活動圖-Activity diagram- 選手填寫訓練狀況.....	37
圖5-3-5	使用活動圖-Activity diagram- 選手查看簽到紀錄.....	38
圖5-3-6	使用活動圖-Activity diagram- 選手上傳飲食照片.....	39
圖5-3-7	使用活動圖-Activity diagram- 選手填寫壓力評估.....	40
圖5-3-8	使用活動圖-Activity diagram- 選手填寫睡眠紀錄.....	41
圖5-3-9	使用活動圖-Activity diagram- 選手動作分析模組.....	42
圖5-3-10	使用活動圖-Activity diagram- 選手使用智能助手.....	43
圖5-3-11	使用活動圖-Activity diagram- 選手LINE Bot綁定帳號.....	44
圖5-3-12	使用活動圖-Activity diagram- 選手LINE Bot查詢出缺席.....	45
圖5-3-13	使用活動圖-Activity diagram- 選手LINE Bot查詢公告.....	46
圖5-3-14	使用活動圖-Activity diagram- 選手LINE Bot今日訓練狀況.....	47
圖5-3-15	使用活動圖-Activity diagram- 選手LINE Bot上傳飲食照片.....	48
圖5-3-16	使用活動圖-Activity diagram- 教練建立訓練菜單.....	49

圖5-3-17	使用活動圖-Activity diagram-	教練編輯訓練菜單.....	50
圖5-3-18	使用活動圖-Activity diagram-	教練刪除訓練菜單.....	51
圖5-3-19	使用活動圖 -Activity diagram-	教練查看已派發訓練菜單.....	52
圖5-3-20	使用活動圖 -Activity diagram-	教練查看選手完成度.....	53
圖5-3-21	使用活動圖 -Activity diagram-	教練發布公告活動圖.....	54
圖5-3-22	使用活動圖 -Activity diagram-	教練刪除公告活動圖	54
圖5-3-23	使用活動圖 -Activity diagram-	教練查看選手訓練狀況.....	55
圖5-3-24	使用活動圖 -Activity diagram-	教練點名.....	56
圖5-3-25	使用活動圖 -Activity diagram-	教練查看飲食紀錄.....	57
圖5-3-26	使用活動圖 -Activity diagram-	教練查看壓力評估	58
圖5-3-27	使用活動圖 -Activity diagram-	教練查看睡眠紀錄.....	59
圖5-3-28	使用活動圖 -Activity diagram-	教練查看生理評估.....	60
圖5-3-29	使用活動圖 -Activity diagram-	管理者新增教練帳號.....	61
圖5-3-30	使用活動圖 -Activity diagram-	個人自介新增.....	62
圖5-3-31	使用活動圖 -Activity diagram-	個人自介查詢.....	62
圖5-3-32	使用活動圖 -Activity diagram-	個人自介修改.....	63
圖5-4-1	使用分析類別圖-Analysis class diagram.		64
圖6-1-1	循序圖-Sequential diagram-	系統登入/登出.....	65
圖6-1-2	循序圖-Sequential diagram-	訓練菜單.....	66
圖6-1-3	循序圖-Sequential diagram-	發布及查看公告.....	67
圖6-1-4	循序圖-Sequential diagram-	訓練狀況.....	68
圖6-1-5	循序圖-Sequential diagram-	選手上傳飲食照片.....	69
圖6-1-6	循序圖-Sequential diagram-	選手使用智能助手.....	70
圖6-1-7	循序圖-Sequential diagram-	選手填寫壓力評估.....	71
圖6-1-8	循序圖-Sequential diagram-	選手填寫睡眠紀錄.....	72
圖6-1-9	循序圖-Sequential diagram-	選手填寫生理評估.....	73
圖6-1-10	循序圖-Sequential diagram-	選手LINE Bot綁定帳號.....	74

圖6-1-11	循序圖-Sequential diagram-	選手LINE Bot查詢出缺勤紀錄.....	75
圖6-1-12	循序圖-Sequential diagram-	選手LINE Bot公告查詢.....	76
圖6-1-13	循序圖-Sequential diagram-	選手LINE Bot今日訓練狀況.....	77
圖6-1-14	循序圖-Sequential diagram-	選手LINE Bot上傳飲食照片.....	78
圖6-1-15	循序圖-Sequential diagram-	選手動作分析模組.....	79
圖6-1-16	循序圖-Sequential diagram-	教練查看飲食紀錄.....	80
圖6-1-17	循序圖-Sequential diagram-	教練查看壓力評估.....	81
圖6-1-18	循序圖-Sequential diagram-	教練查看睡眠紀錄.....	82
圖6-1-19	循序圖-Sequential diagram-	教練查看生理評估.....	83
圖6-2-1	設計類別圖-Design class diagram.		84
圖7-1-1	佈署圖-Deployment diagram.		85
圖7-1-2	佈署圖-Deployment diagram.		86
圖7-2-1	套件圖-Package diagram.		87
圖7-2-2	套件圖-Package diagram.		87
圖7-3-1	元件圖-Component diagram-	選手端.....	88
圖7-3-2	元件圖-Component diagram-	教練端.....	88
圖7-3-3	元件圖-Component diagram-	管理者端.....	89
圖7-3-4	元件圖-Component diagram-	AI動作分析模組.....	89
圖7-4-1	狀態機-State machine-	選手教練登入登出.....	90
圖7-4-2	狀態機-State machine-	選手訓練完成度.....	91
圖7-4-3	狀態機-State machine-	選手查看公告.....	92
圖7-4-4	狀態機-State machine-	選手填寫表單	92
圖7-4-5	狀態機-State machine-	選手編輯狀態表單.....	93
圖7-4-6	狀態機-State machine-	選手上傳飲食照片.....	94
圖7-4-7	狀態機-State machine-	選手LINE Bot 綁定帳號.....	95
圖7-4-8	狀態機-State machine-	LINE Bot 選手查詢出缺勤紀錄.....	96
圖7-4-9	狀態機-State machine-	LINE Bot選手公告查詢.....	97

圖7-4-10	狀態機-State machine- LINE Bot 選手今日訓練狀況.....	97
圖7-4-11	狀態機-State machine- LINE Bot 選手上傳飲食照片.....	98
圖7-4-12	狀態機-State machine- 選手動作分析模組.....	99
圖7-4-13	狀態機-State machine- 選手使用智能助手.....	100
圖7-4-14	狀態機-State machine- 選手填寫壓力評估.....	101
圖7-4-15	狀態機-State machine- 選手填寫睡眠紀錄.....	102
圖7-4-16	狀態機-State machine- 選手填寫生理評估.....	103
圖7-4-17	狀態機-State machine- 教練派發訓練菜單.....	104
圖7-4-18	狀態機-State machine- 教練發布公告.....	105
圖7-4-19	狀態機-State machine- 教練編輯公告.....	105
圖7-4-20	狀態機-State machine- 教練點名.....	106
圖7-4-21	狀態機-State machine- 教練查看點名紀錄.....	106
圖7-4-22	狀態機-State machine- 教練查看飲食紀錄.....	107
圖7-4-23	狀態機-State machine- 教練查看壓力評估.....	108
圖7-4-24	狀態機-State machine- 教練查看睡眠紀錄.....	109
圖7-4-25	狀態機-State machine- 教練查看生理評估.....	110
圖7-4-26	狀態機-State machine- 個人自介查詢.....	111
圖7-4-27	狀態機-State machine- 修改自我介紹.....	111
圖8-1-1	資料庫關聯表.....	112
圖8-2-1	資料庫描述-使用者.....	113
圖8-2-2	資料庫描述-教練.....	113
圖8-2-3	資料庫描述-選手.....	113
圖8-2-4	資料庫描述-公告.....	113
圖8-2-5	資料庫描述-飲食紀錄.....	114
圖8-2-6	資料庫描述-訓練計畫.....	114
圖8-2-7	資料庫描述-出缺席.....	114
圖8-2-8	資料庫描述-訓練紀錄.....	114

圖8-2-9	資料庫描述-心理狀況.....	115
圖8-2-10	資料庫描述-生理狀況.....	115
圖8-2-11	資料庫描述-管理員.....	115
圖12-1-1	帳號註冊.....	149
圖12-1-2	登入介面.....	150
圖12-1-3	教練個人資訊顯示.....	150
圖12-1-4	教練介面功能顯示.....	151
圖12-1-5	公告列表.....	151
圖12-1-6	新增公告.....	152
圖12-1-7	派發任務.....	152
圖12-1-8	訓練分析.....	153
圖12-1-9	睡眠紀錄.....	153
圖12-1-10	心理追蹤.....	154
圖12-1-11	飲食紀錄.....	155
圖12-1-12	點名系統.....	155
圖12-1-13	選手登入介面.....	156
圖12-1-14	選手登入後的介面，含選手資訊與教練發的公告.....	157
圖12-1-15	選手功能.....	157
圖12-1-16	聊天機器人.....	158
圖12-1-17	每日訓練狀況.....	159
圖12-1-18	訓練狀況圖表.....	159
圖12-1-19	查看點名紀錄.....	160
圖12-1-20	上傳訓練資料.....	160
圖12-1-21	訓練評估表單.....	161
圖12-1-22	睡眠紀錄.....	161
圖12-1-23	飲食紀錄.....	162
圖12-1-24	管理員後台.....	162

圖12-1-25 上傳影片.....	163
圖12-1-26 啟動運動姿勢分析.....	164
圖12-1-27 動作姿勢分析回饋.....	164
圖12-1-28 Linebot公告查詢.....	165
圖12-1-29 Linebot出缺勤紀錄查詢.....	165
圖12-1-30 Linebot今日訓練狀況.....	166
圖12-1-31 Linebot飲食控制.....	167
圖12-1-32 Linebot Richmenu.....	168
圖12-1-33 State Transition Diagram.....	169
圖A-2-1 名稱設定.....	177
圖A-2-2 OS設定.....	178
圖A-2-3 硬體設定.....	178
圖A-2-4 金鑰設定.....	179
圖A-2-5 網路設定.....	179
圖A-2-6 網路設定(Security Group).....	180
圖A-2-7 硬碟設定.....	180
圖A-3-1 EC2 Connect.....	181
圖A-3-2 Linux連線方式.....	181
圖A-3-3 Windows連線方式.....	182
圖A-5-1 Linux連線方式.....	186
圖A-5-2 選取instance.....	186
圖A-5-3 選擇EC2所在的VPC.....	187
圖A-5-4 設定health checks.....	187
圖A-5-5 選擇instance為target.....	188
圖A-5-6 選ALB.....	188
圖A-5-7 填入基本資訊、選擇要公開 ALB.....	189

圖A-5-8 讓ALB同時部署在兩個AZ以上.....	189
圖A-5-9 在ALB身上掛 Security Group.....	190
圖A-5-10 Listener 設定.....	190
圖A-5-11 透過DNS對ALB發出請求.....	191
圖A-6-1 更新Name Server records成AWS hosted zone.....	192
圖A-6-2 在Route53裡管理DNS records.....	193
圖A-6-3 新增CNAME record.....	193
圖A-7-1 上ACM請求憑證.....	194
圖A-7-2 選擇公開憑證.....	194
圖A-7-3 輸入所有需要驗證的domain names.....	194
圖A-7-4 Validation method 選擇 DNS.....	195

表目錄

表2-2-1 Business Model Canvas.....	14
表2-4-1 競爭力分析.....	15
表2-4-2 SWOT分析.....	16
表3-2-1 系統架構.....	18
表3-3-1 使用標準與工具.....	19
表4-1-1 甘特圖.....	21
表4-2-1 專案組織與分工.....	22
表4-2-2 專案組織與分工.....	24
表9-1-1 主要檔案清單.....	118
表9-1-2 主要模組清單.....	119
表9-2-1 主要程式清單.....	122
表10-2-1 測試個案與結果.....	147
表11-1 系統安裝元件資訊表.....	148
表11-2 系統安裝說明表.....	148

第一章 前言

1-1 背景介紹

隨著現代人對健康與運動的重視日益提升，專業運動員與教練團隊也逐漸意識到「數據化管理」與「科技輔助訓練」的重要性。傳統的訓練紀錄多仰賴紙本或簡單口述方式，由於團隊人數眾多，容易遺漏選手資訊，導致教練在擬定訓練計畫與追蹤進展時效率較低。

本系統旨在提供一個數位化、系統化的平台，協助專業運動員紀錄每日訓練內容與身心狀況，並讓教練能即時掌握每位選手的狀態，進而進行個別化的調整與建議。

1-2 開發動機

本專案的開發者成員中有專業運動員，並發現運動員日常訓練中缺少了系統化的訓練紀錄及科技輔助，希望藉由這些系統與科技的幫助，提升行政作業以及訓練的效率。因此，希望透過這套系統改善目前紀錄與資訊無法整合資訊的問題，打造一個專業訓練管理平台，提升教練與選手之間的溝通效率，強化訓練的精準性與成效。

1-3 系統目的與目標

建立一個角色分明、操作簡便的「運動員訓練紀錄與評估系統」，提供運動員與教練一站式服務，減輕教練的行政工作，並完整、數據化的紀錄選手的專業技術訓練情況，方便教練與選手的追蹤與審視，並透過AI技術協助選手精進自身實力。

1-4 預期成果

系統預期能成功建構出一套穩定且具實用性的 Web 應用平台，確保不同使用者依權限進行操作與資料管理。

1. 使用者能每日輸入完整的專業及能訓練資料與評估紀錄，系統能儲存並展示這些數據。
2. 教練能夠透過系統後台檢視選手出缺席、訓練趨勢與異常，提升訓練計畫調整的科學性以及行政作業的便利性。
3. 運用AI訓練進行動作分析。
4. 透過此系統，提供更系統化的訓練，呈現直觀的訓練紀錄及成果。

第二章 營運計畫

2-1 可行性分析

市場可行性分析

- 市場現況與發展潛力

現今社會對於健康與運動的重視日益提高，各級學校設有桌球隊的比例也逐年攀升，顯示此領域有穩定且具發展性的基礎市場。加上數位學習、遠距訓練與科技教學的推廣，使得教練與選手對於「訓練數據化」、「訓練可視化」的需求愈發明確。

- 潛在目標族群明確

以桌球選手為主軸，涵蓋從國小至大學體系之學生選手與教練群，此外亦可拓展至運動社團、體育班使用。此類使用者對於紀錄自身訓練數據、追蹤表現變化及獲得教練回饋有實際需求。

2-2 商業模式分析

表2-2-1 Business Model Canvas

關鍵夥伴 (Key Partner)	關鍵活動 (Key Activity)	價值主張 (Value Proposition)	客戶關係 (Customer Relation)	客戶區隔 (Customer Segment)
學校 運動團隊	- 功能開發 - 使用者資料處理 - 教練端與選手端 - 資料庫應用	- 操作簡便 - 功能完整、集中紀錄與可視化 - 滿足教練與選手對訓練追蹤的需求 - 方便學校行政處理	- 教學影片 - 討論版 - 使用功能	- 桌球隊選手（國小～大學） - 體育班學生 - 教練、助教 - 有訓練紀錄需求
	核心資源 (Key Resource) 開發團隊與運動		通路 (Channel) 學校合作	

	相關知識 • 架設平台主機與後台資料庫 • UI/UX 設計資源		推廣 • 系統網站 Web App	之一般運動者
成本結構(Cost Structure) • 人力開發成本 • 雲端伺服器租用與維護 • 系統更新維護費			收入來源(Revenue Stream) • 教育授權收入 • 開發專案收費 • 月費 / 年費訂閱制 • AI 分析模組收入	

2-3 市場分析－STP

● Segmentation 市場區隔

1. Targeting 目標市場選擇

年齡層：國小至大學運動員 身分別：桌球隊選手、體育班學生、教練

2. 裝置偏好：瀏覽器 Web 使用者

● Targeting 目標市場選擇

鎖定已有組隊訓練需求與紀錄習慣的學校團隊與教練單位。

包含國小、國中、高中及大學桌球隊，具穩定使用頻率與統一管理需求。

● Positioning 市場定位

將系統定位為「專為桌球與學校訓練單位設計」的數據化紀錄平台。

以 LINE 與網頁操作為主要使用模式，簡易上手、角色清楚，並導入AI支援訓練，便於全面導入學校訓練流程。

2-4 競爭力分析

表2-4-1 競爭力分析

優勢 Strengths	劣勢 Weaknesses
● 開發者成員中有運動員，能精準掌握使用需求與操作習慣。 ● 系統功能涵蓋完整 ● 使用者介面簡潔直觀，容易上手，便於實際應用。 ● 支援角色分權，強化系統管理彈性。	● 這是一個互動性的系統，教練與選手都需定期上傳資料與評估內容。 ● 若使用者（教練或選手）配合度不足，可能導致數據不完整，影響分析準確性。
機會 Opportunities	威脅 Threats
● 市面已有運動紀錄或健身 App，競爭激烈。 ● 若無針對性、獨特性功能，系統可能相似度高，使用率下降。 ● 資料安全與隱私保障將是使用者導入時的顧慮。	● 無明顯技術差異時，難以突顯產品優勢。 ● 使用者對於健身 App 或教練系統已有既定習慣與偏好。 ● 無法快速建立信任或實績案例，會導致使用意願降低。

表2-4-2 SWOT 分析

OT/SW	有利因素(S)	不利因素(W)
外部機會(O)	SO 策略 ● 強調系統為運動訓練專屬平台，符合訓練流程需求，直接切入教練與學生情境。 ● 將紀錄轉為現代化圖表，成為學校或團隊訓練回饋的最佳工具。	WO 策略 ● 初期聚焦核心功能穩定性，逐步推廣並累積校際實例強化可信度。 ● 建立教練端操作範例與成果展示，提升使用誘因。
外部威脅(T)	ST 策略 ● 持續強化針對訓練紀錄的核心價值，提升資料分析深度與精準性，區隔競爭門檻。	WT 策略 ● 發展「針對性」與「專業性」的技術定位，避免與大眾健身類 App 混淆，凸顯系統特色。 ● 採漸進式開發模式，優先強化使用穩定與便捷性，再拓展功能模組

第三章 系統規格

3-1 系統架構：

本系統採用三層式架構，將使用者操作介面（Browser）、應用邏輯層（Flask 應用伺服器）與資料儲存層（MySQL 資料庫）分離，有助於系統維護與擴充。使用者透過網頁瀏覽器進行操作，傳送 HTTP 請求至 Flask 應用伺服器，由伺服器處理邏輯並存取 MySQL 資料庫後回傳結果。

AI擴充模型則以使用者操作介面(Streamlit)結合預訓練動作預測模型(Vjepa2+Random Forest分類器)加上物件偵測模型(Yolov8)組成，使用者透過網頁瀏覽器進行操作，將影片轉換成數據並分析，給予回饋。

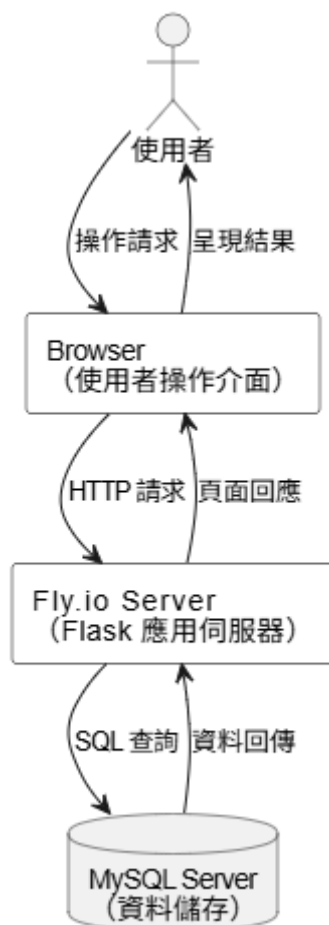


圖3-1-1系統架構

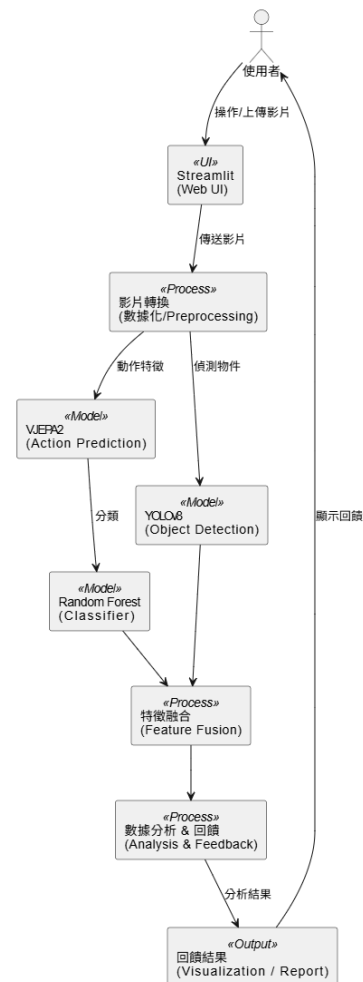


圖3-1-2系統架構

資料流程如下：

1. 使用者在瀏覽器中操作介面。
2. 瀏覽器發出 HTTP 請求至 Flask 應用伺服器。
3. 應用伺服器與 MySQL 資料庫進行讀寫操作。
4. 資料處理完成後將結果回傳至前端。
5. 瀏覽器呈現資訊供使用者查看。

3-2 系統軟、硬體需求與技術平台

本系統可在任何具備瀏覽器之裝置上操作，包含桌上型電腦、筆記型電腦、平板與手機。

表3-2-1 系統架構

項目	內容
作業系統	Windows11
開發平台	Flask (Python Framework)
資料庫系統	MySQL
前端技術	HTML,CSS,JavaScript,Bootstrap,Streamlit
後端技術	Python、Flask
AI模組	VJEPa2+Random Forest,YOLOv8、MediaPipe,OpenCV

伺服器平台	EC2
雲端服務	AWS
記憶體需求	建議至少8GB RAM
網路環境	需可連線 Wi-Fi / 4G / 5G 網路

3-3 使用標準與工具

表3-3-1 使用標準與工具

工具類別	使用工具
作業系統	Windows 11
前端開發	HTML, CSS, JavaScript, Bootstrap, Streamlit
後端開發	Python, Flask
資料庫	MySQL
動作分類	VJEPa(特徵提取) + Random Forest(分類)

物件偵測	YOLOv8
姿勢分析	MediaPipe + OpenCV
IDE / 編輯器	Visual Studio Code
版本管理	Git, GitHub
視覺化繪圖	draw.io, Chart.js
簡報與文件	Microsoft Word, Canva
溝通工具	Discord, LINE

第四章 專案時程與組織分工

4-1 專案時程：甘特圖

表4-1-1 甘特圖

預估時間



實際時間



年	114年										
月	2	3	4	5	6	7	8	9	10	11	12
技術學習											
題目構想											
後端框架 編寫											
資料庫建 置											
前端開發											
分析模型 訓練											
系統測試											
系統整合											
文件製作											
簡報製作											

4-2 專案組織與分工

表4-2-1 專案組織與分工

●主要負責人 ○次要負責人

項目/組員		11146021 曾順暉	11146027 高偉倫	11146035 張禎益	11146036 陳頤謙
前端開發	選手頁面		○		●
	訓練狀況		●		
	選手/教練個人資料編輯		●		
	登入/註冊介面設計				●
	登入頁面			●	
	點名系統	●			
	訓練菜單				●
	睡眠紀錄			●	
	壓力評估		○	●	
	智能助手		●		
	飲食紀錄	●			
	LINE Bot	●			
	動作分析模型				●
後端開發	每日訓練狀況		●		
	選手頁面		○		●
	登入頁面			●	
	資料庫建置		○		●
	登入註冊功能		○		●
	教練主控台		●		

	選手/教練個人資料		●	○	
	公告管理		●		
	點名系統	●			
	管理員主控台		●		
	訓練狀況		●		○
	訓練菜單				●
	使用者管理		●		
	睡眠紀錄		●	○	
	壓力評估			●	
	智能助手		●		
	飲食紀錄	●			
	LINE Bot	●			
	動作分析模型				●
美術設計	web介面設計		○		●
	色彩設計		○		●
	logo設計			●	
	UI/UX		●		○
文件撰寫	統整	○		●	○
	第一章 前言				●
	第二章 營運計畫				●
	第三章 系統規格				●
	第四章 專題時程與組織分工			●	
	第五章 需求模型	○	○	●	
	第六章 設計模型	○		●	○
	第七章 實作模型	○	○	●	○
	第八章 資料庫設計	●	○		
	第九章 程式	○	○		●
	第十章 測試模型	●		○	○
	第十一章 操作手冊	○		○	●
	第十二章 使用手冊	○		○	●
報告	簡報製作		○	○	●

表4-2-2 專案組織與分工

序號	姓名	工作內容	貢獻度
1	組長 <u>張禎益</u>	前端:登入頁面、睡眠紀錄、壓力評估 後端:個人資料、睡眠紀錄、壓力評估 美術:logo設計 文件:手冊四~七章及九~十二章、手冊統整、系統簡介、競賽報名、競賽文件、簡報製作 其他:組內行政	<u>19%</u>
2	組員 <u>高偉倫</u>	前端:選手頁面、個人資料、壓力評估、智能助手 後端:資料庫、登入註冊、教練主控台、個人資料、公告、管理員主控台、使用者管理、睡眠紀錄、智能助手 美術:web、色彩、UI/UX 文件:手冊五及七~九章、競賽文件、簡報製作	<u>27%</u>
3	組員 <u>陳頤謙</u>	前端:選手頁面、各式表單、登入註冊介面、動作分析模型 後端:資料庫、登入註冊、訓練紀錄、訓練狀況、今日訓練紀錄、動作分析模組 美術:web、色彩、UI/UX 文件:手冊一~三章及九~十二章、手冊統整、簡報製作、競賽文件、影片剪輯	<u>35%</u>
4	組員 <u>曾順暉</u>	前端:點名系統、飲食紀錄、LINE Bot 後端:點名系統、飲食紀錄、LINE Bot 文件:手冊八~十二章、手冊統整、競賽文件、簡報製作	<u>19%</u>
			<u>總計:100%</u>

4-3 上傳GitHub紀錄

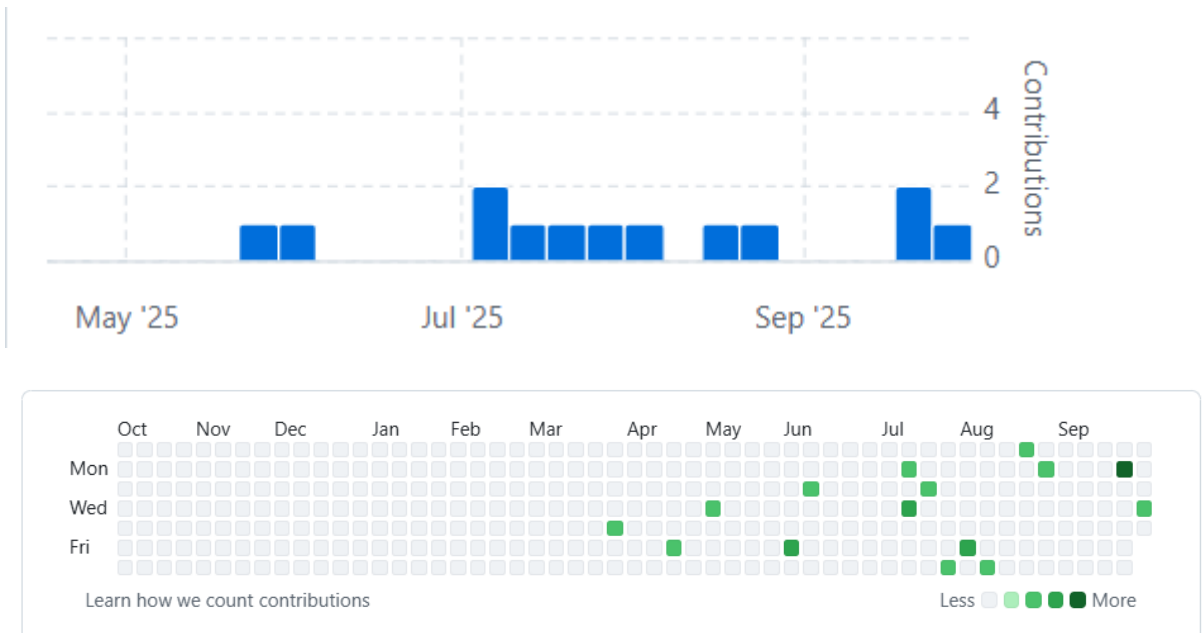


圖4-3-1-11146021 曾順暉上傳github紀錄

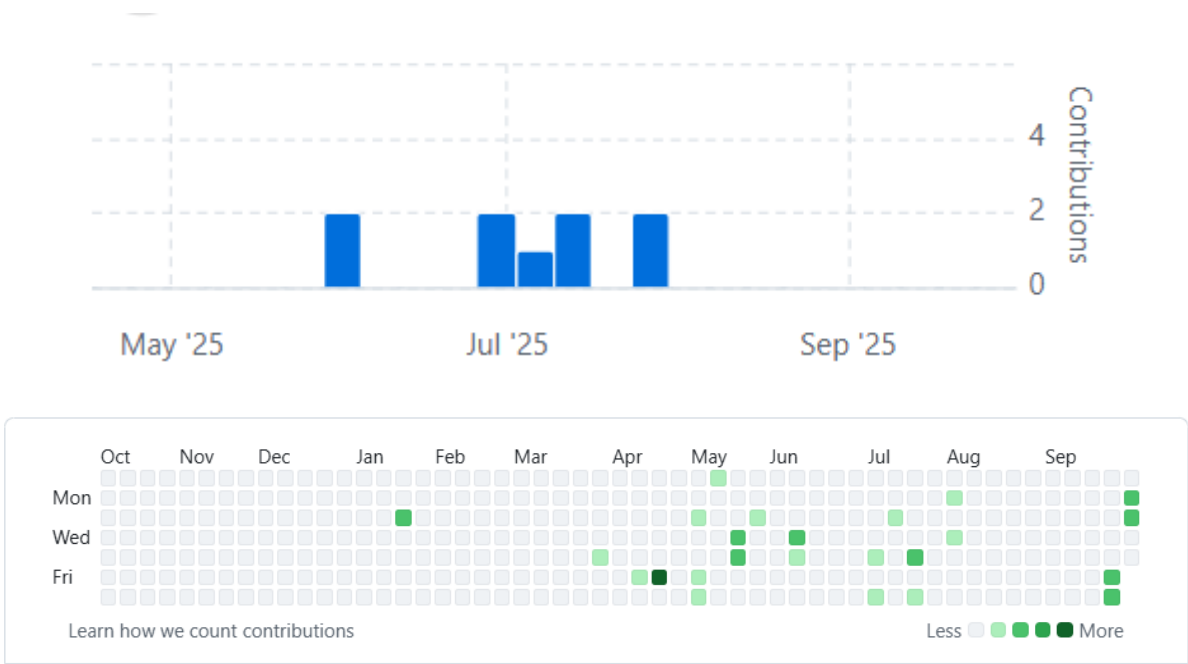


圖4-3-2-11146035 張禎益上傳github紀錄

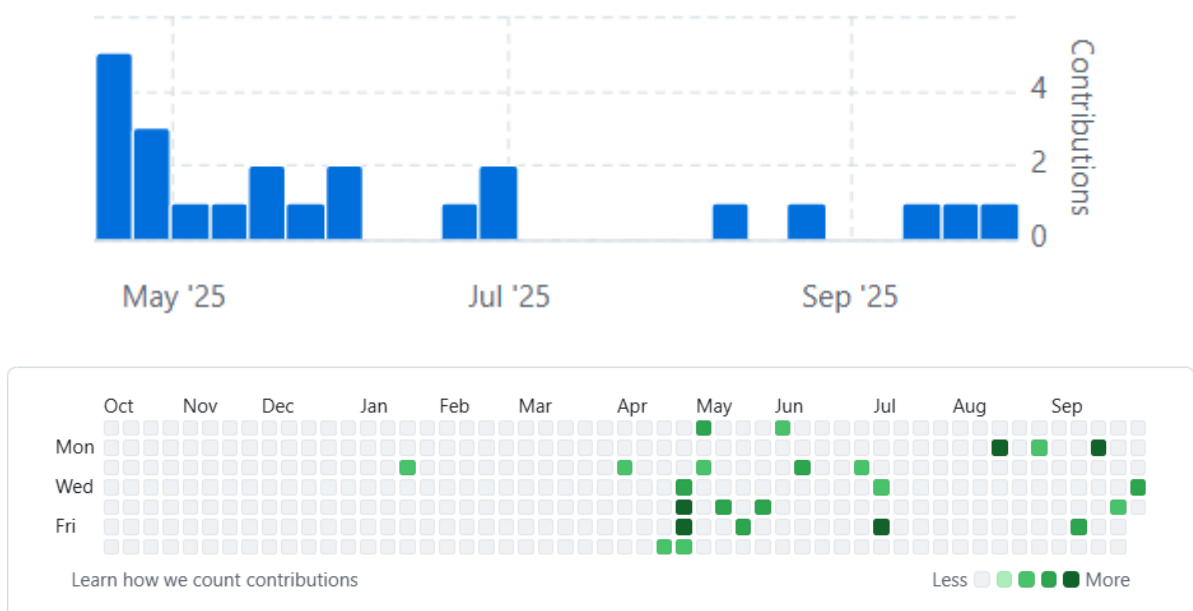


圖4-3-3-11146027 高偉倫上傳github紀錄

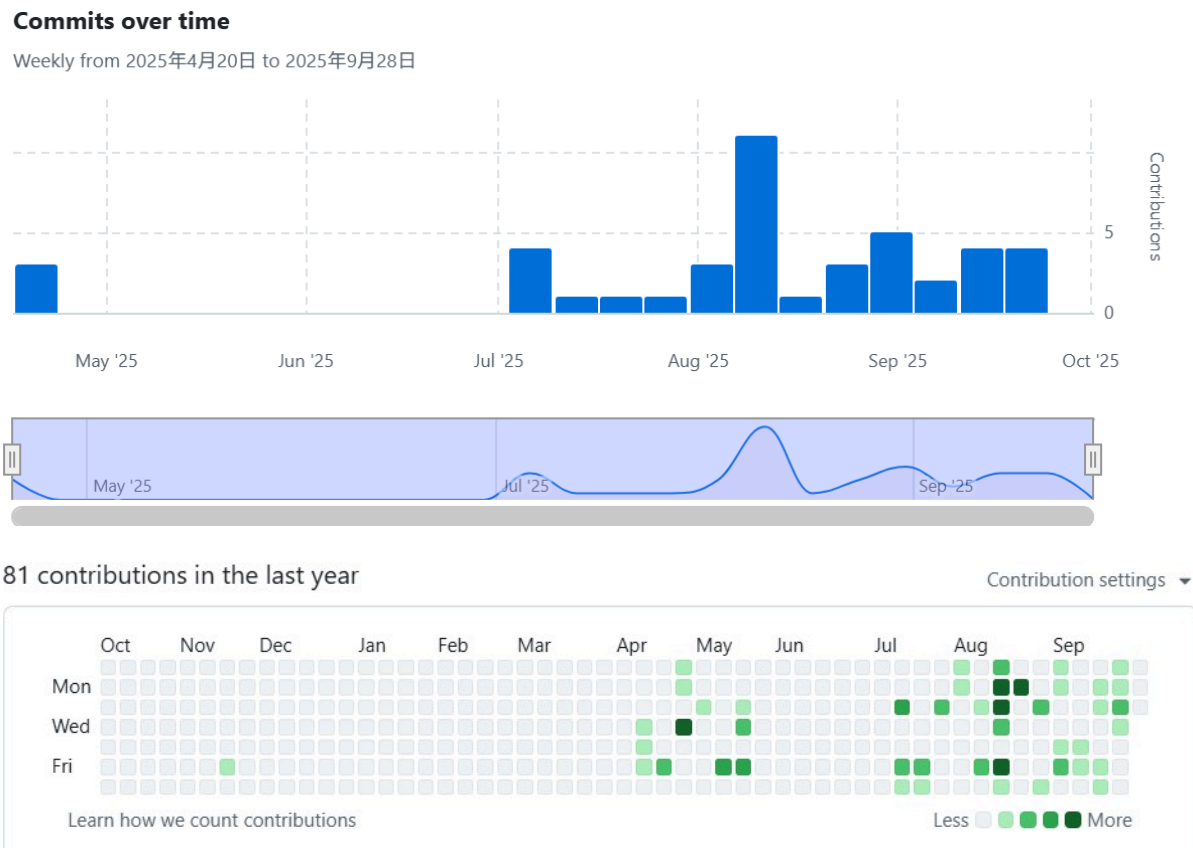


圖4-3-4-11146036 陳頤謙上傳github紀錄

第五章 需求模型

5-1 使用者需求：需求清單及其說明

一、使用者角色與權限

1. 學生端（選手）

- 訓練: AI動作辨識、AI聊天機器人。
- 紀錄: 查看公告、出席點名確認、訓練計畫執行、訓練狀況紀錄、心理量表填寫、飲食狀況填寫、睡眠紀錄填寫、LineBot紀錄。

2. 教練端：新增公告、出席點名紀錄、派發訓練計畫、查看訓練狀況、查看心理量表、查看飲食狀況、查看睡眠紀錄、LineBot紀錄。

3. 管理員端：使用者帳號管理與系統參數設定。

二、功能性需求

✓ 學生端功能

1. 帳號與個人資料管理

- 登入 / 登出支援使用者帳號管理。
- 個人資料管理：可填寫與修改個人資訊欄位，包括：姓名、性別、身高、體重、照片等。

2. 查看訓練菜單

- 日曆元件：可選擇任一日期，查看當日訓練紀錄。
- 教練派發訓練內容：訓練內容分為「體能訓練」與「技術訓練」。
- 完成確認：選手可勾選已完成的訓練項目，系統自動統計並顯示完成度百分比。

3. 訓練狀況評估

- 每日需填寫的欄位包括：身體疲勞、心理壓力、睡眠品質、自我評估
- 資料管理：評估紀錄可編輯或刪除。
- 視覺化：系統以折線圖顯示歷史紀錄，追蹤成長與變化軌跡。

4. 公告查看

- 即時查看教練發布的公告。
- 可依日期、類別（重要、提醒、閒聊）分類。

5. 出席點名確認

- 提供每日訓練課程的出缺席確認(出席、缺席、請假)。

6. 訓練計畫執行紀錄

- 查看教練排定的訓練計畫。
- 訓練項目完成後可勾選紀錄。
- 系統自動計算完成率，並以顯示於訓練總覽頁面

7. 心理壓力量表

- 提供心理狀態問卷（賽後壓力指數）。
- 選手可賽後填寫並提交。
- 紀錄整合至訓練狀況分析模組。

8. 飲食紀錄

- 上傳每日飲食照片或文字描述。
查詢教練回饋。

9. 睡眠紀錄

- 填寫每日睡眠時數與睡眠品質。

10. 動作分析模組姿勢分析：

- 可分析選手正手與反手動作姿勢。
- 影片上傳分析：選手上傳訓練影片，系統結合 AI 模型（VJEPa2 + RF、YOLOv8）進行動作辨識。
- 回饋建議：系統提供專業訓練建議與修正方向。

11. 智能助手AI聊天機器人：

- 選手可透過文字輸入，詢問桌球相關問題。
- 即時回應：AI 智能助手即時回答並提供參考資料。

12. LINE Bot整合

- 行動便利性：透過 LINE 介面即可操作，無需進入網頁。
- 主要功能：上傳飲食資料
- 查詢公告：查看今日訓練、查詢出缺席狀況

✓ 教練端功能

1. 帳號與個人資料管理

- 登入 / 登出支援使用者帳號管理。
- 個人資料管理：可填寫與修改個人資訊欄位，包括：姓名、性別、身高、體重、照片等。

2. 發布訓練菜單

- 日曆元件：可選擇任一日期，指定當日訓練菜單。
- 派發訓練內容：訓練內容分為「體能訓練」與「技術訓練」。
- 顯示選手完成度長條圖

3. 公告發布

- 即時發布的公告。
- 可依日期、類別（重要、提醒、閒聊）分類。

4. 查看訓練狀況

- 查看選手訓練狀況

5. 出席點名確認

- 紀錄選手每日訓練課程的出缺席（出席、缺席、請假）。

6. 查看心理壓力量表

- 查看選手填寫之心理壓力狀況

7. 飲食紀錄

- 查看選手之上傳之每日飲食照片。
- 教練提供回饋。

8. 睡眠紀錄

- 查看選手上傳之每日睡眠時數與睡眠品質。

✓ 管理員端功能

1. 使用者帳號管理

- 新增、刪除帳號。
- 管理整體系統使用者資訊、權限調整。

2. 系統設定與參數維護

- 系統維護

5-2 使用個案圖(Use case diagram)

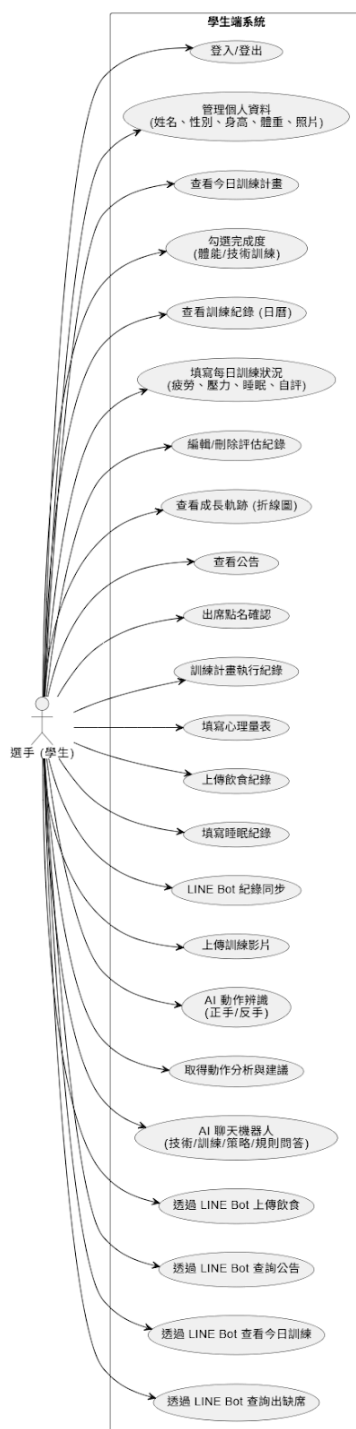


圖5-2-1

使用個案圖-Use case diagram-選手個案圖

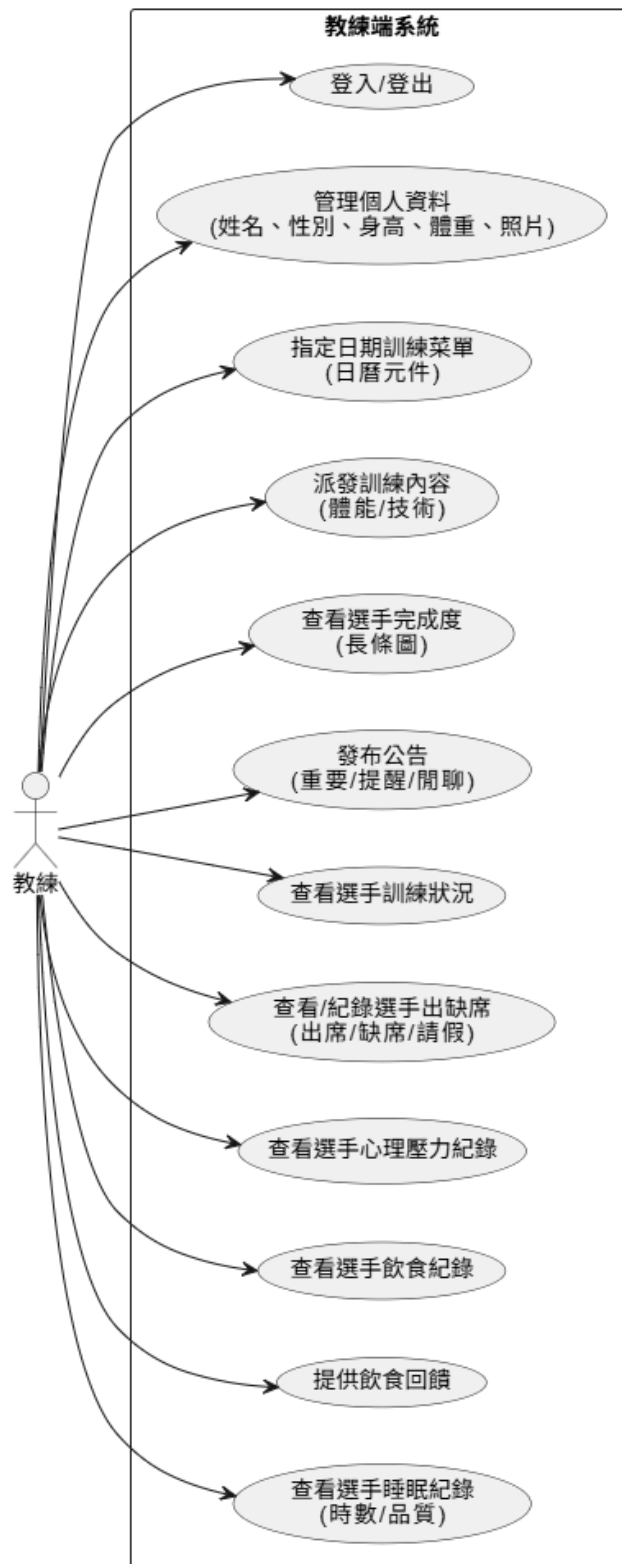


圖5-2-2

使用個案圖-Use case diagram- 教練個案圖

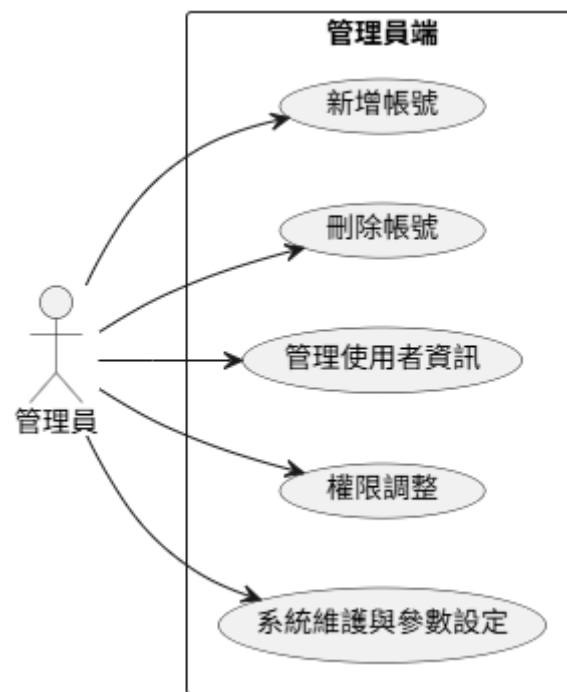


圖5-2-3

使用個案圖-Use case diagram- 管理者個案圖

5-3 使用個案描述：活動圖(Activity diagram)

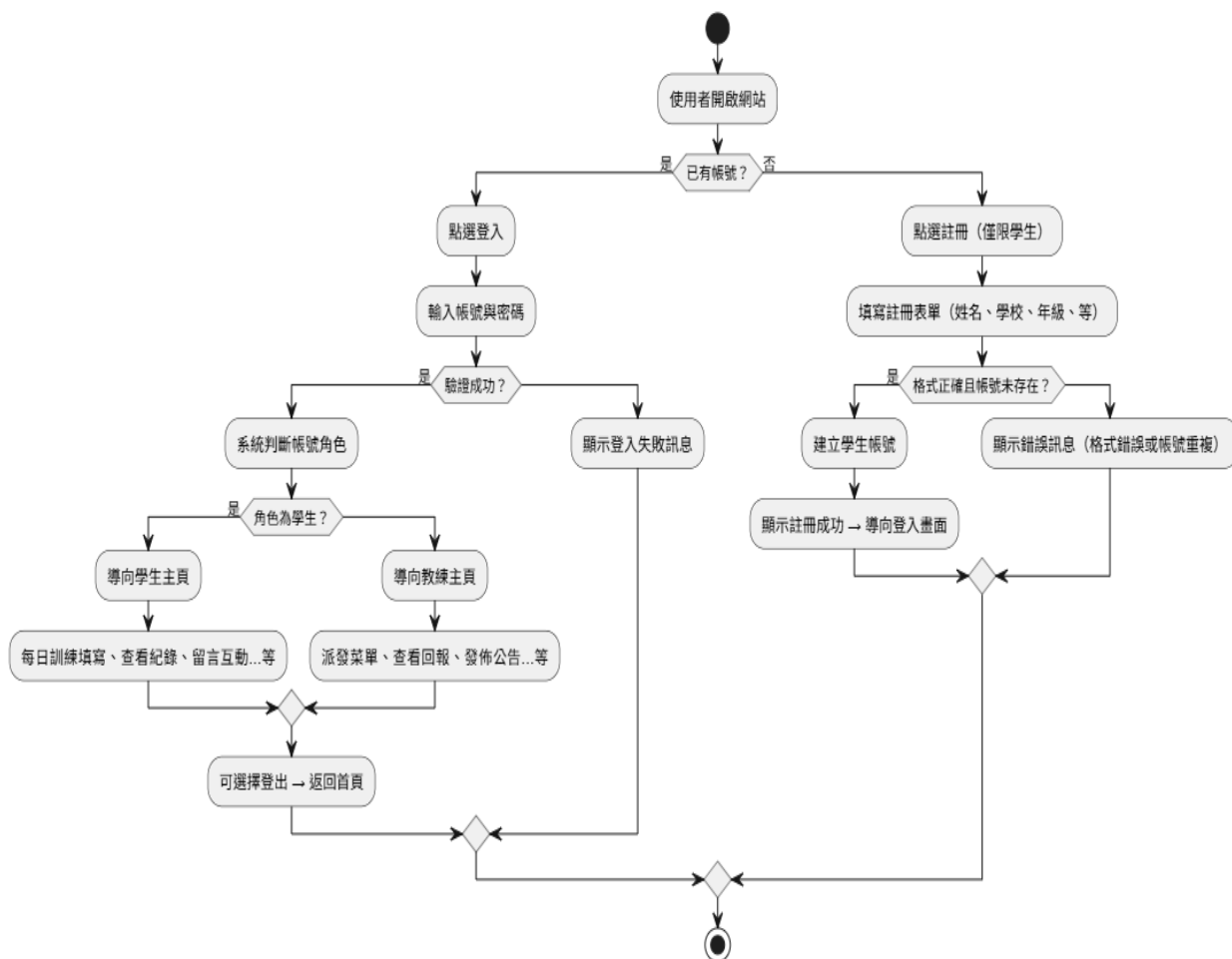


圖5-3-1

使用活動圖-Activity diagram- 選手註冊登入、教練登入

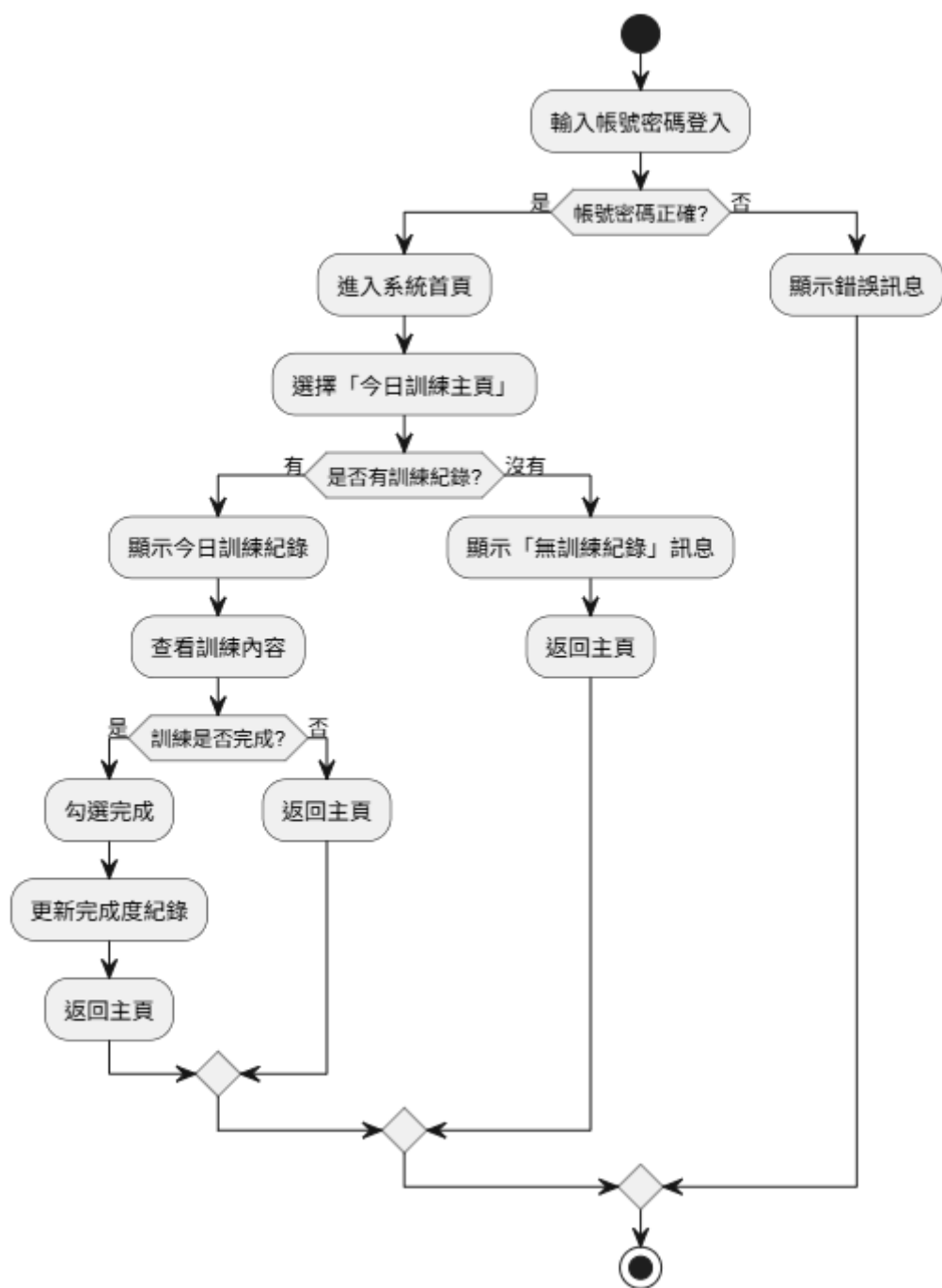


圖5-3-2

使用活動圖-Activity diagram- 選手勾查看及選訓練紀錄

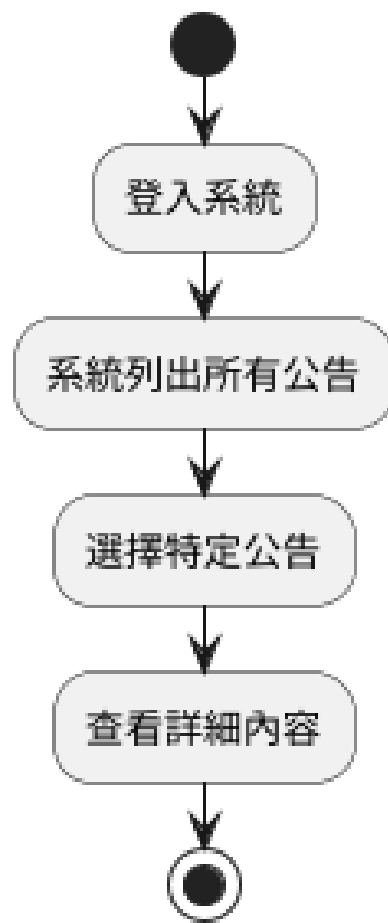


圖5-3-3

使用活動圖-Activity diagram- 選手查看公告

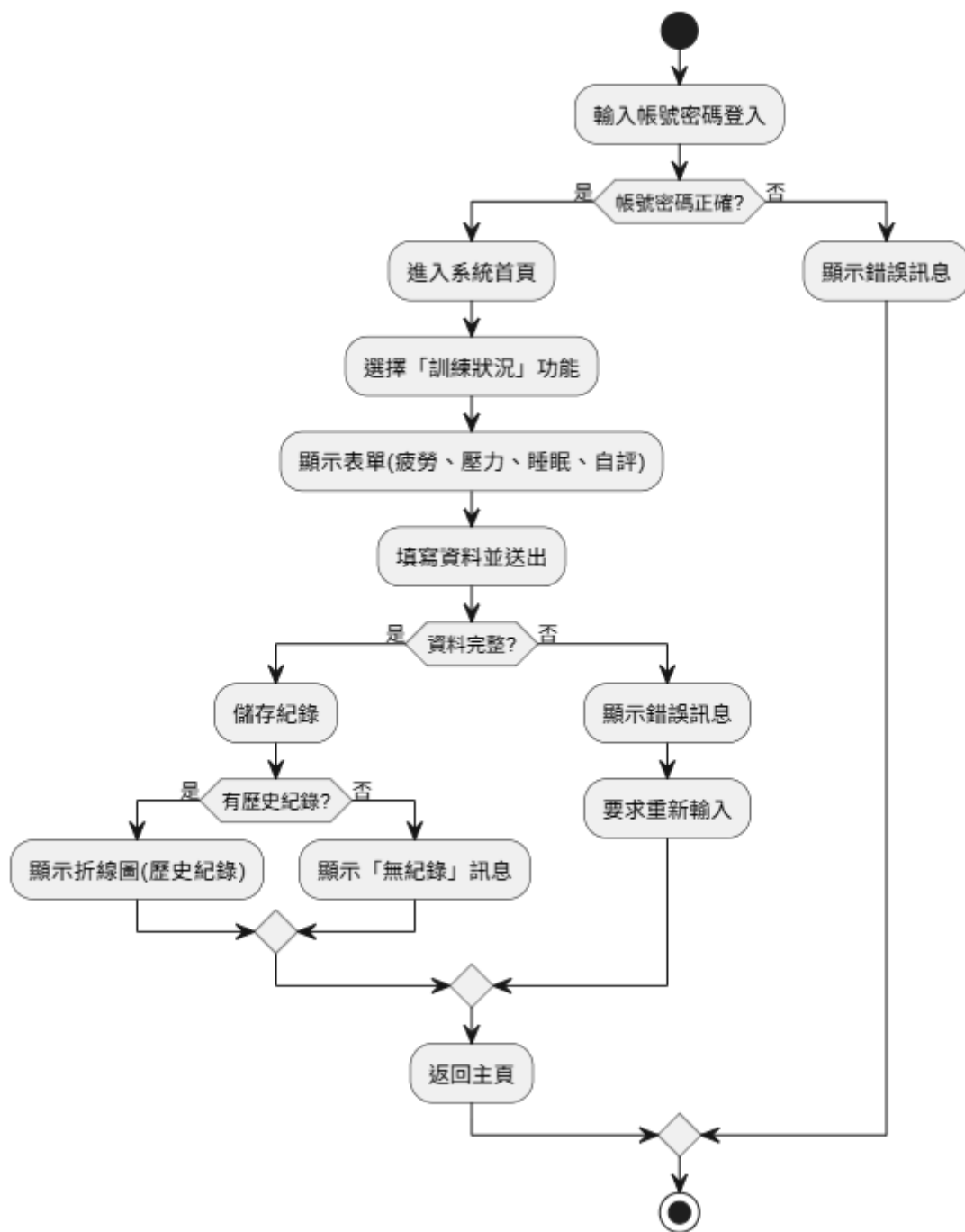


圖5-3-4

使用活動圖-Activity diagram- 選手填寫訓練狀況

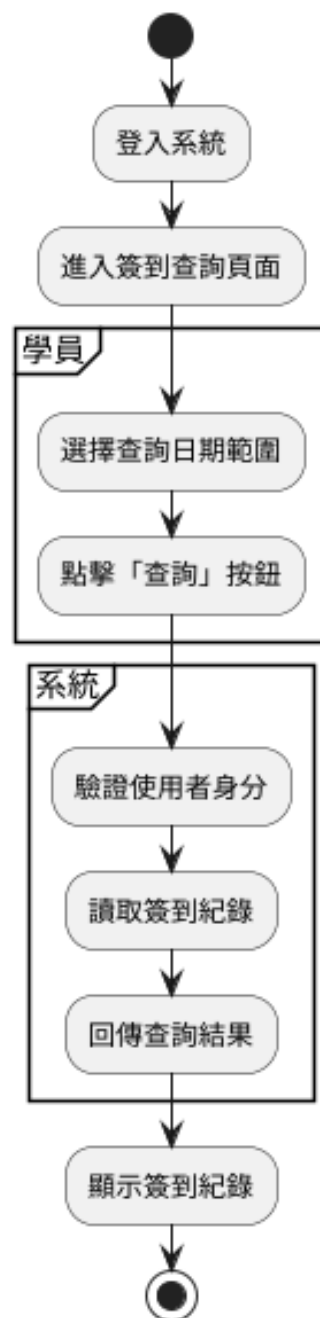


圖5-3-5

使用活動圖 -Activity diagram - 選手查看簽到紀錄

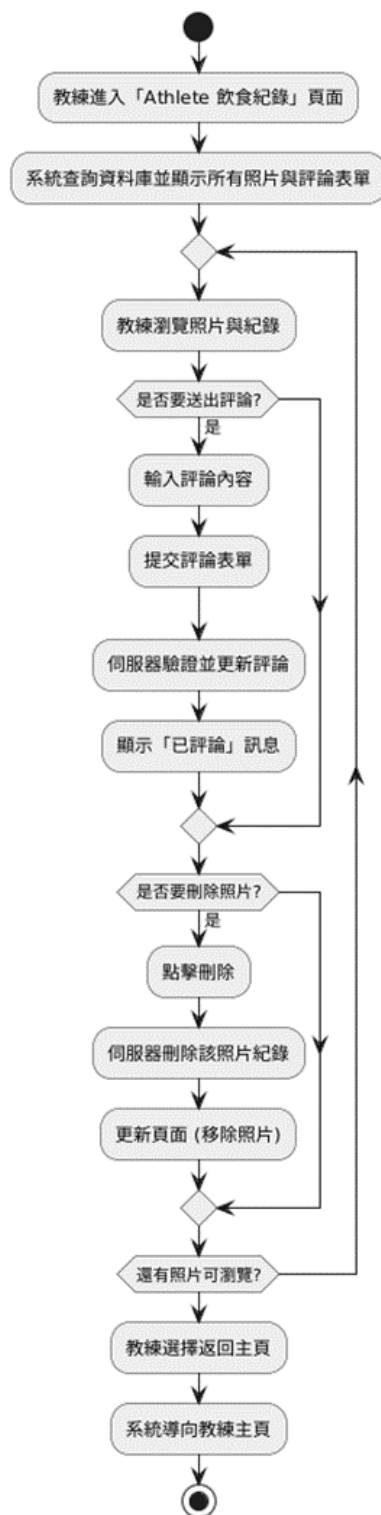


圖5-3-6

使用活動圖 -Activity diagram - 選手上傳飲食照片

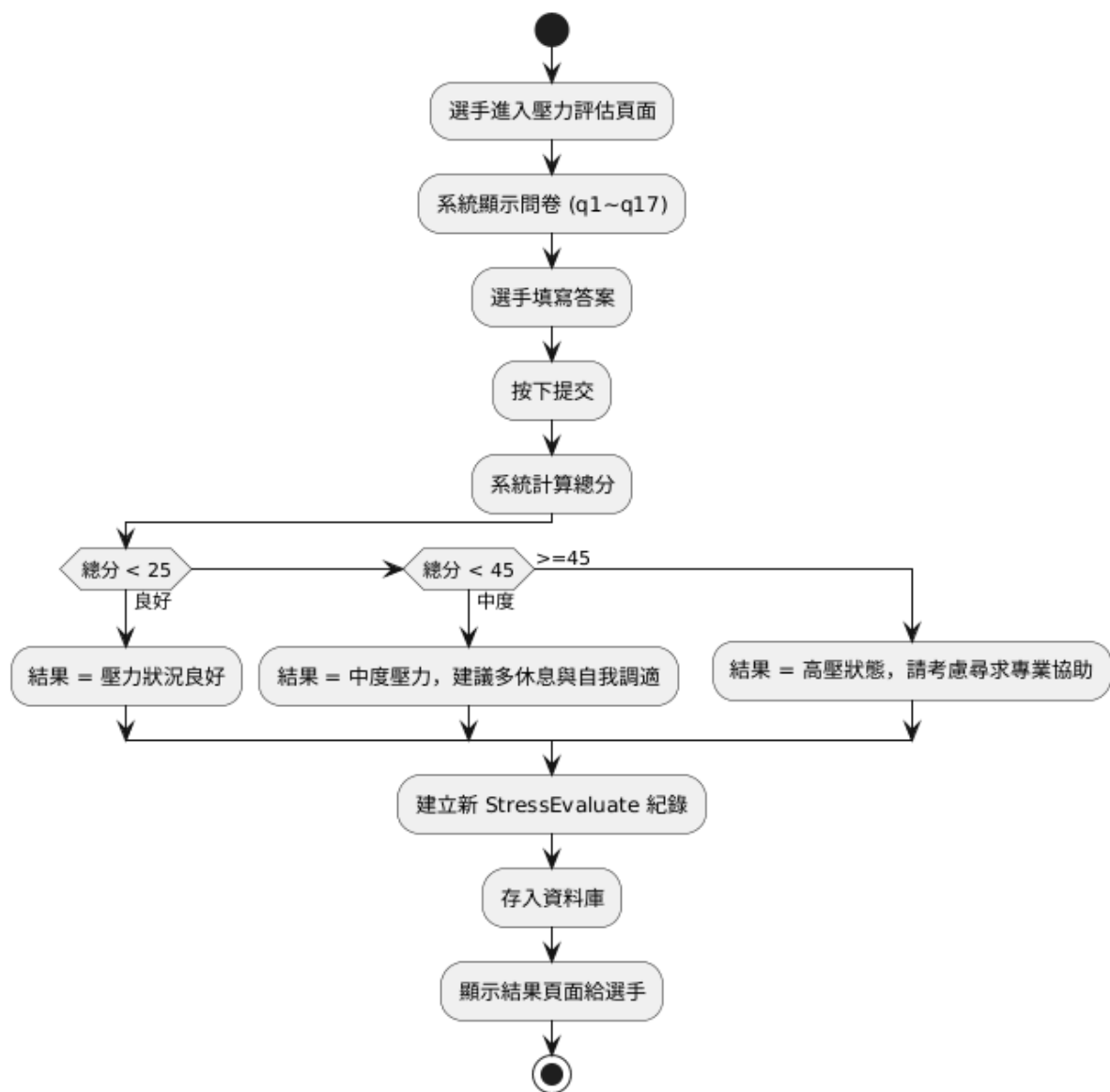


圖5-3-7

使用活動圖 -Activity diagram - 選手填寫壓力評估

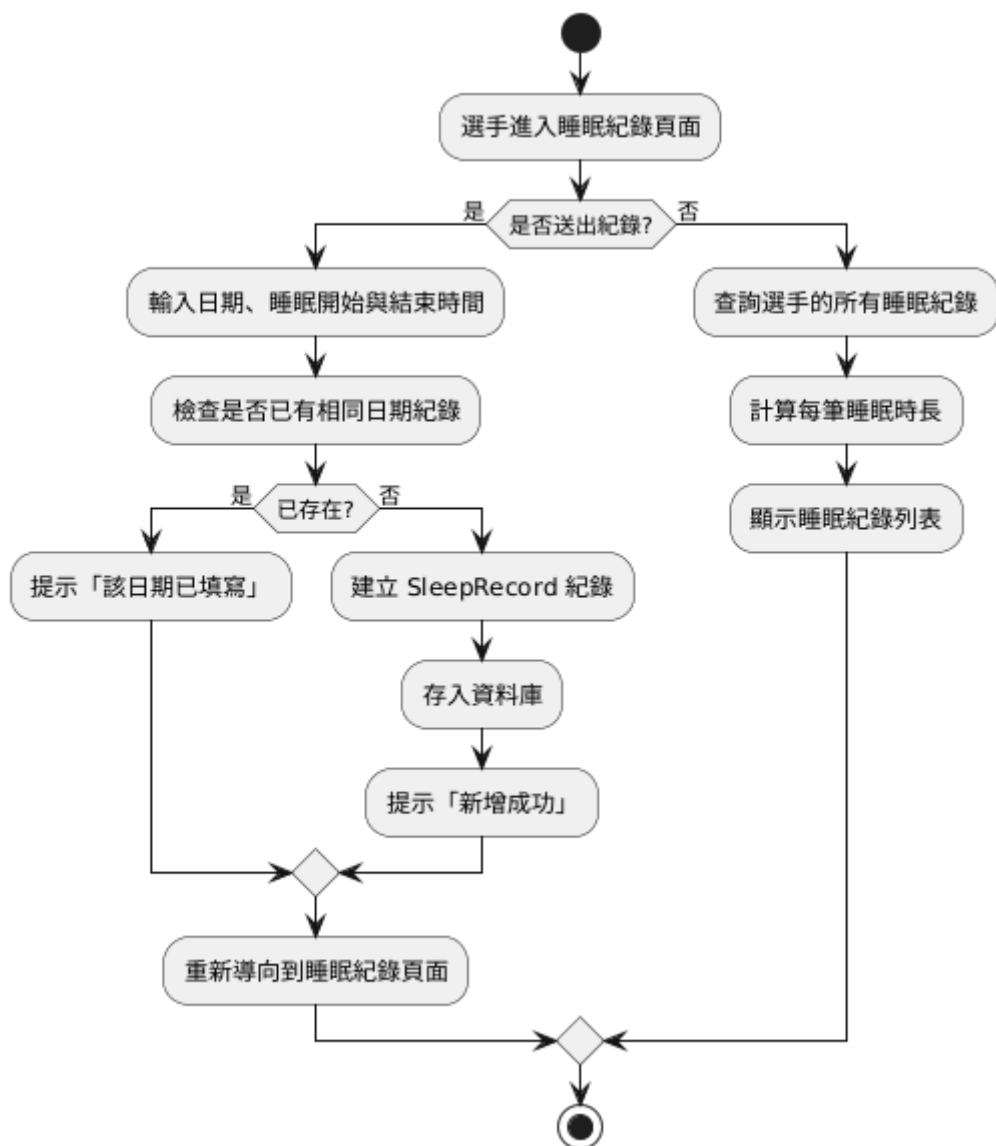


圖5-3-8
使用活動圖 -Activity diagram - 選手填寫睡眠紀錄

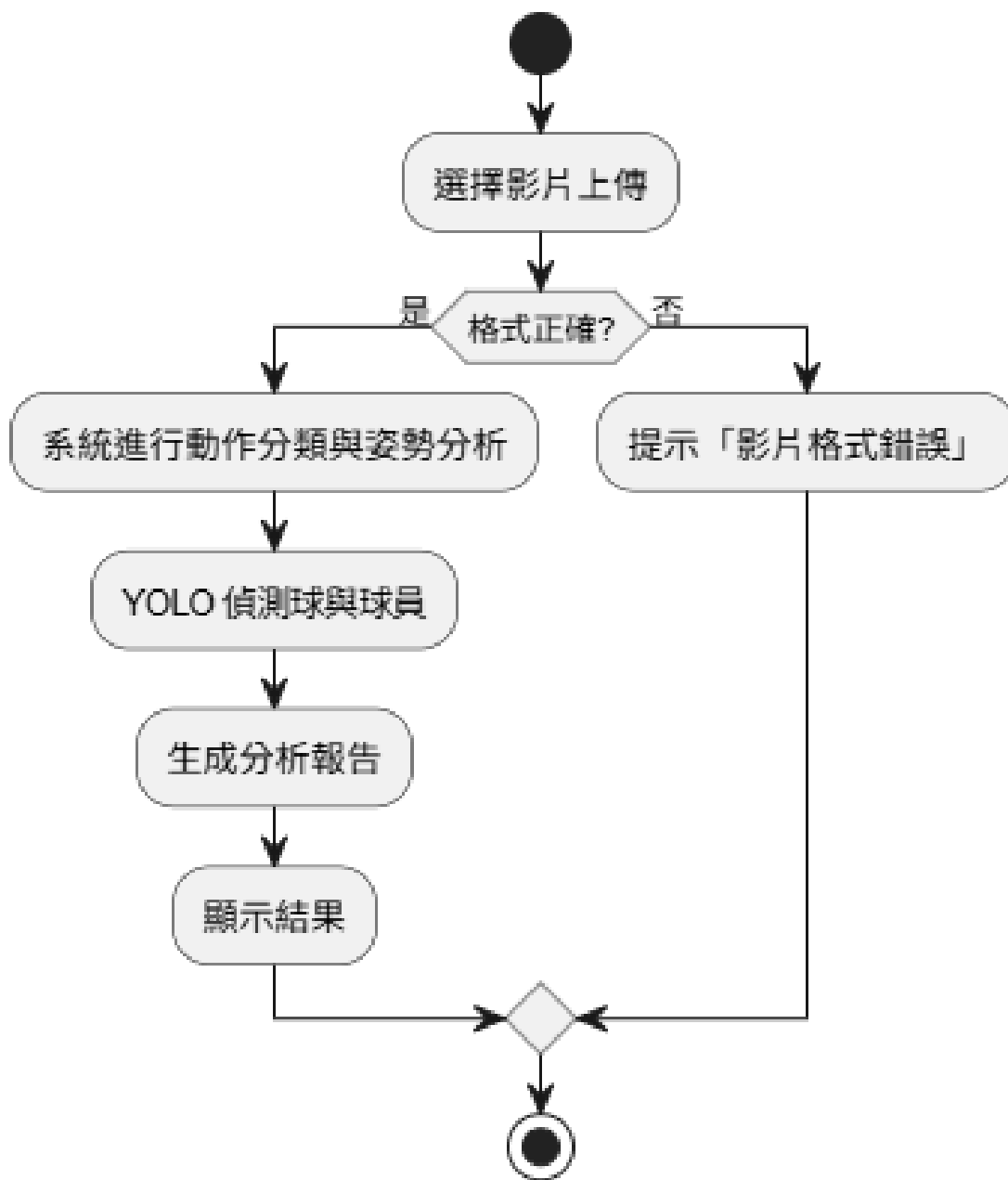


圖5-3-9
使用活動圖 -Activity diagram - 選手動作分析模組

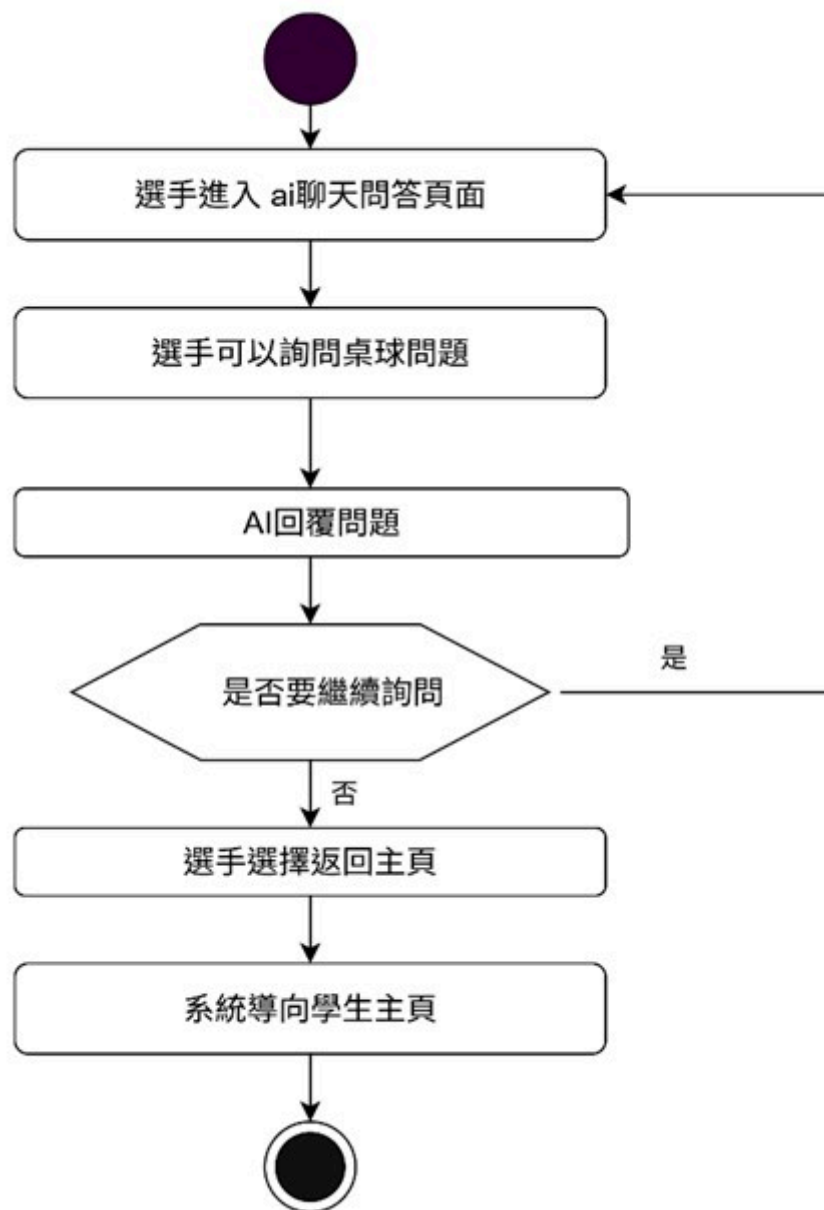


圖5-3-10
使用活動圖 -Activity diagram - 選手使用智能助手

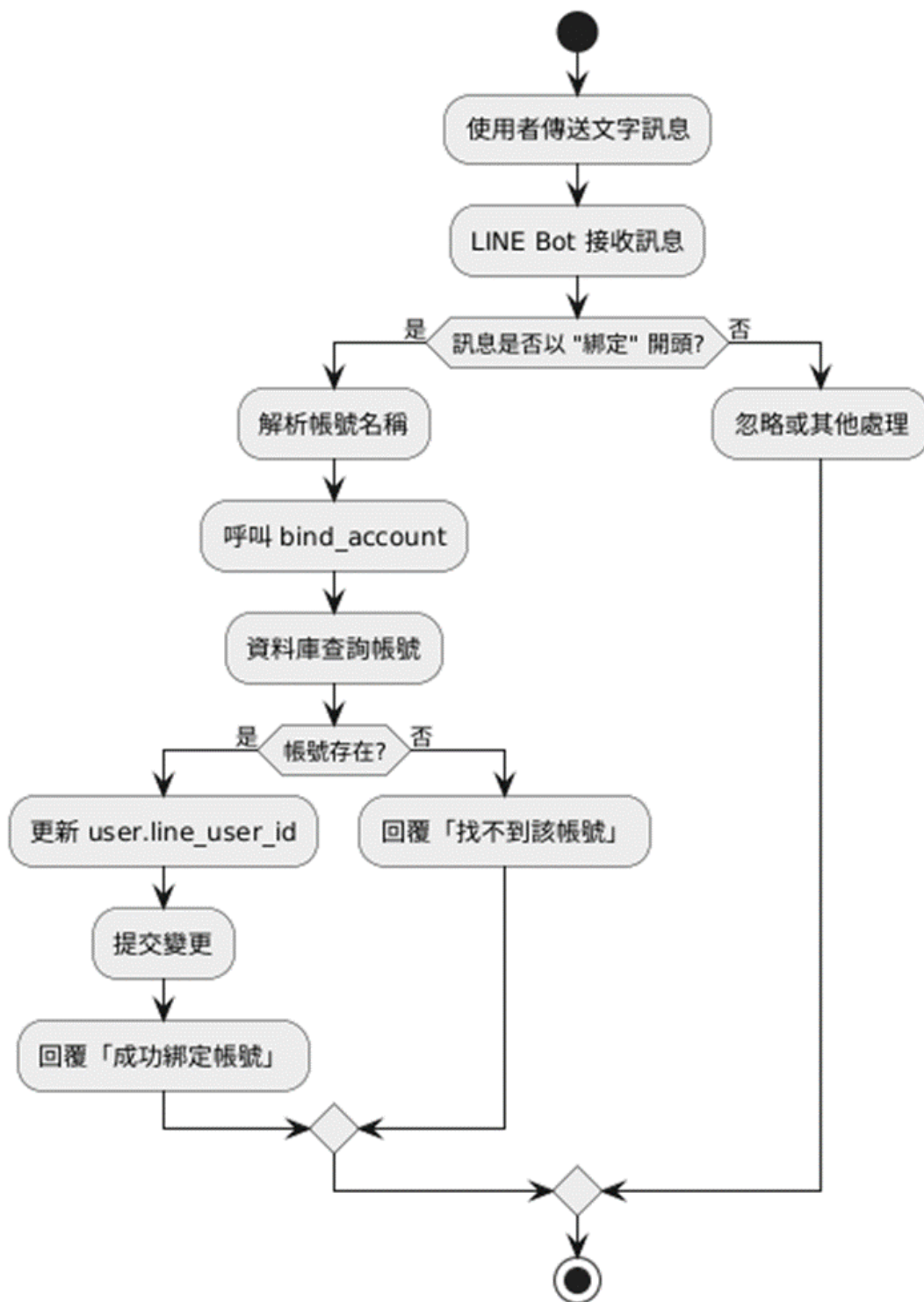


圖5-3-11

使用活動圖 -Activity diagram - 選手LINE Bot綁定帳號

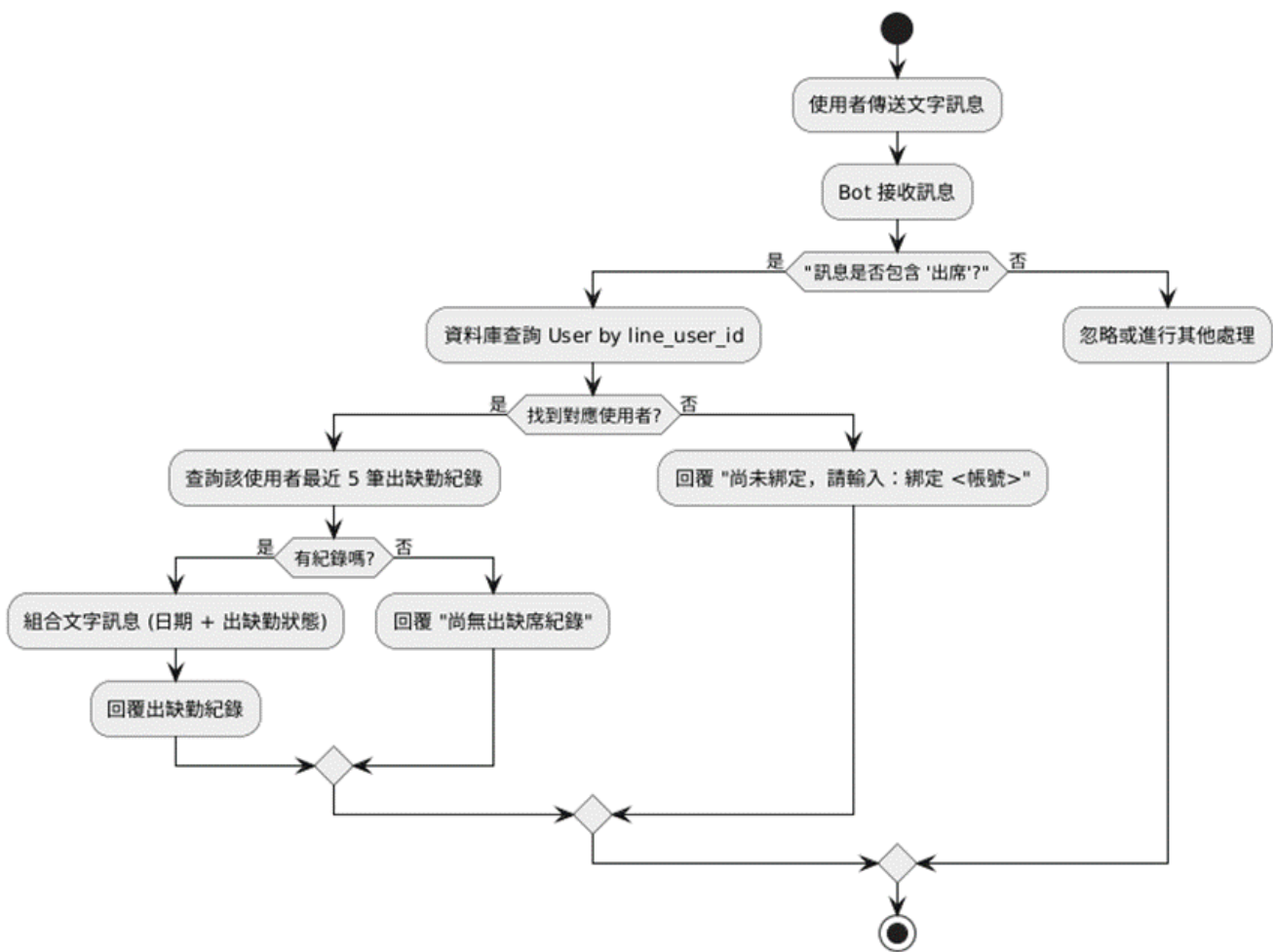


圖5-3-12

使用活動圖 -Activity diagram - 選手LINE Bot查詢出缺席

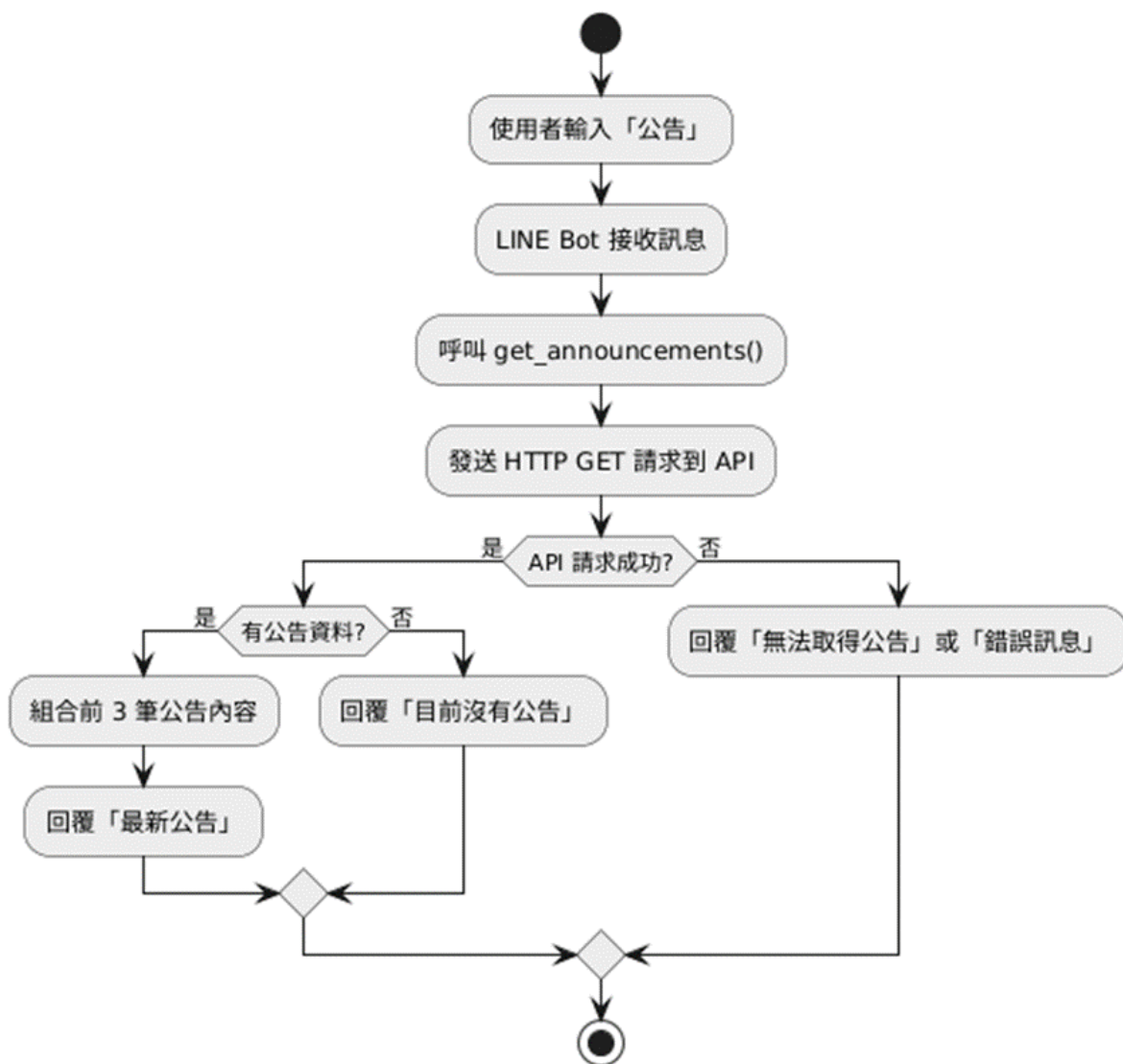


圖5-3-13

使用活動圖 -Activity diagram - 選手LINE Bot查詢公告

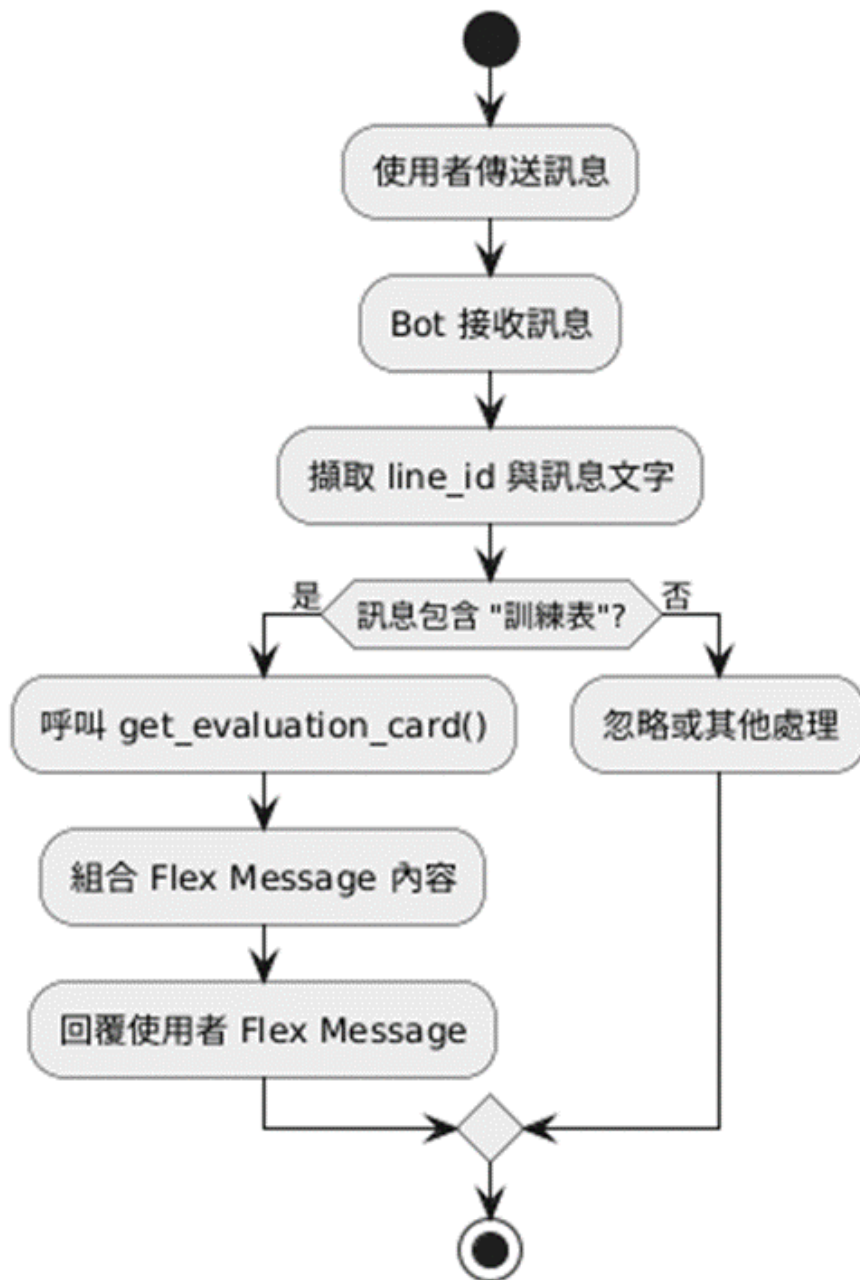


圖5-3-14
使用活動圖 -Activity diagram - 選手LINE Bot今日訓練狀況

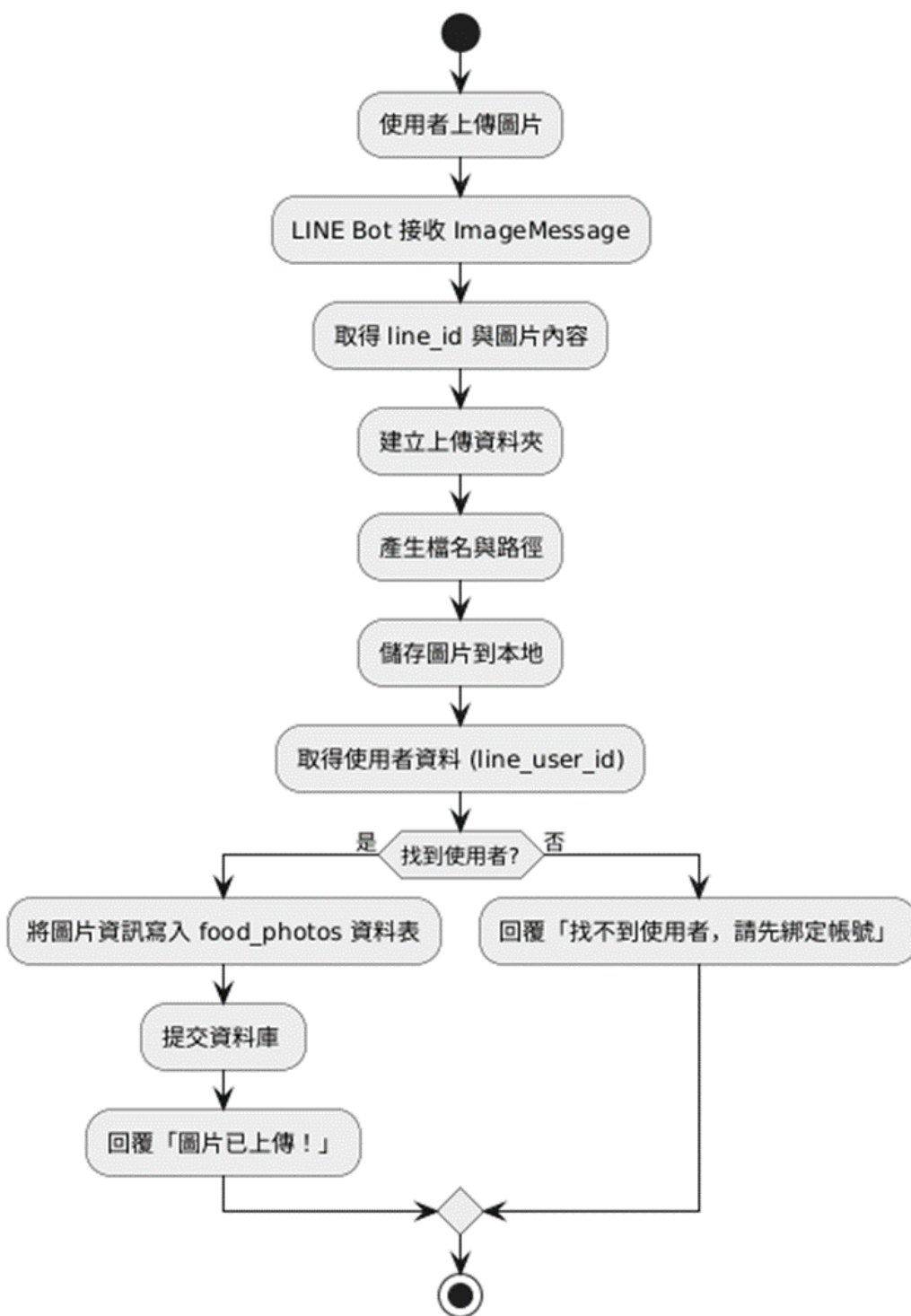


圖5-3-15

使用活動圖 -Activity diagram - 選手LINE Bot上傳飲食照片

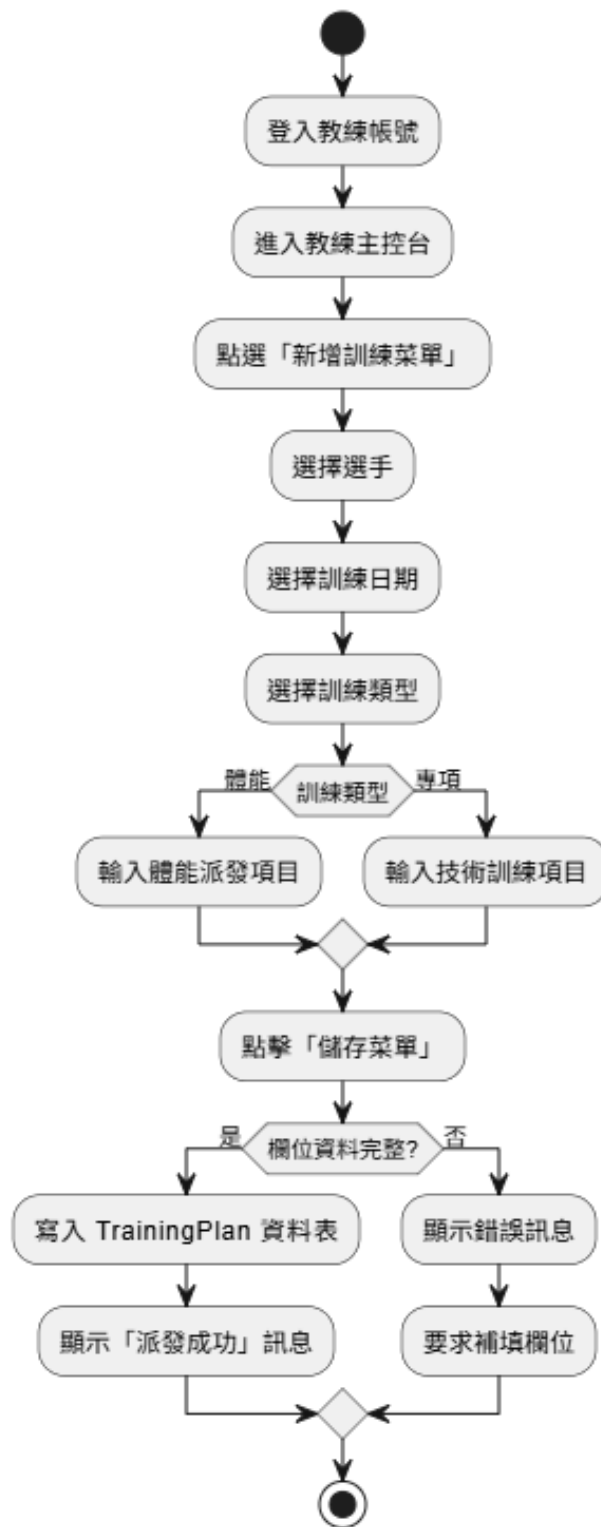


圖5-3-16

使用活動圖-Activity diagram- 教練建立訓練菜單

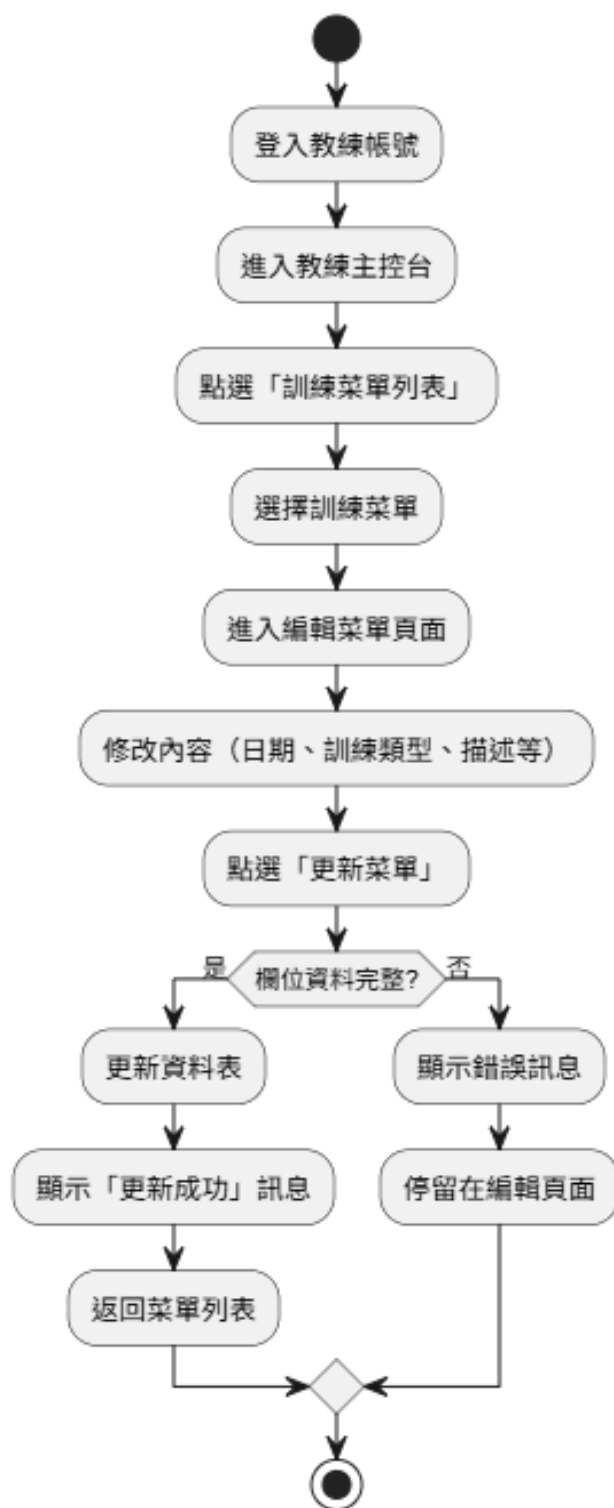


圖5-3-17

使用活動圖-Activity diagram- 教練編輯訓練菜單

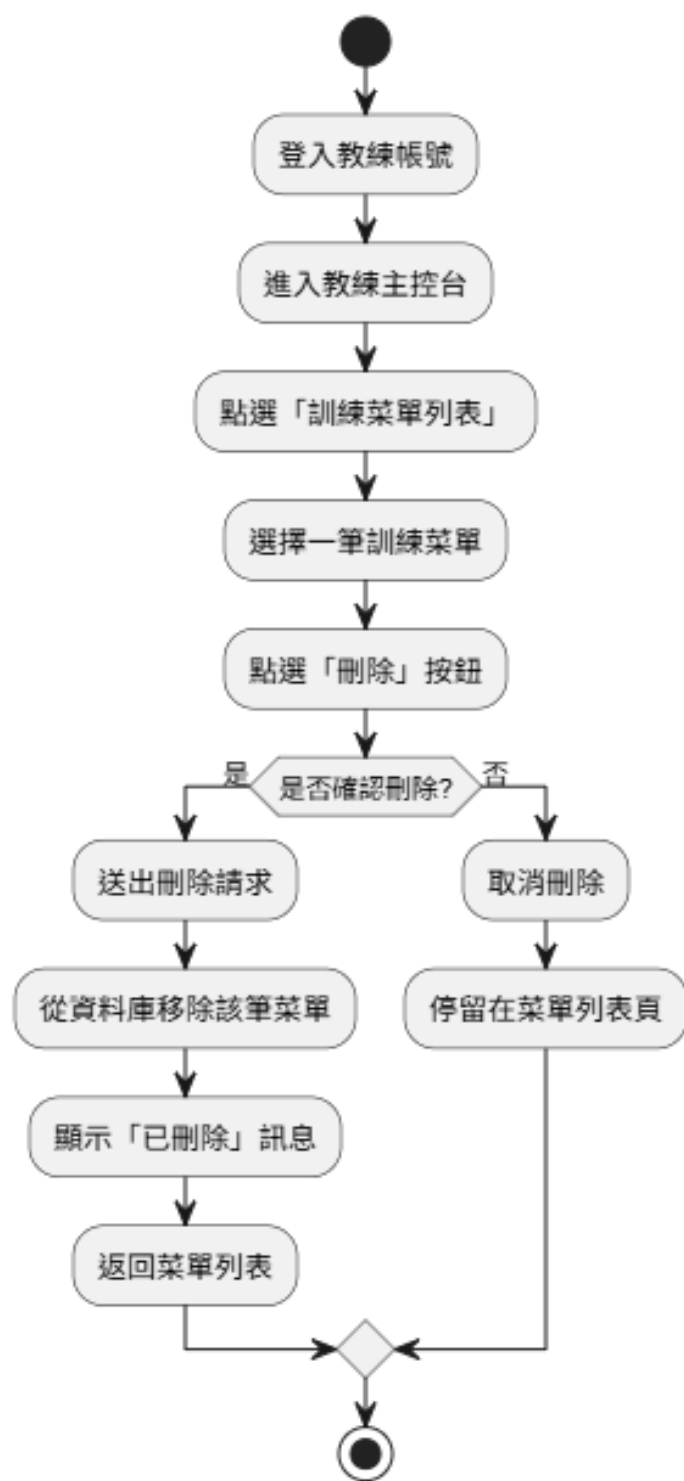


圖5-3-18

使用活動圖-Activity diagram- 教練刪除訓練菜單

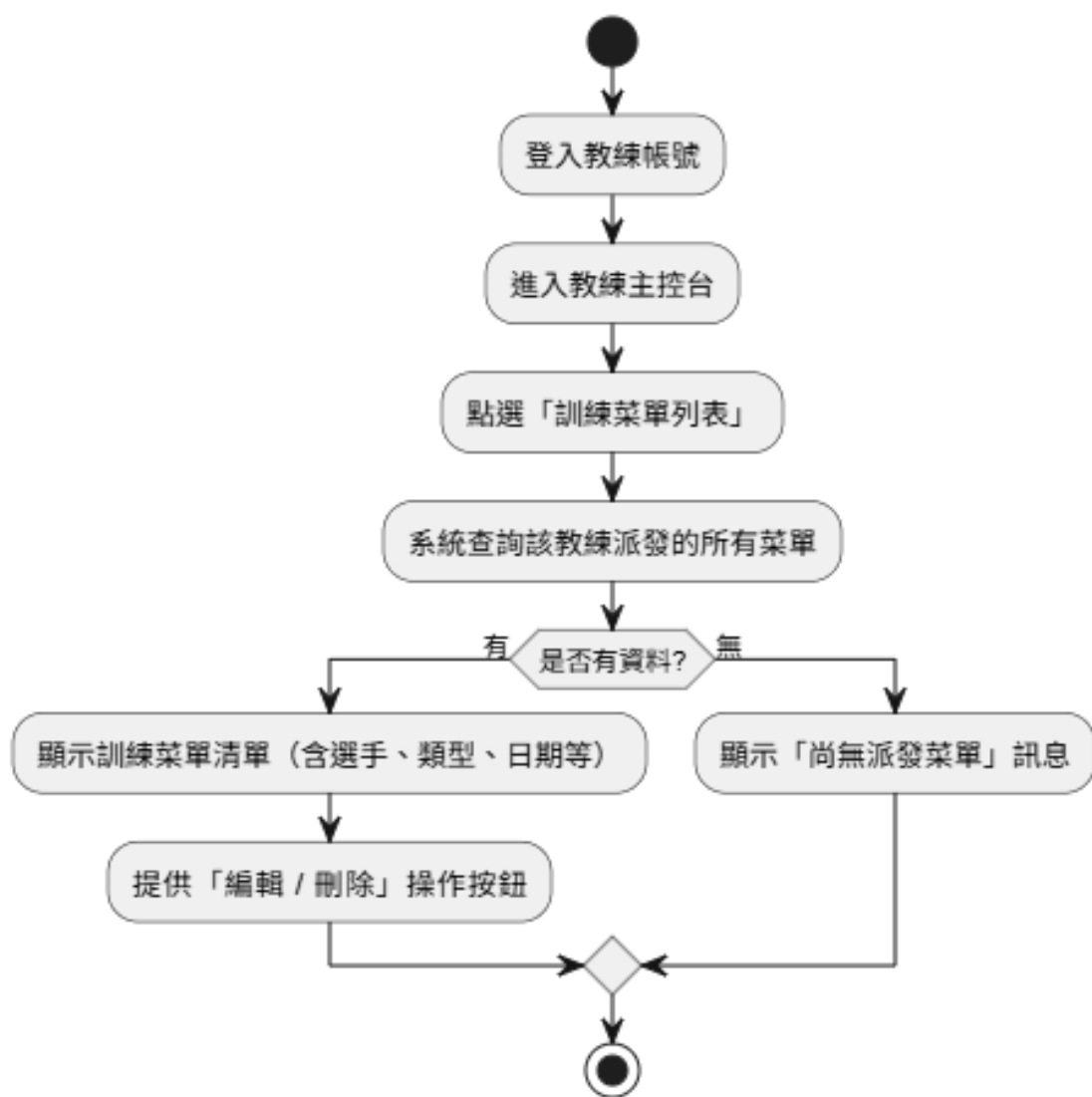


圖5-3-19

使用活動圖-Activity diagram- 教練查看已派發訓練菜單

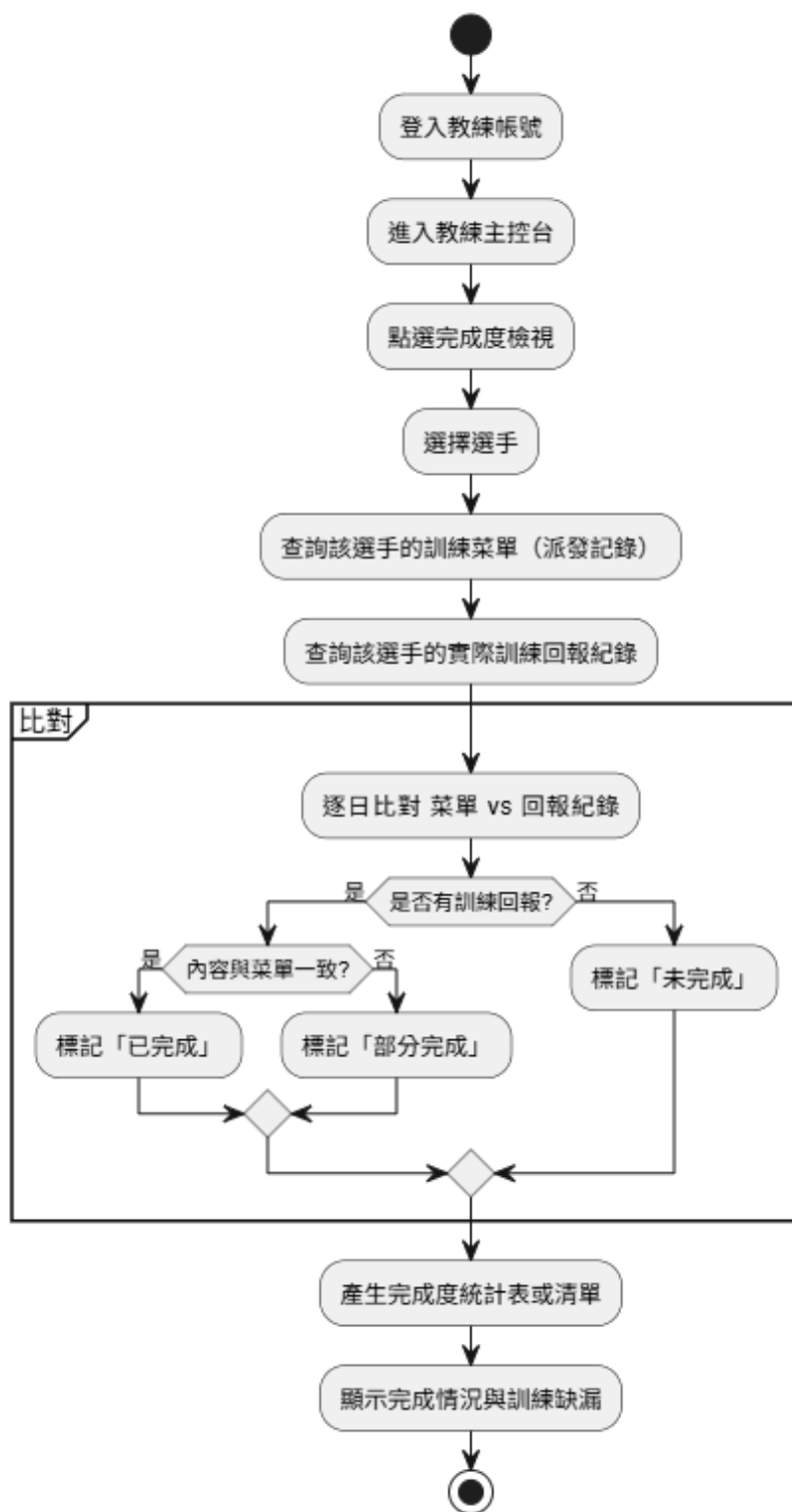


圖5-3-20

使用活動圖-Activity diagram- 教練查看選手完成度

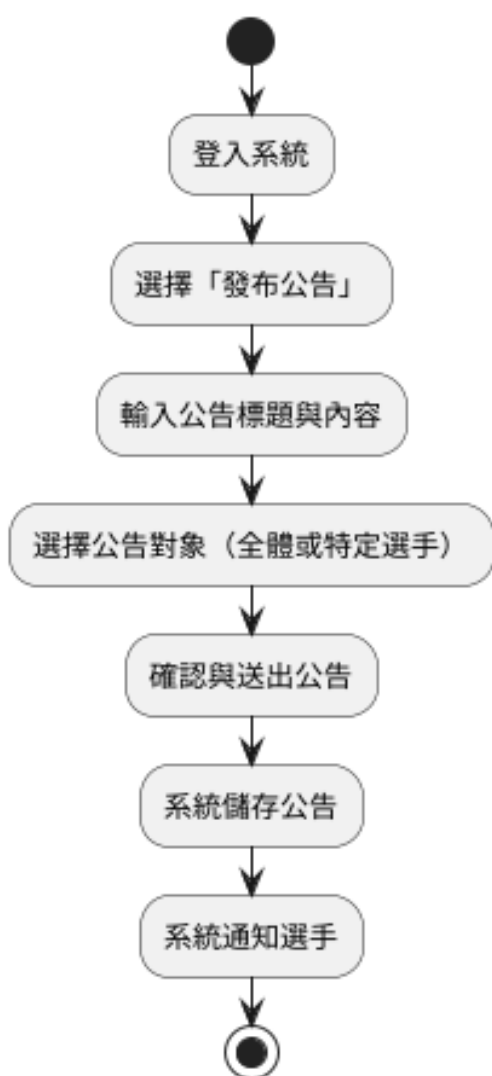


圖5-3-21使用活動圖-Activity diagram
- 教練發布公告活動圖

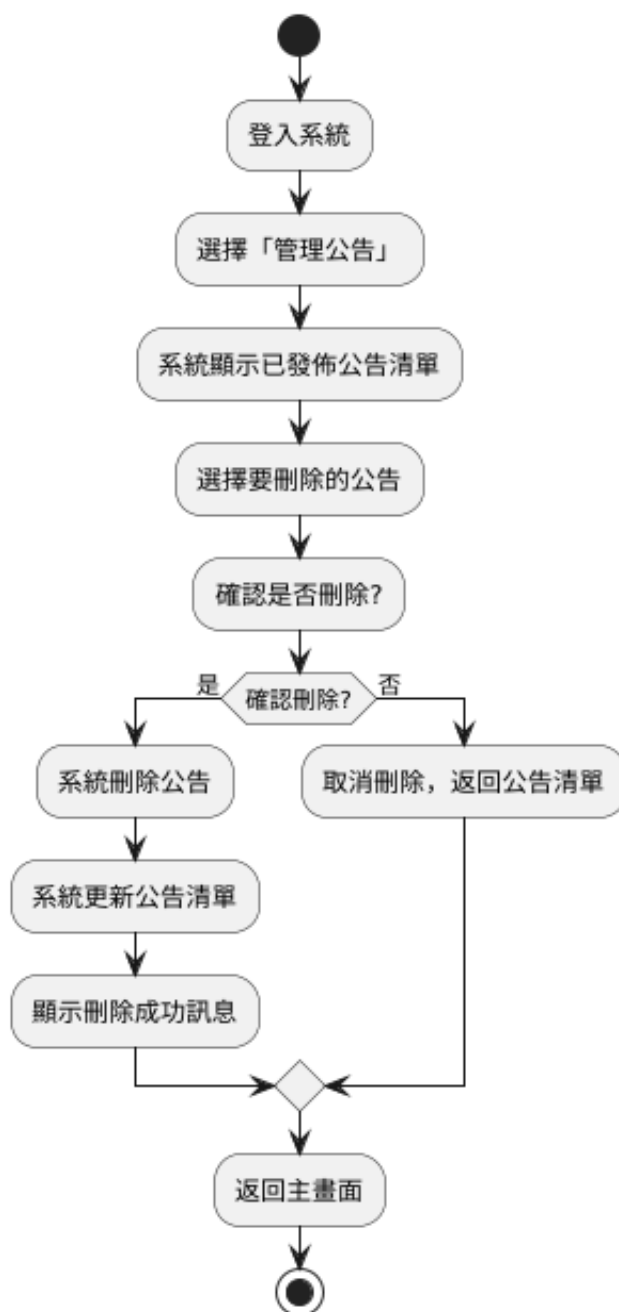


圖5-3-22使用活動圖-Activity diagram
- 教練刪除公告活動圖

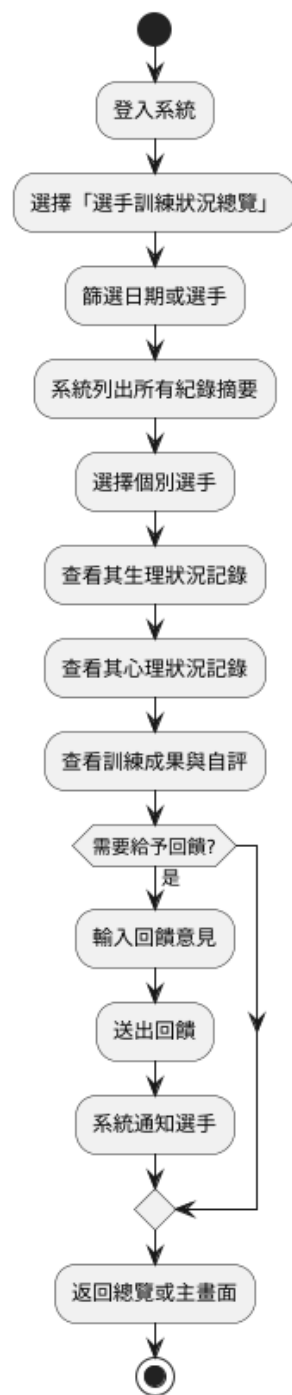


圖5-3-23

使用活動圖-Activity diagram- 教練查看選手訓練狀況



圖5-3-24

使用活動圖-Activity diagram - 教練點名

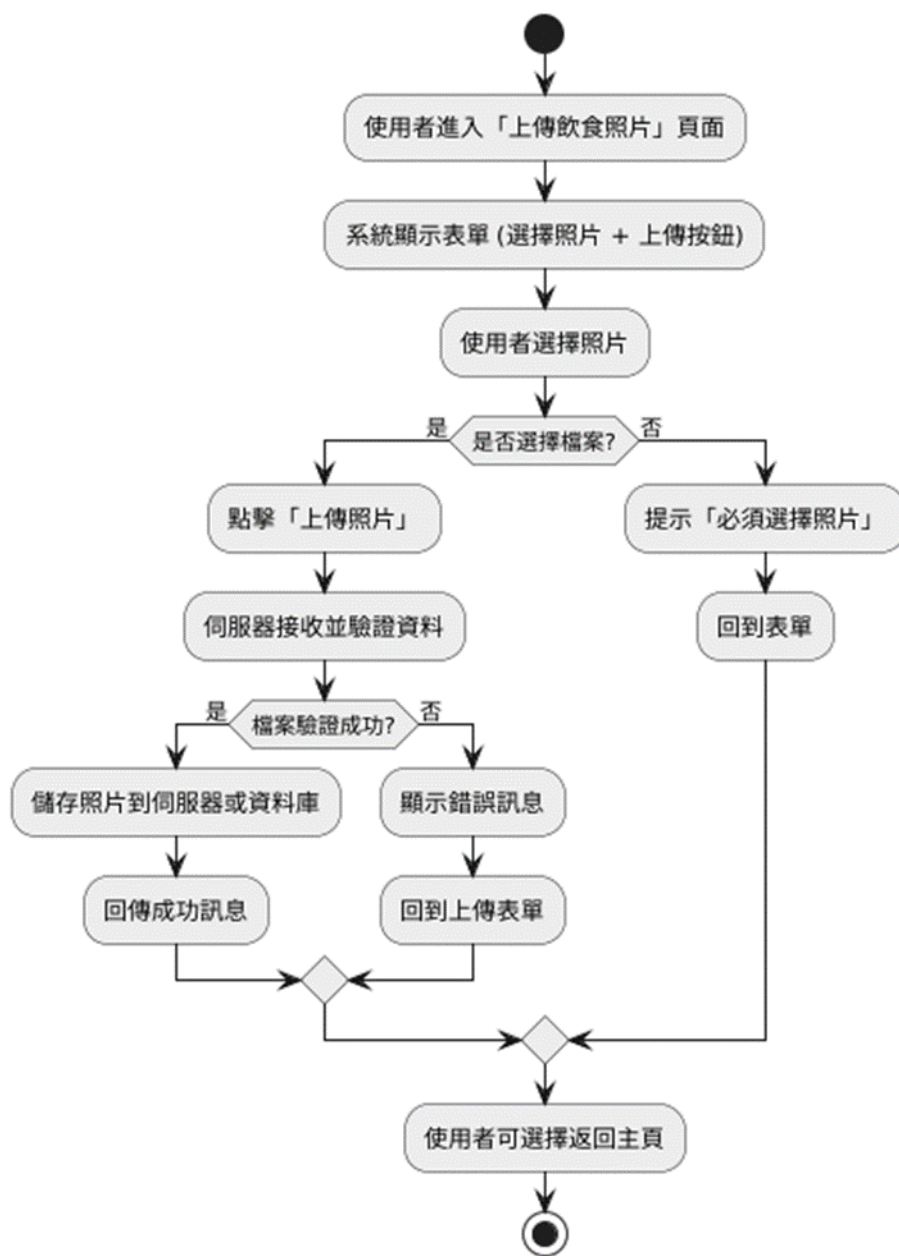


圖5-3-25
使用活動圖 -Activity diagram -
教練查看飲食紀錄

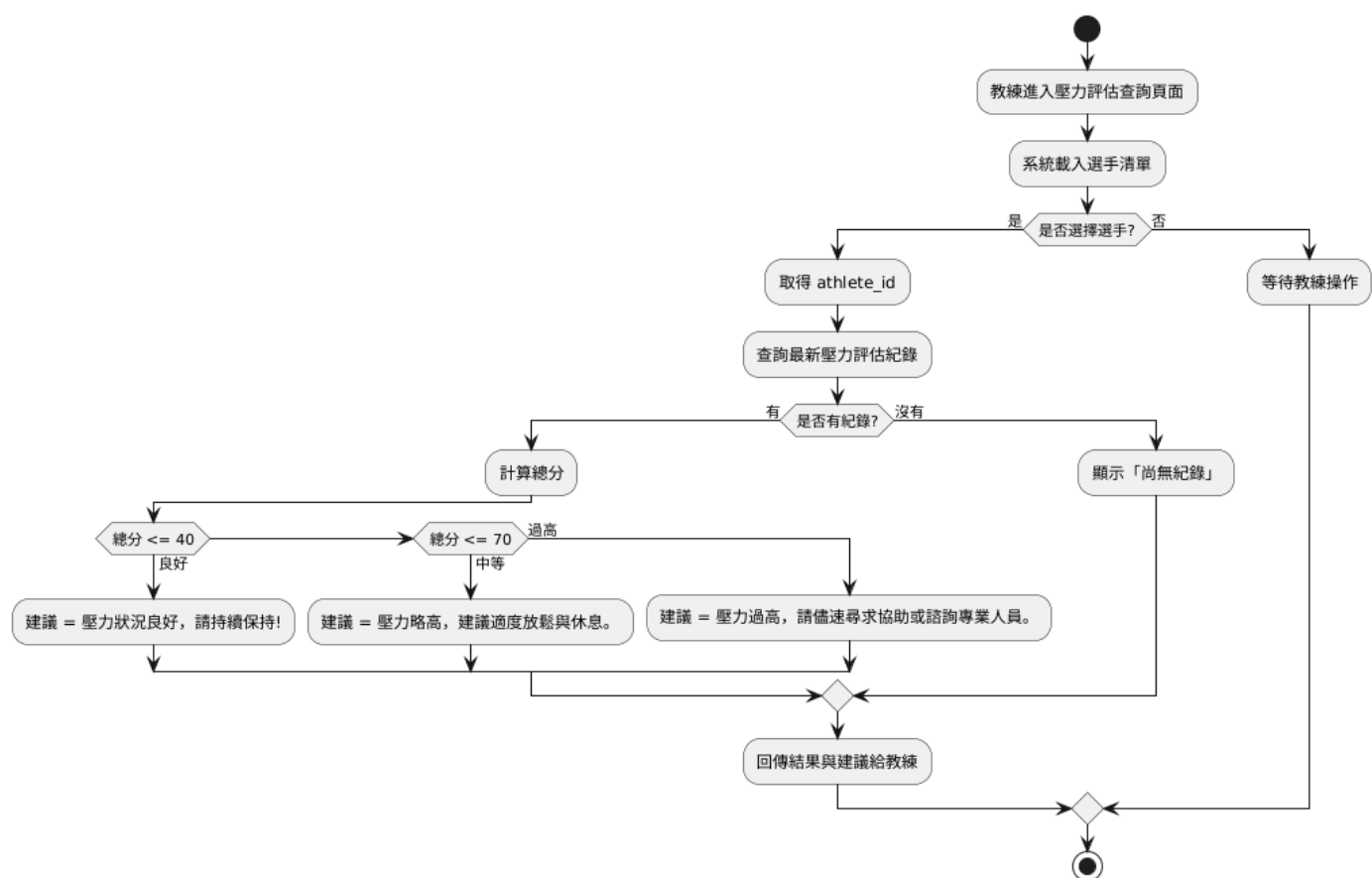


圖5-3-26
使用活動圖 -Activity diagram -
教練查看壓力評估

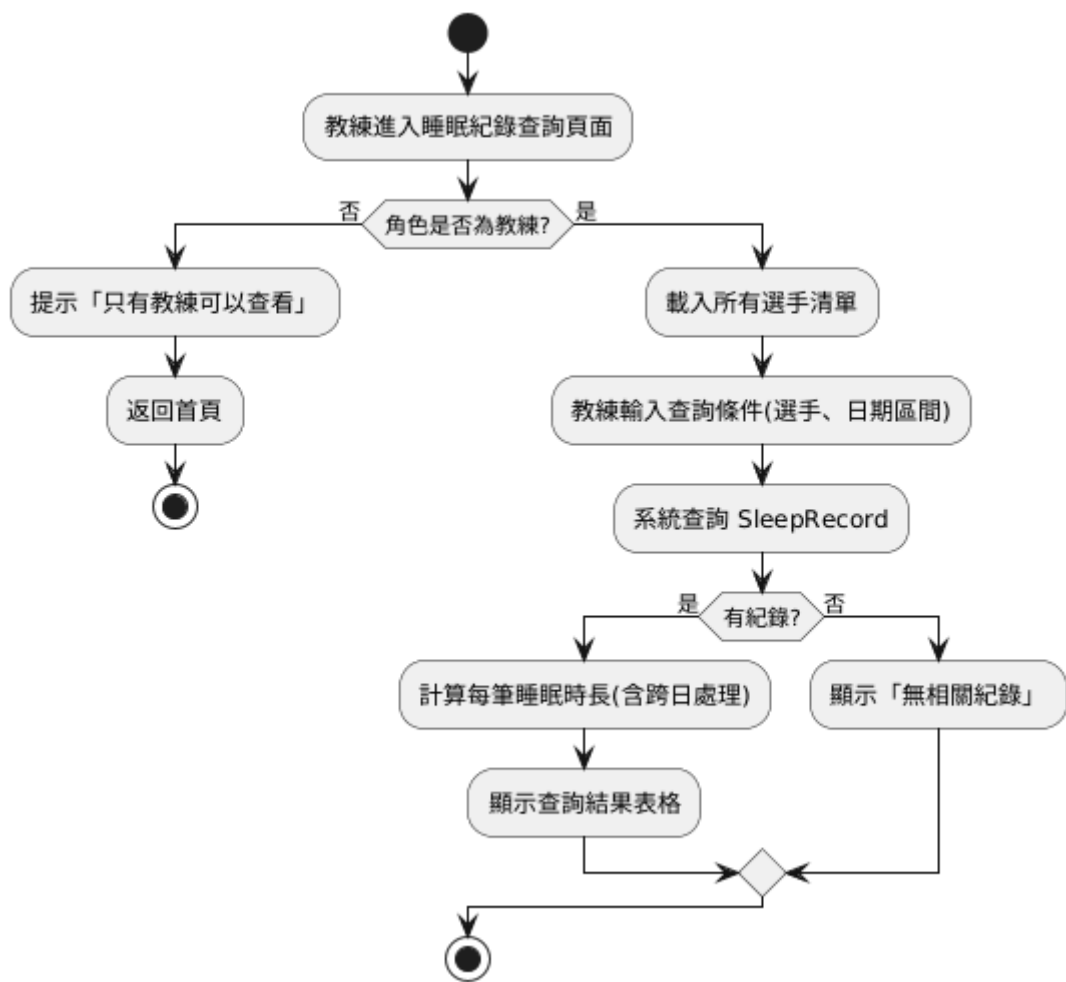


圖5-3-27
使用活動圖 -Activity diagram -
教練查看睡眠紀錄

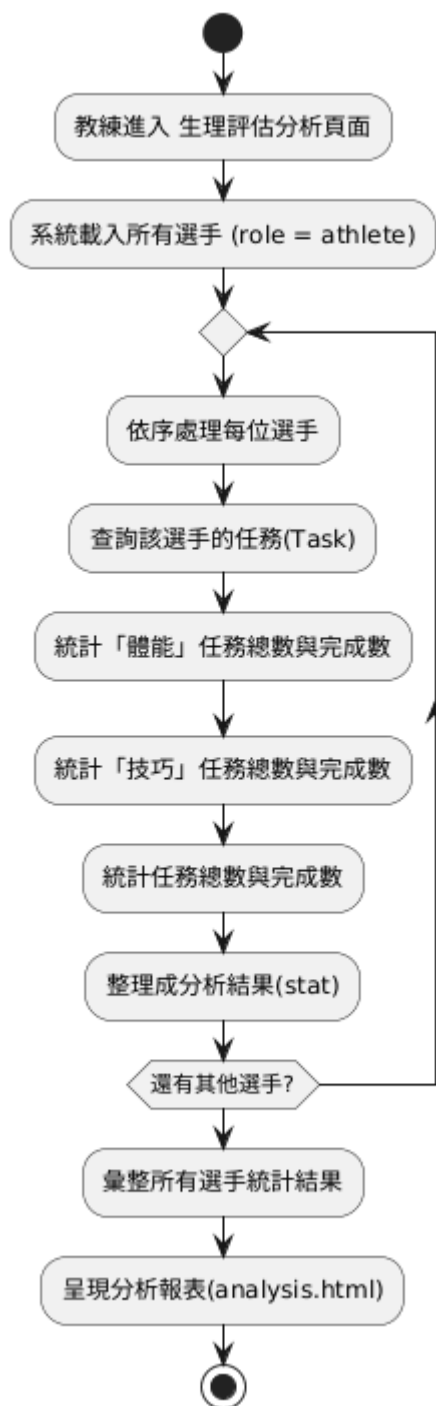


圖5-3-28
使用活動圖 -Activity diagram -
教練查看生理評估

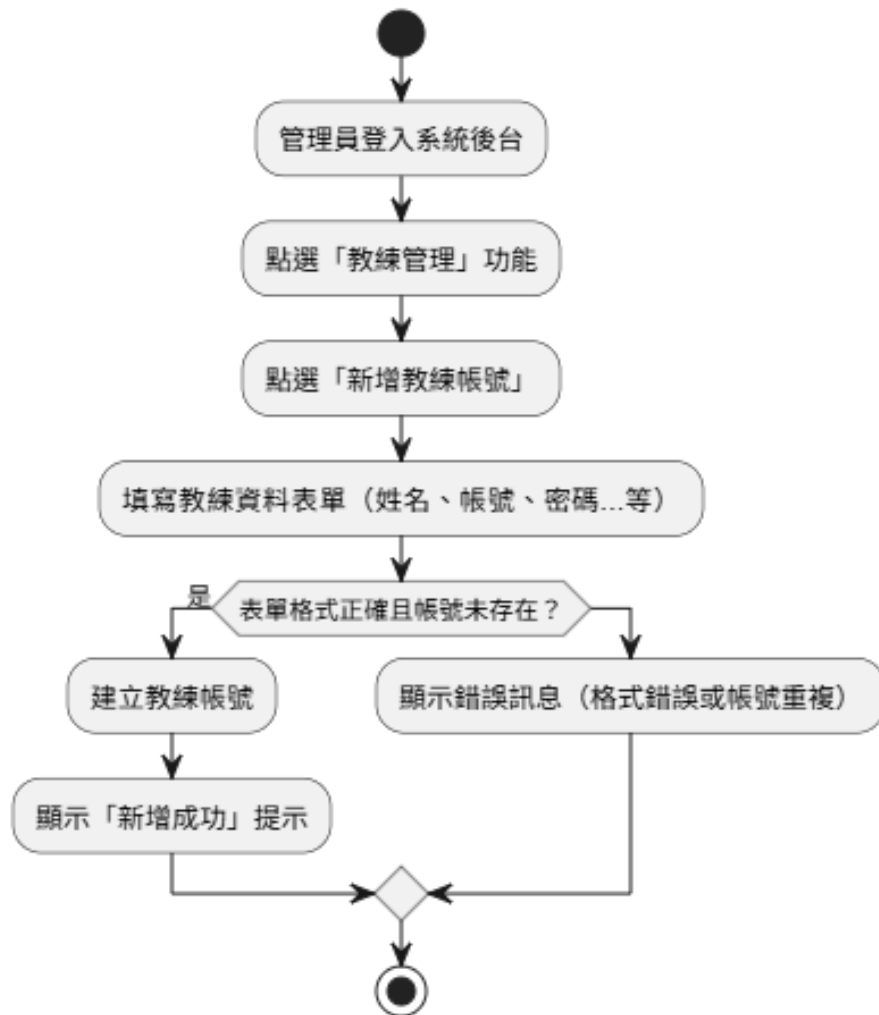


圖5-3-29

使用活動圖-Activity diagram- 管理者新增教練帳號



圖5-3-30-使用活動圖 -Activity diagram - 個人自介新增



圖5-3-31-使用活動圖 -Activity diagram - 個人自介查詢



圖5-3-32

使用活動圖 -Activity diagram - 個人自介修改

5-4 分析類別圖(Analysis class diagram)

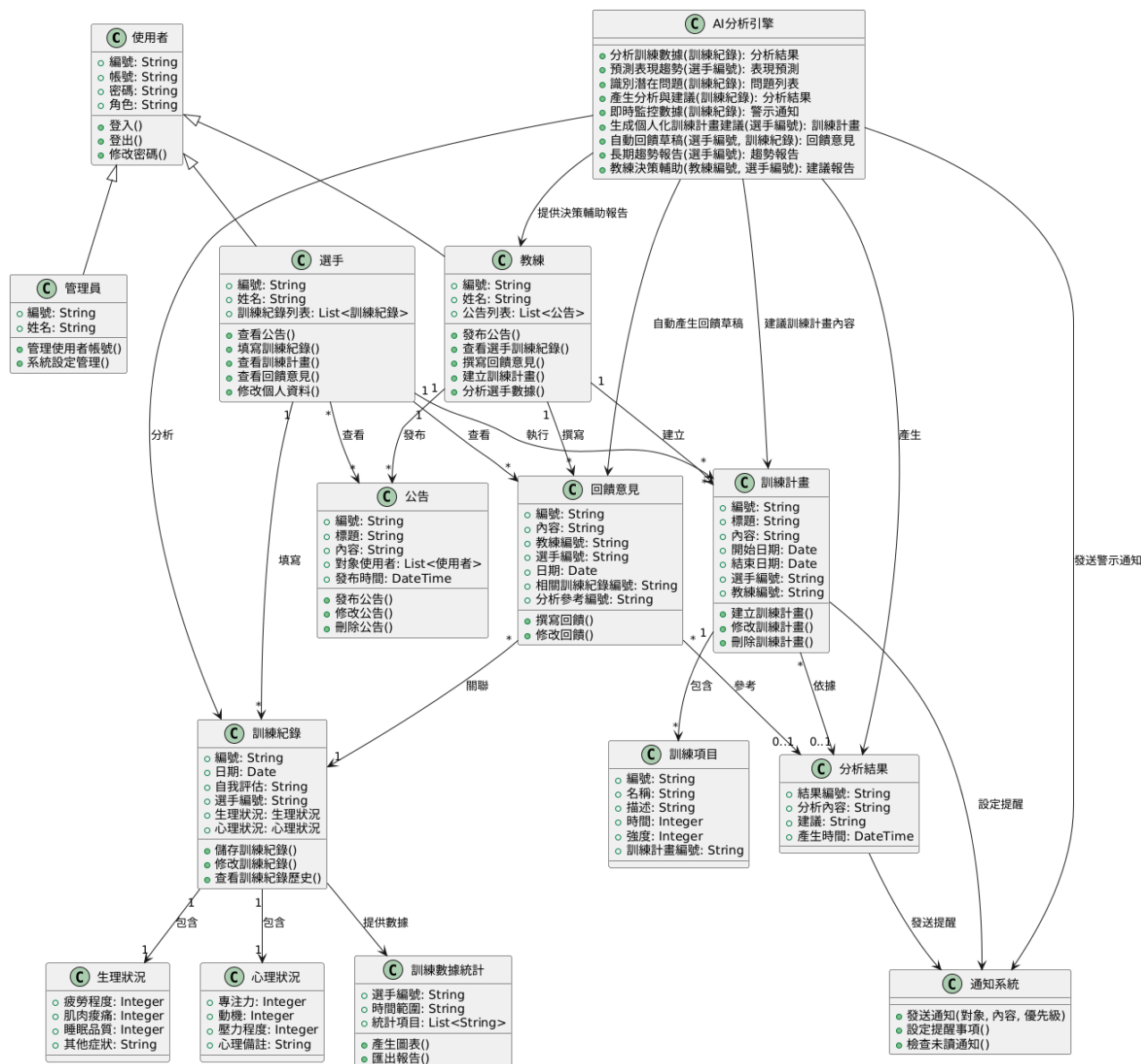


圖5-4-1

使用分析類別圖-Analysis class diagram

第六章 設計模型

6-1 循序圖(Sequential diagram)

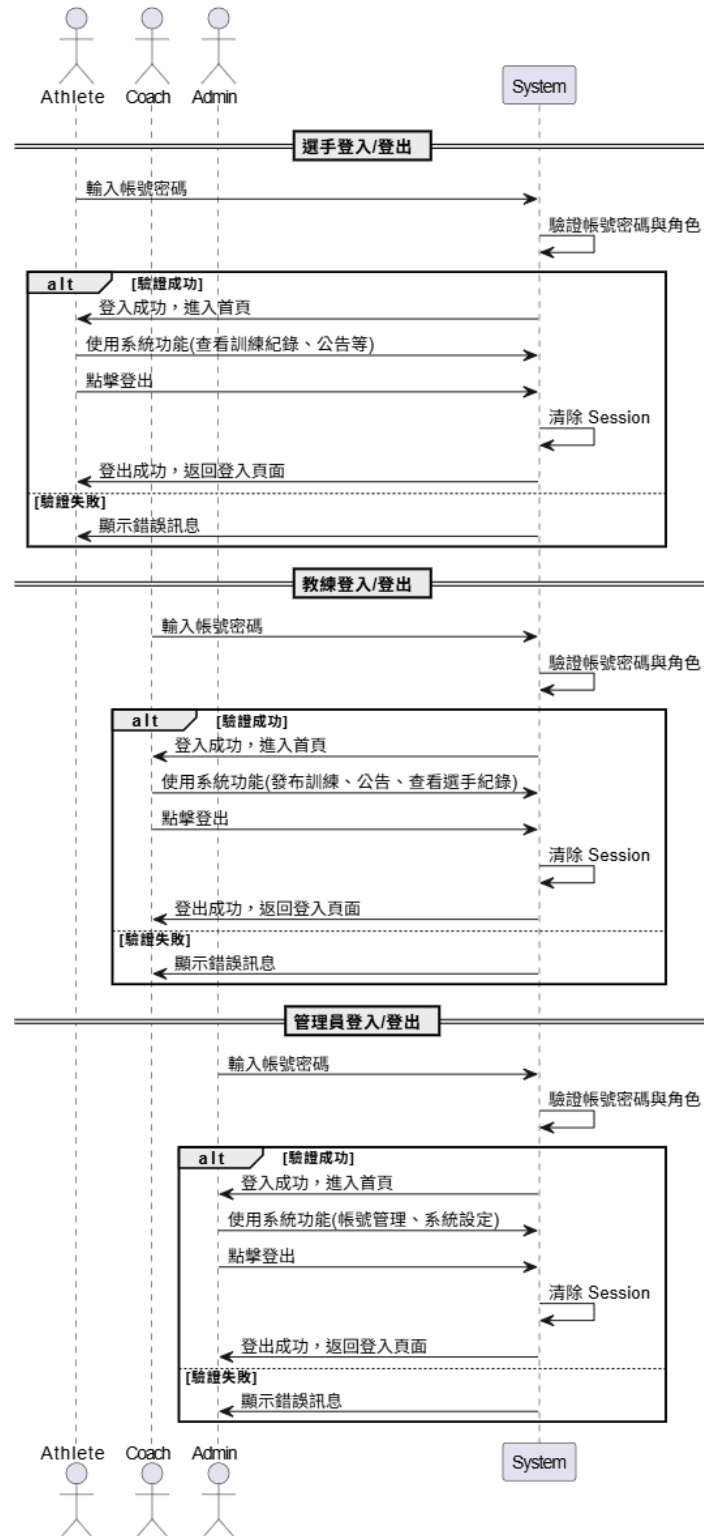


圖6-1-1-循序圖-Sequential diagram-系統登入/登出

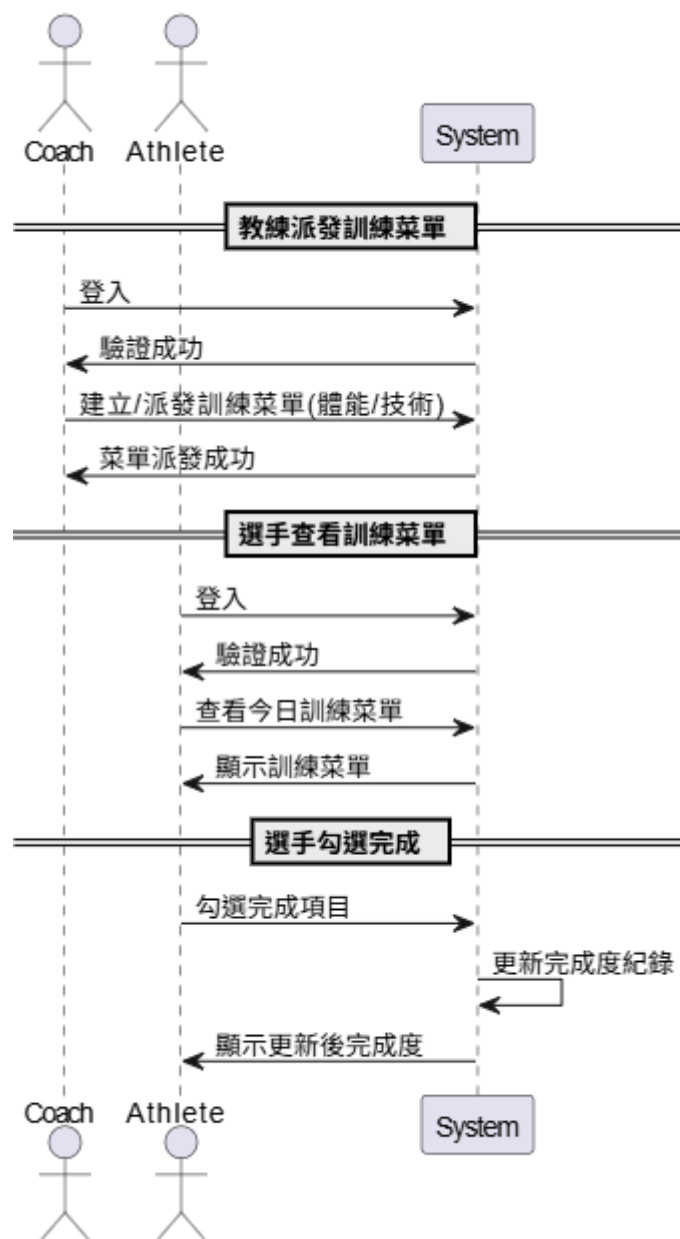


圖6-1-2-循序圖-Sequential diagram-訓練菜單

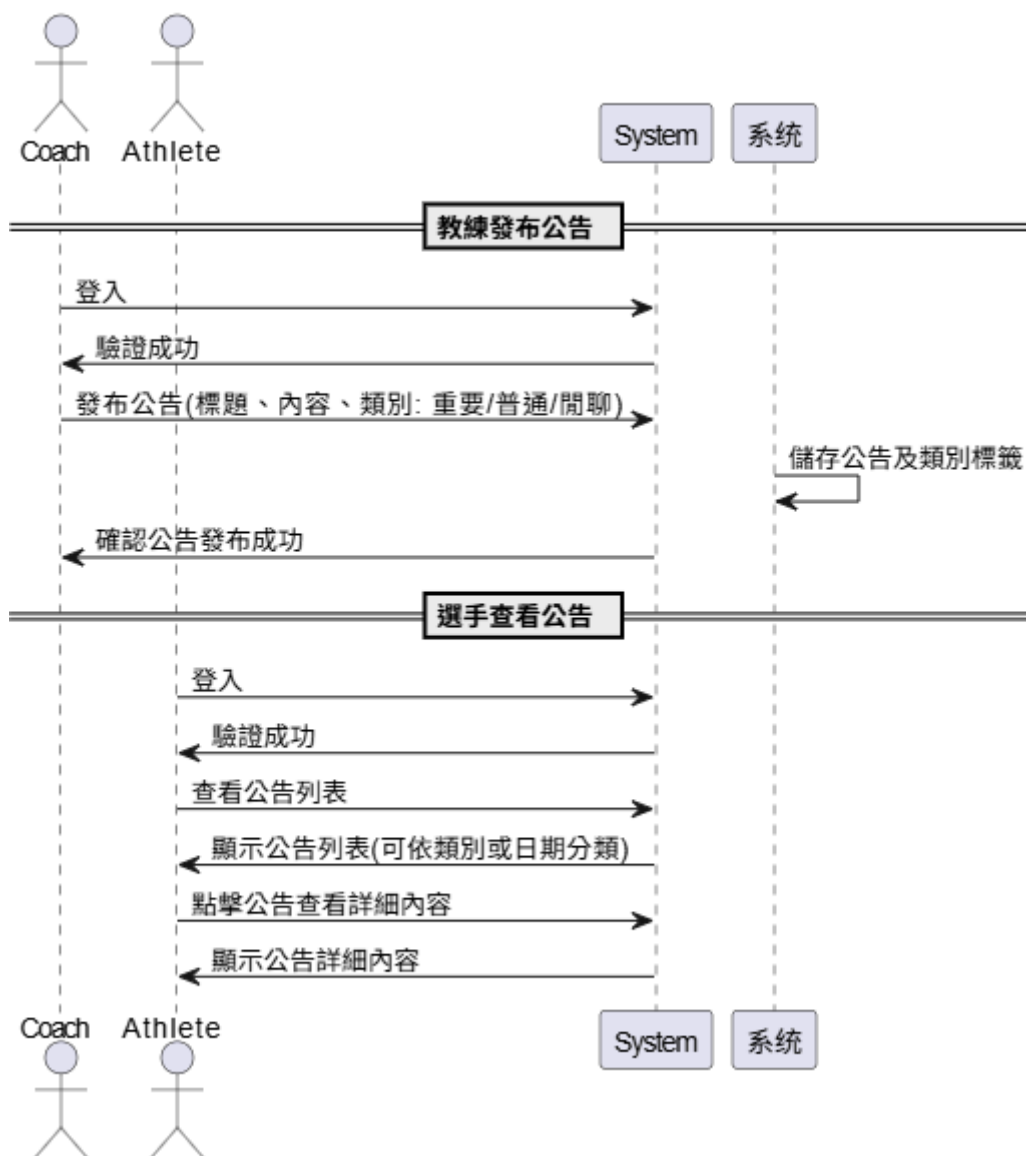


圖6-1-3

循序圖-Sequential diagram- 發布及查看公告

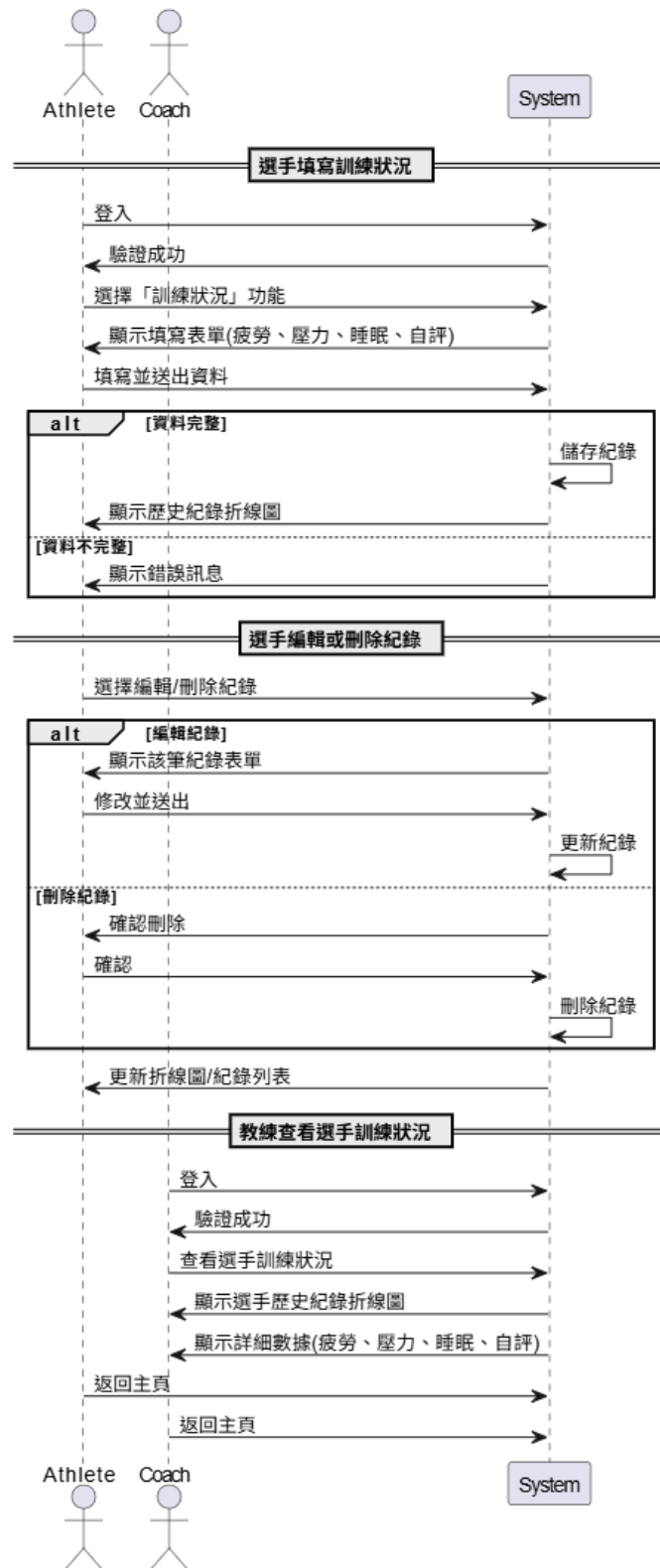


圖6-1-4

循序圖-Sequential diagram- 訓練狀況

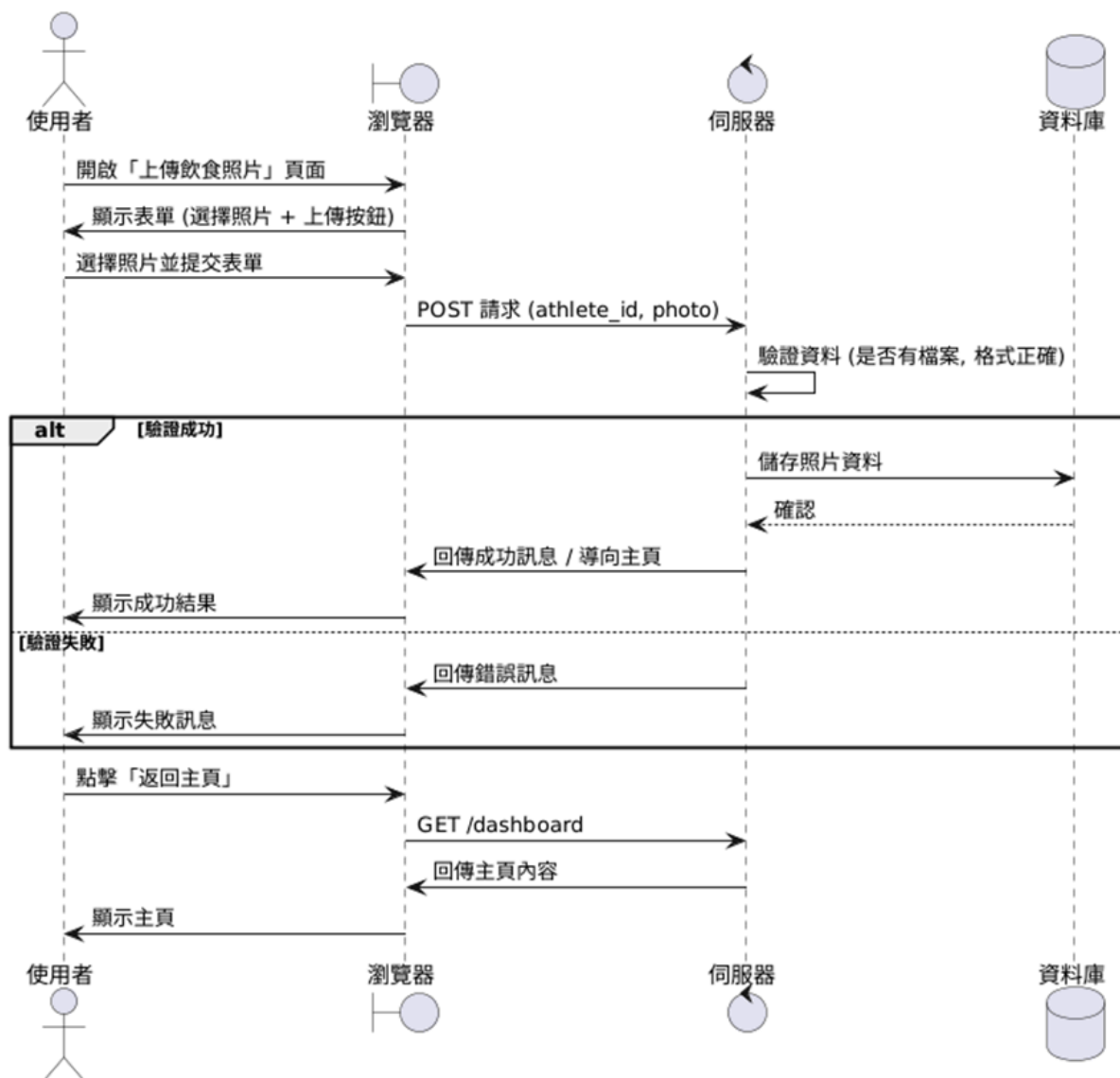


圖6-1-5-

循序圖-Sequential diagram- 選手上傳飲食照片

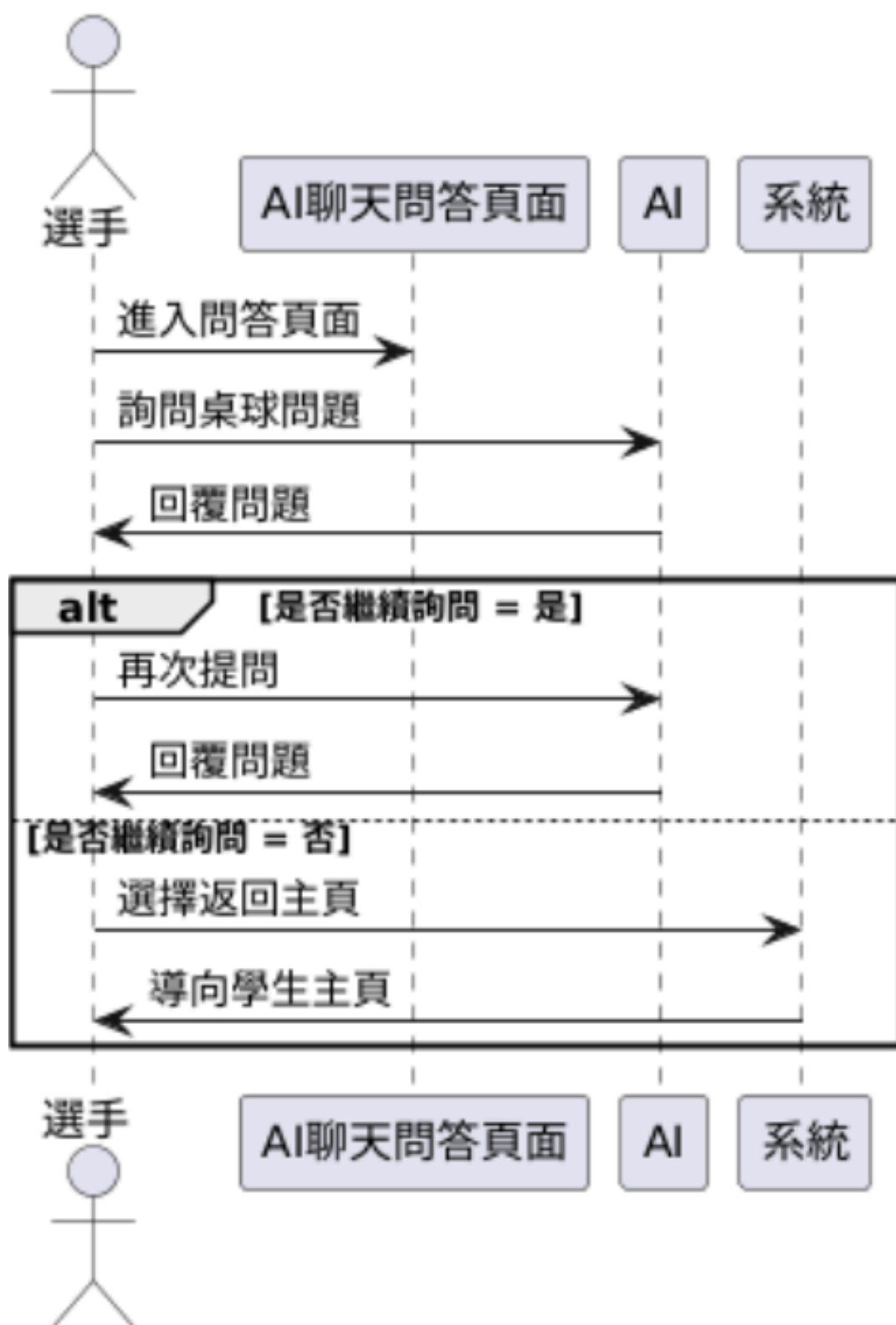


圖6-1-6-循序圖-Sequential diagram- 選手使用智能助手

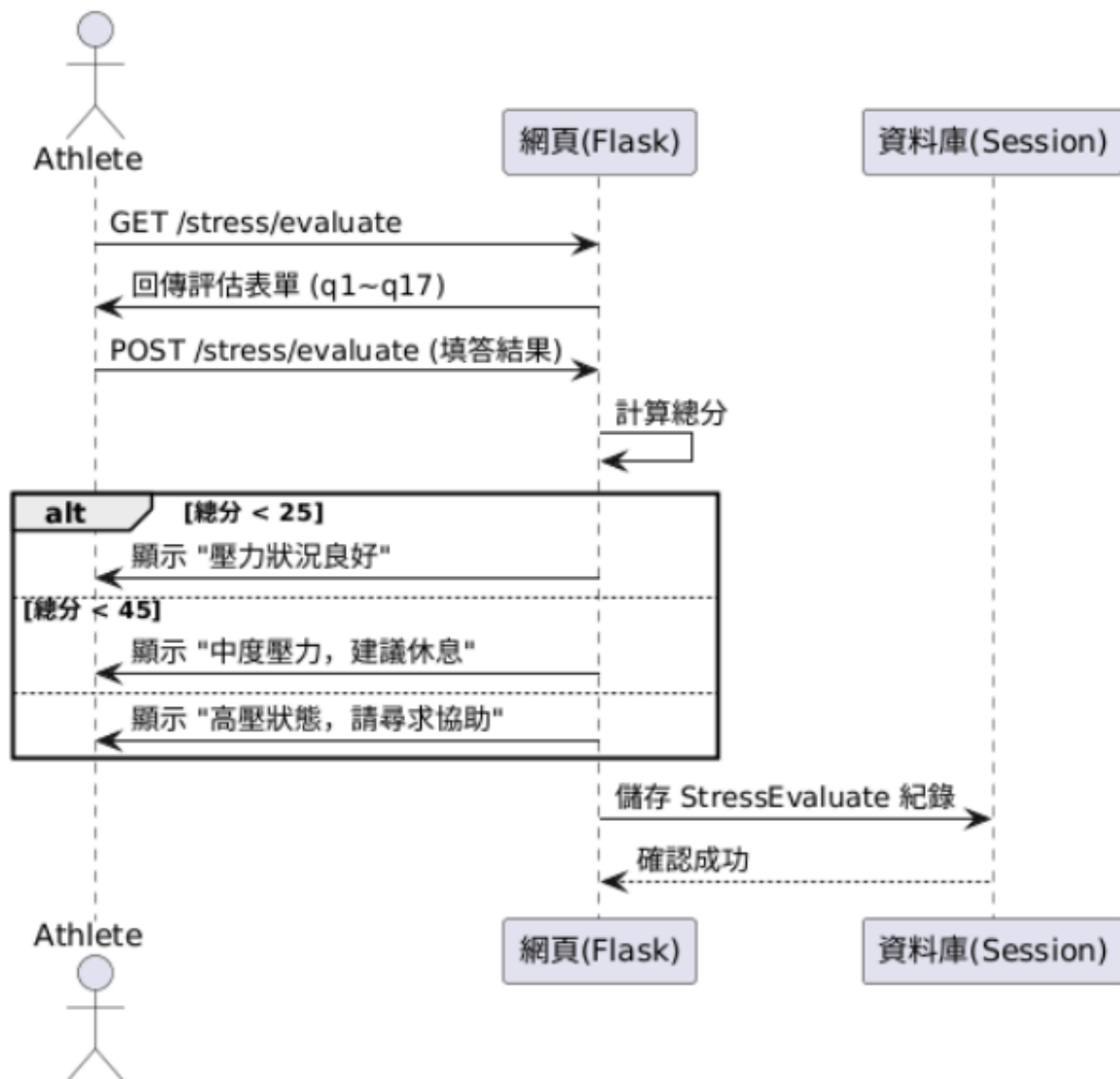


圖6-1-7-循序圖-Sequential diagram- 選手填寫壓力評估

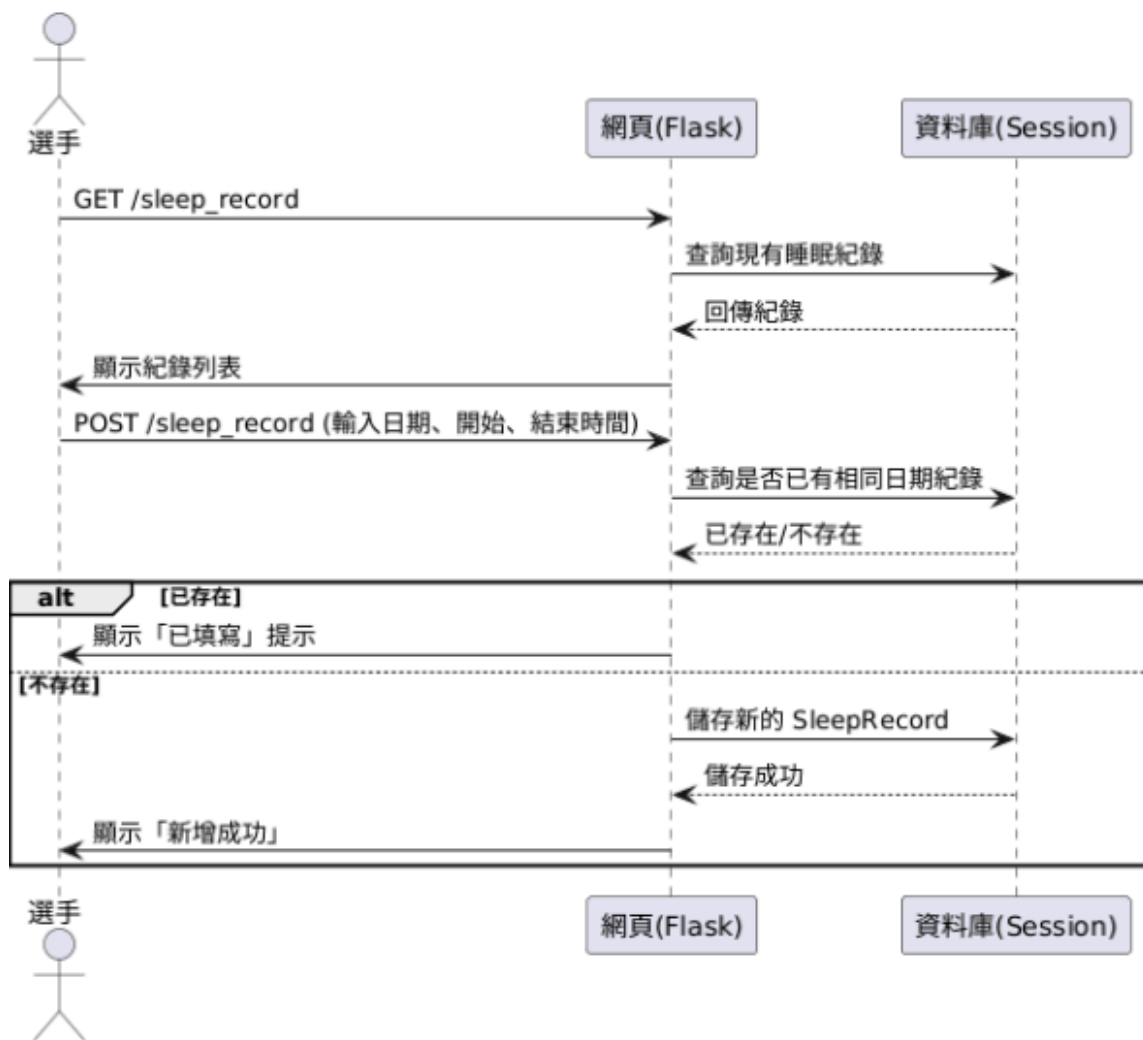


圖6-1-8-循序圖-Sequential diagram- 選手填寫睡眠紀錄

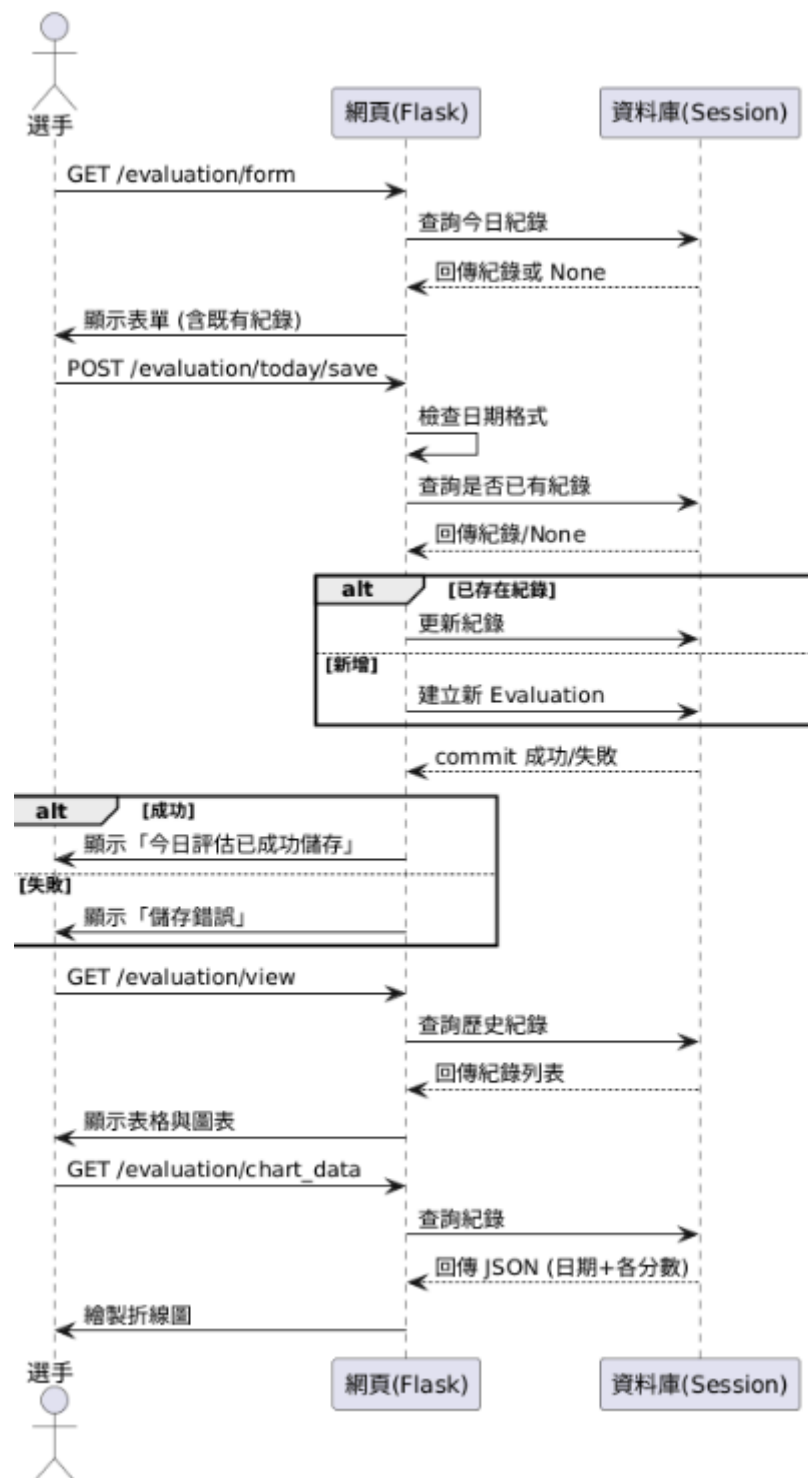


圖6-1-9-循序圖-Sequential diagram- 選手填寫生理評估

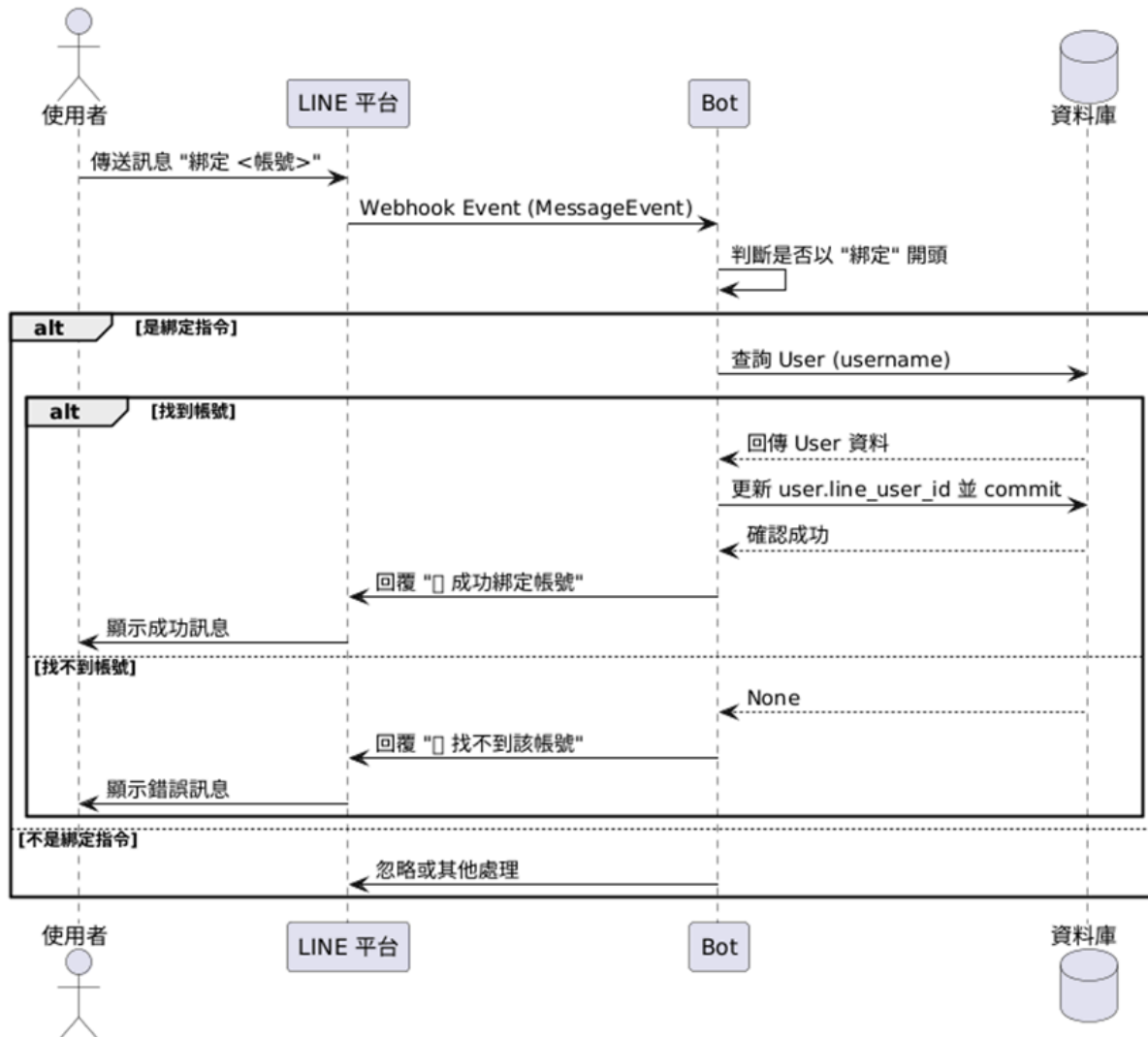


圖6-1-10-循序圖-Sequential diagram- 選手LINE Bot 綁定帳號

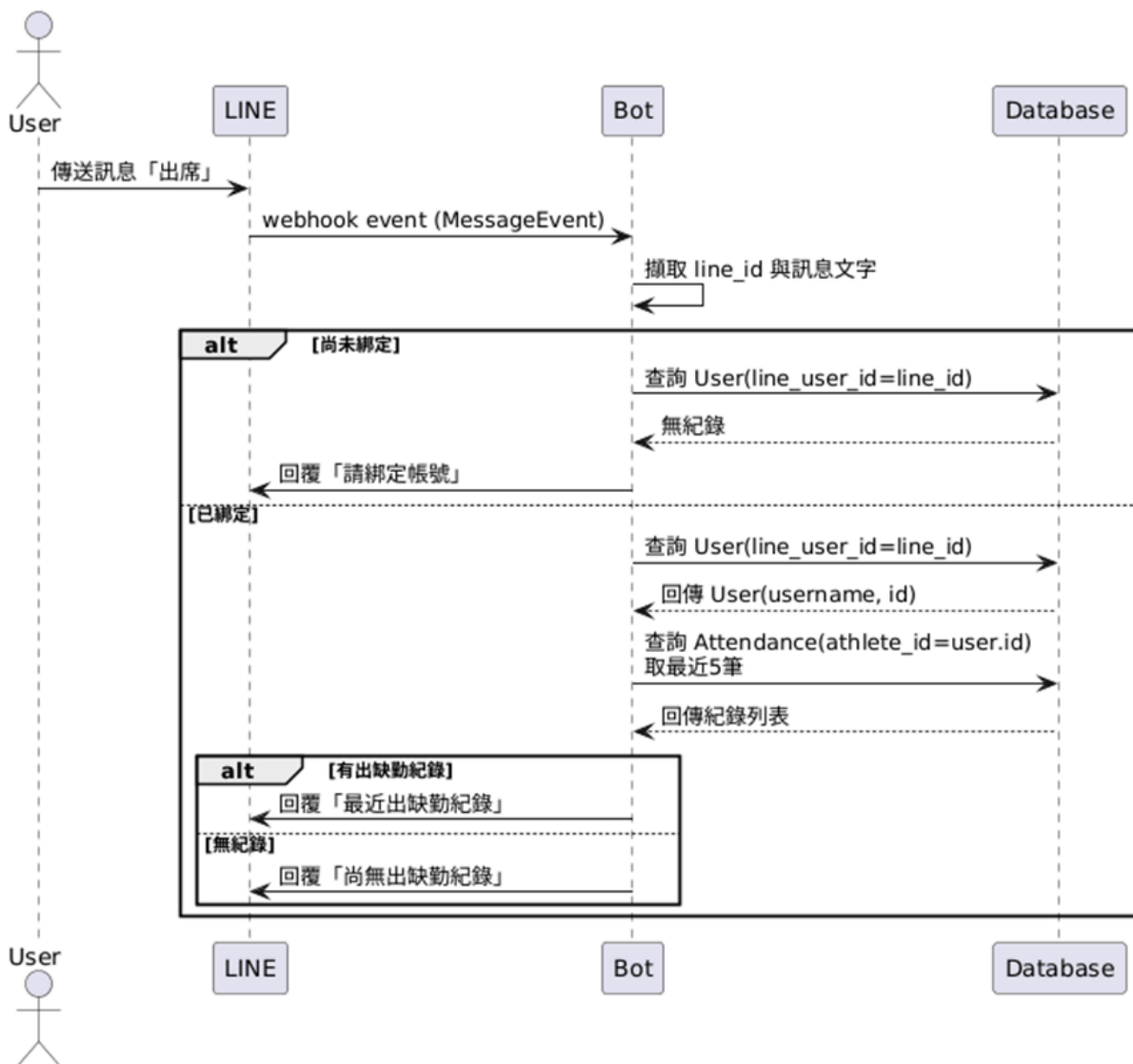


圖6-1-11-循序圖-Sequential diagram- 選手LINE Bot查詢出缺勤紀錄

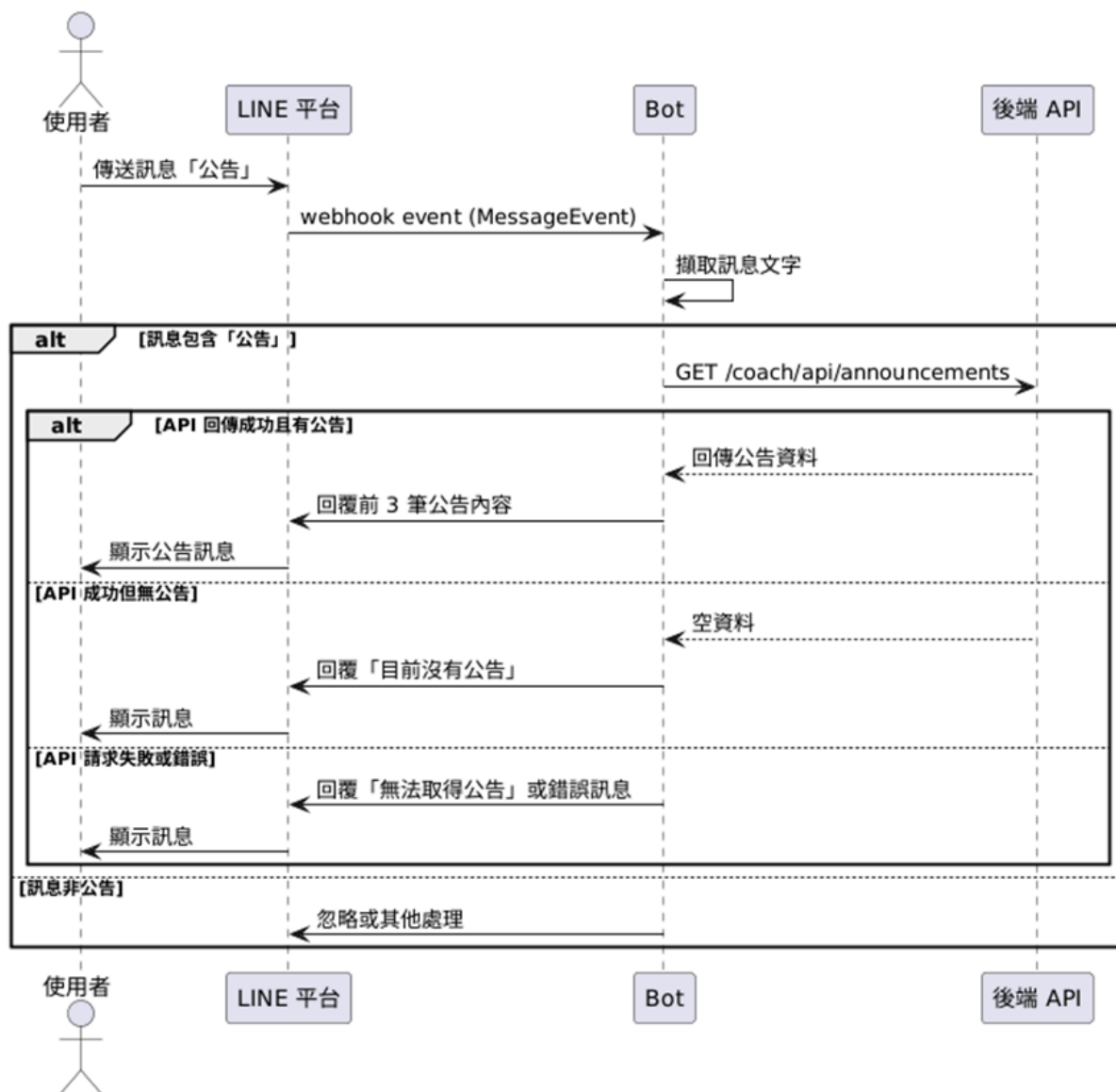


圖6-1-12-循序圖-Sequential diagram- 選手LINE Bot公告查詢

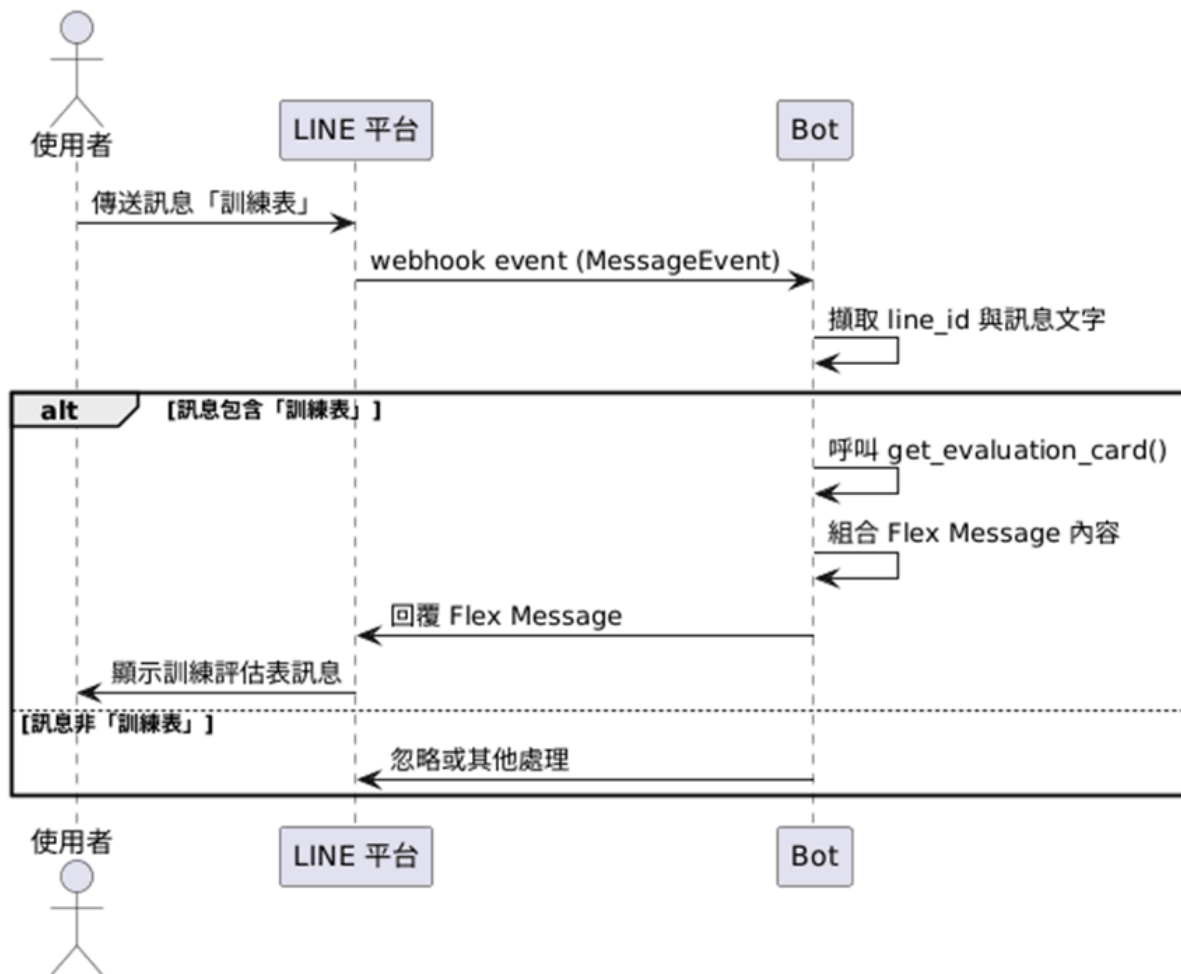


圖6-1-13-循序圖-Sequential diagram- 選手LINE Bot今日訓練狀況

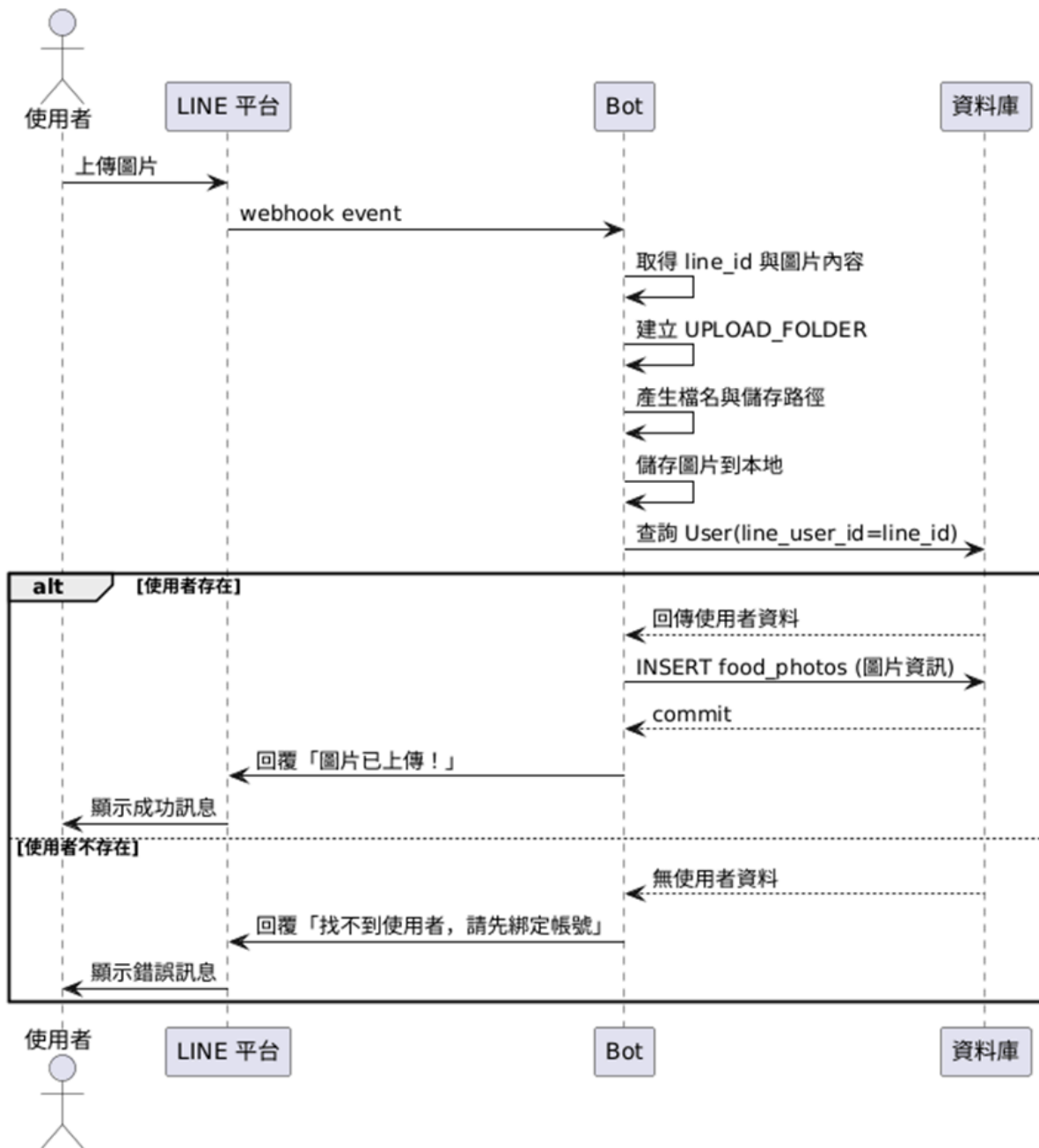


圖6-1-14-循序圖-Sequential diagram- 選手LINE Bot上傳飲食照片

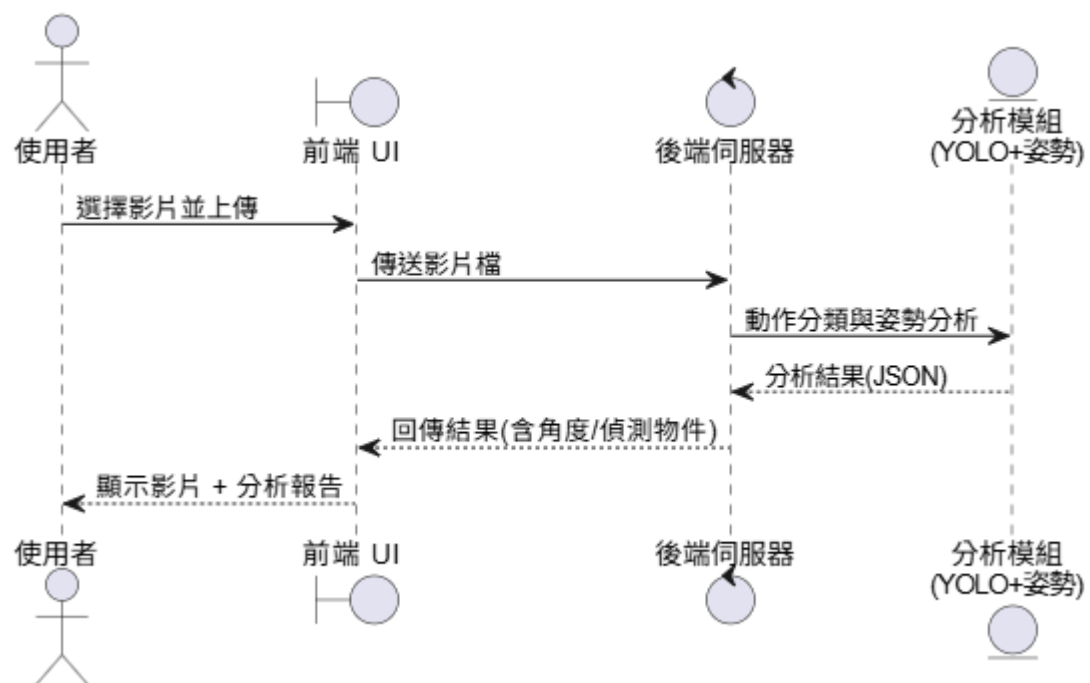


圖6-1-15-循序圖-Sequential diagram- 選手動作分析模組



圖6-1-16-循序圖-Sequential diagram- 教練查看飲食紀錄

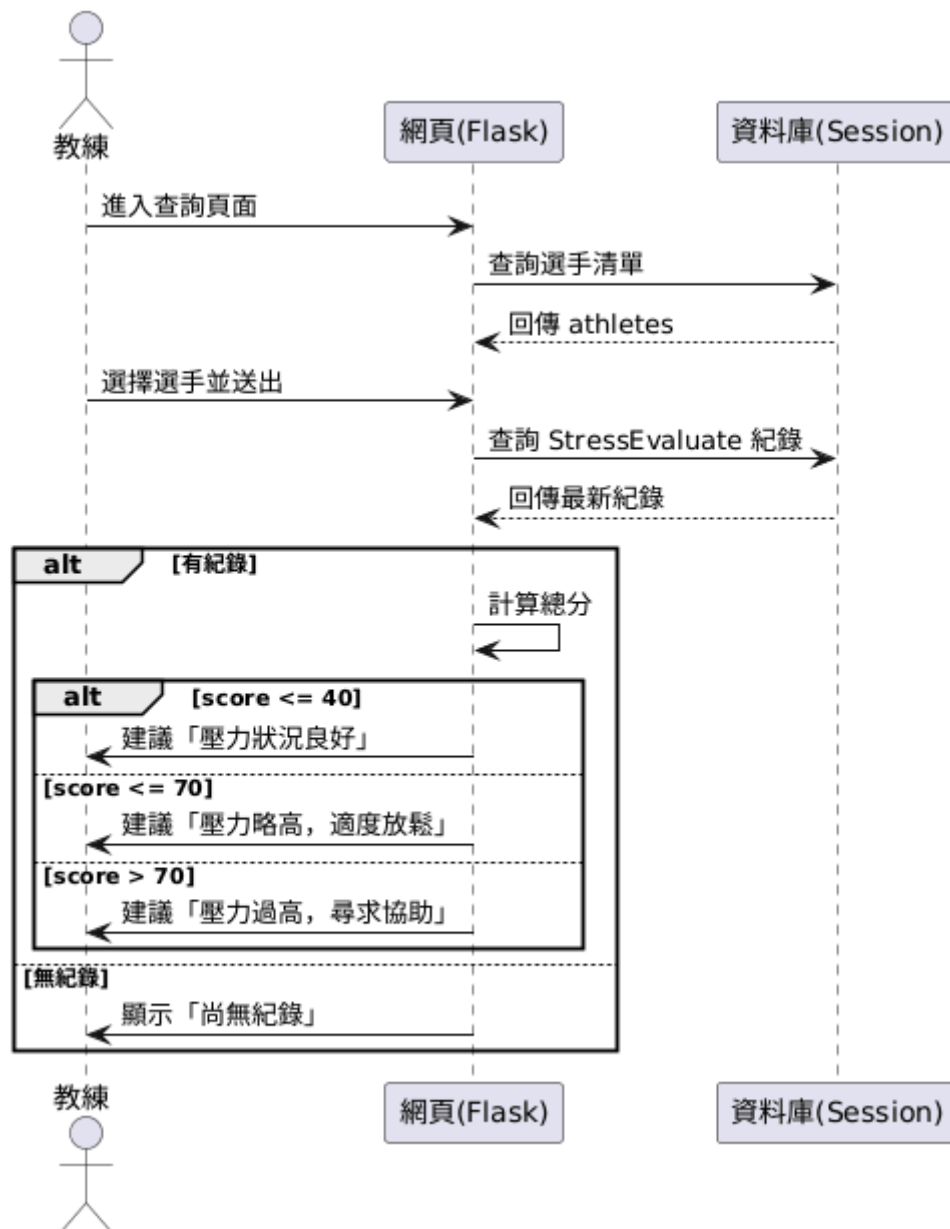


圖6-1-17-循序圖-Sequential diagram- 教練查看壓力評估

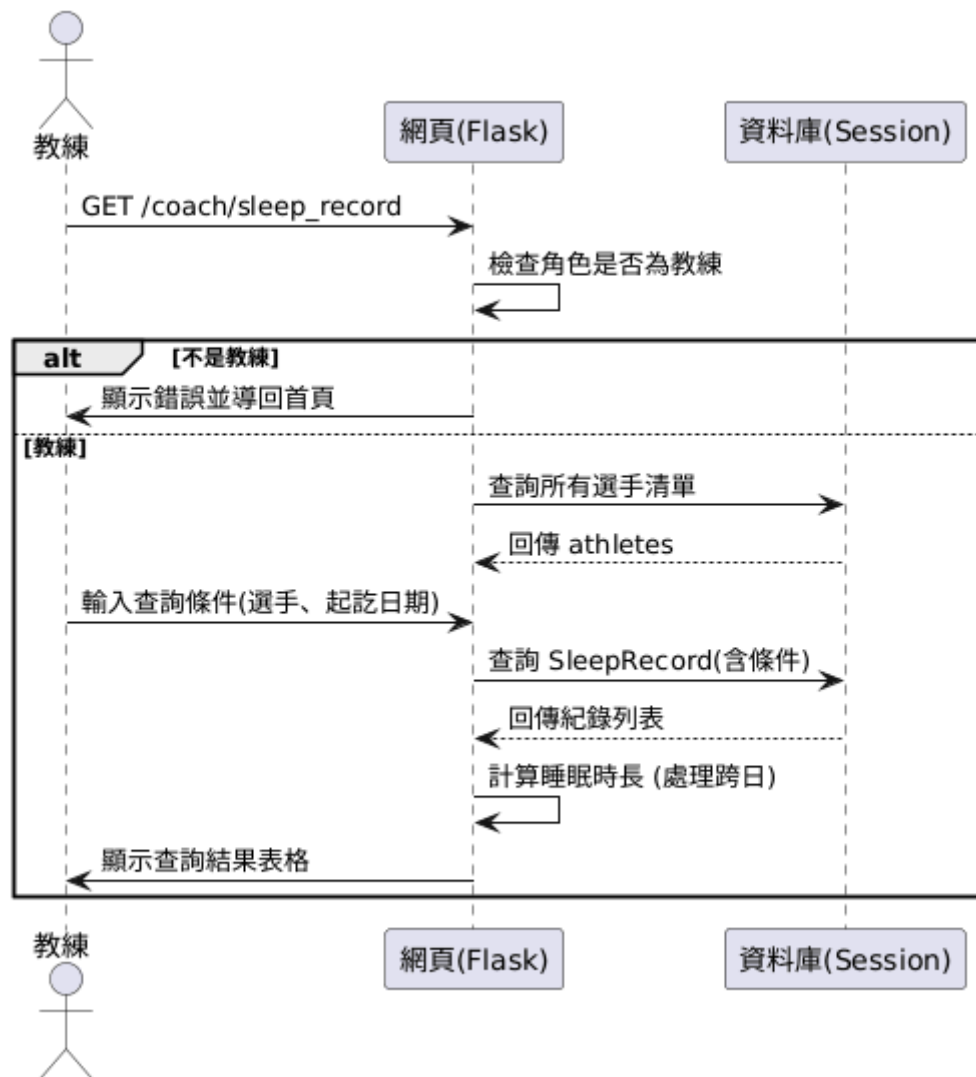


圖6-1-18-循序圖-Sequential diagram- 教練查看睡眠紀錄

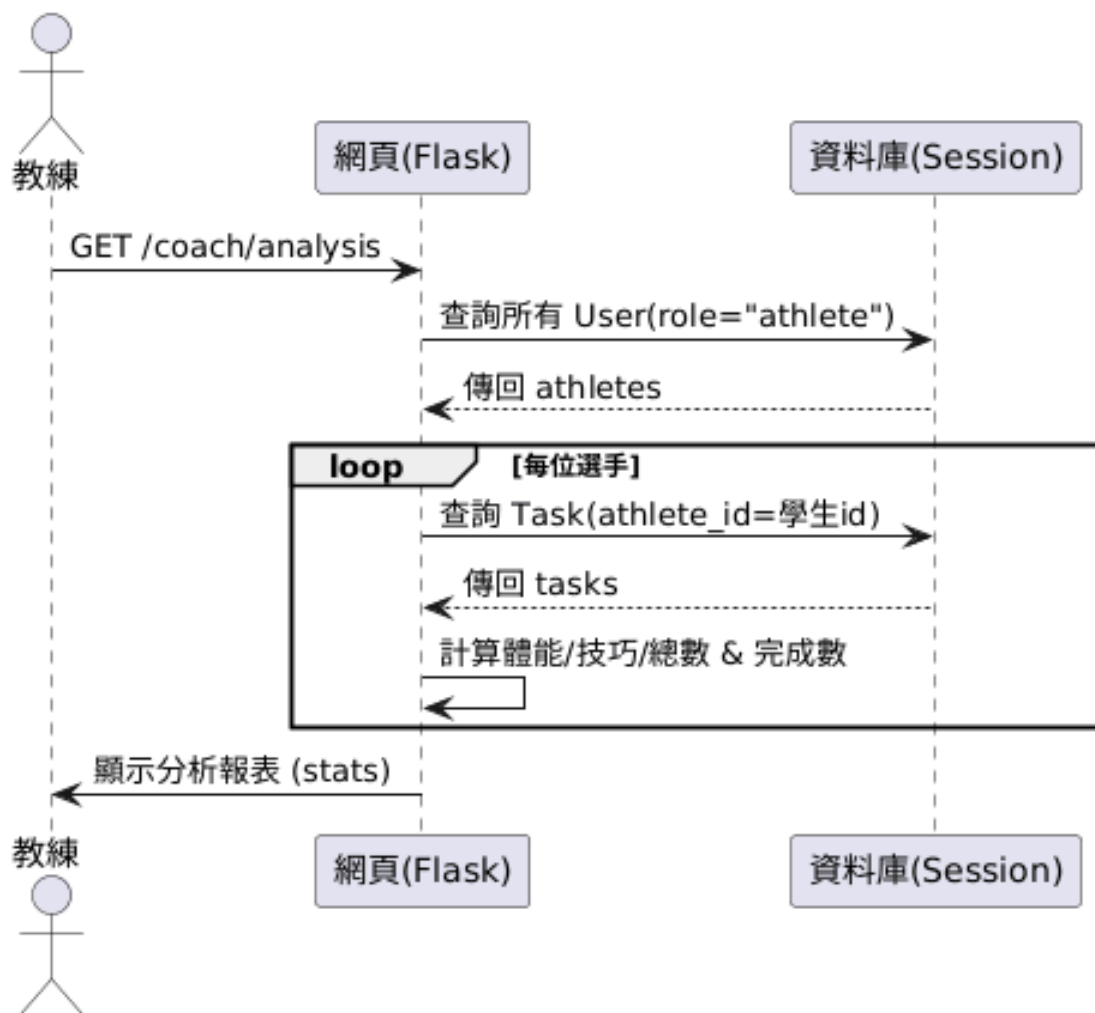


圖6-1-19-循序圖-Sequential diagram- 教練查看生理評估

6-2 設計類別圖(Design class diagram)

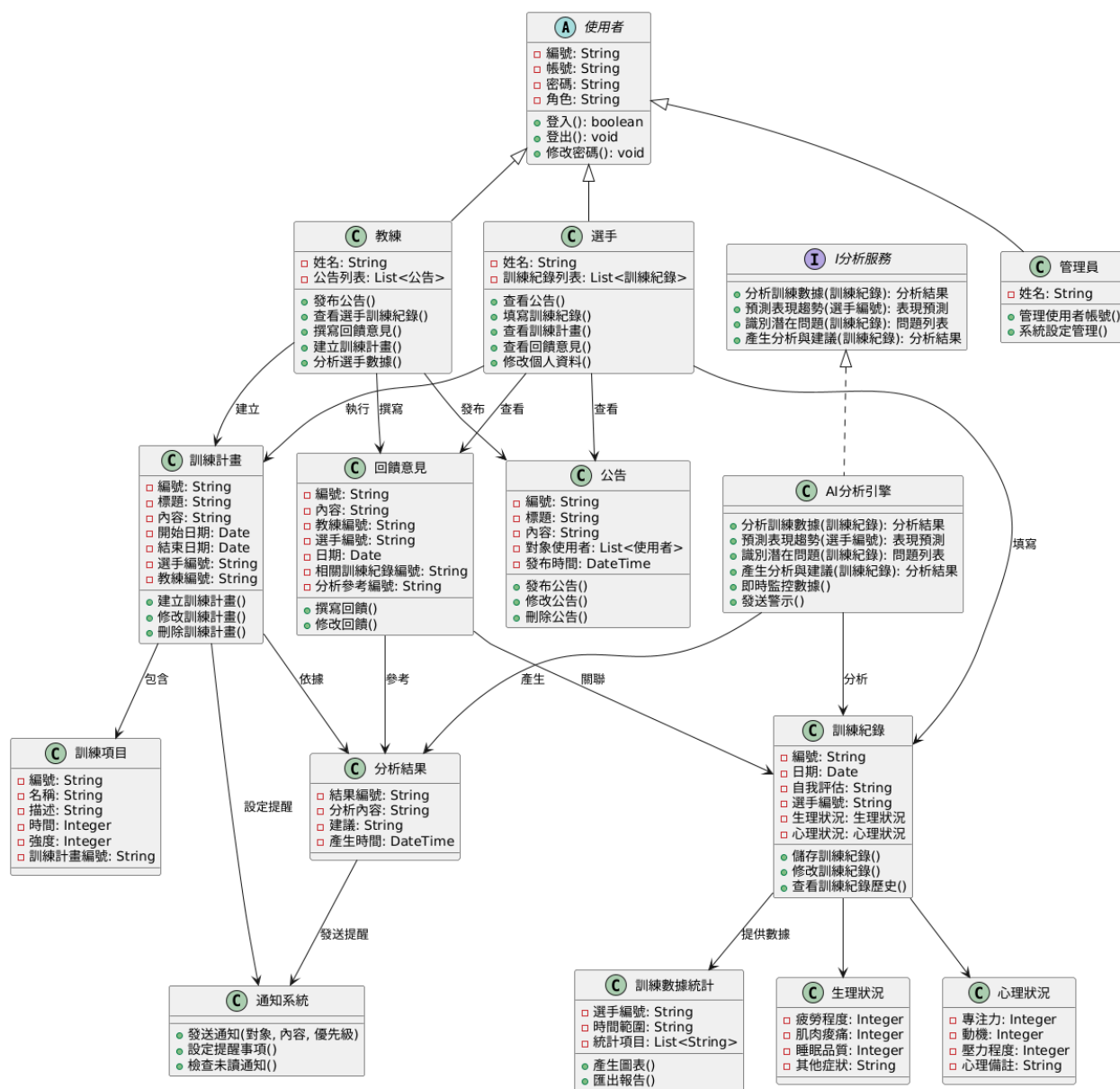


圖6-2-1-設計類別圖-Design class diagram

第七章 實作模型

7-1 佈署圖(Deployment diagram)。

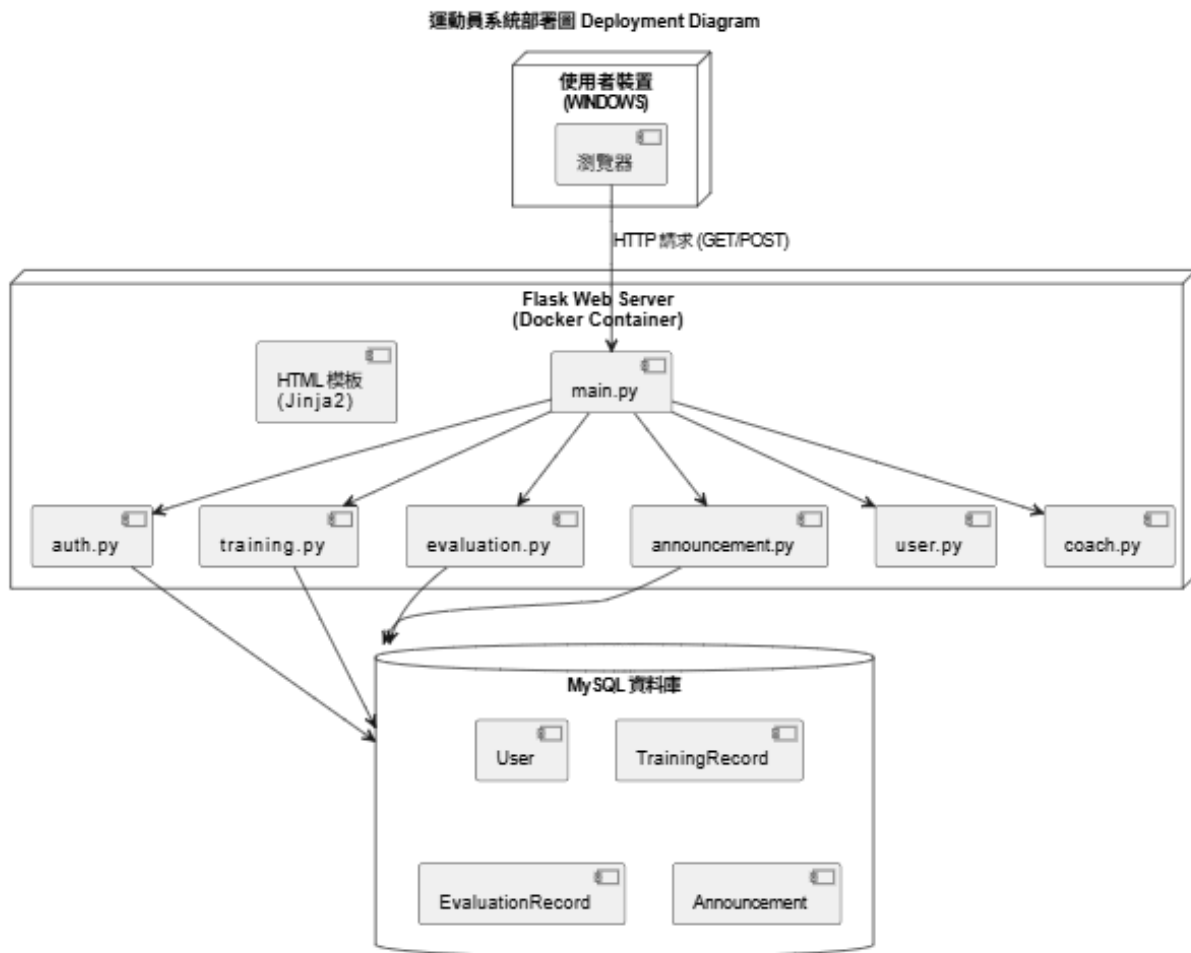


圖7-1-1-佈署圖-Deployment diagram

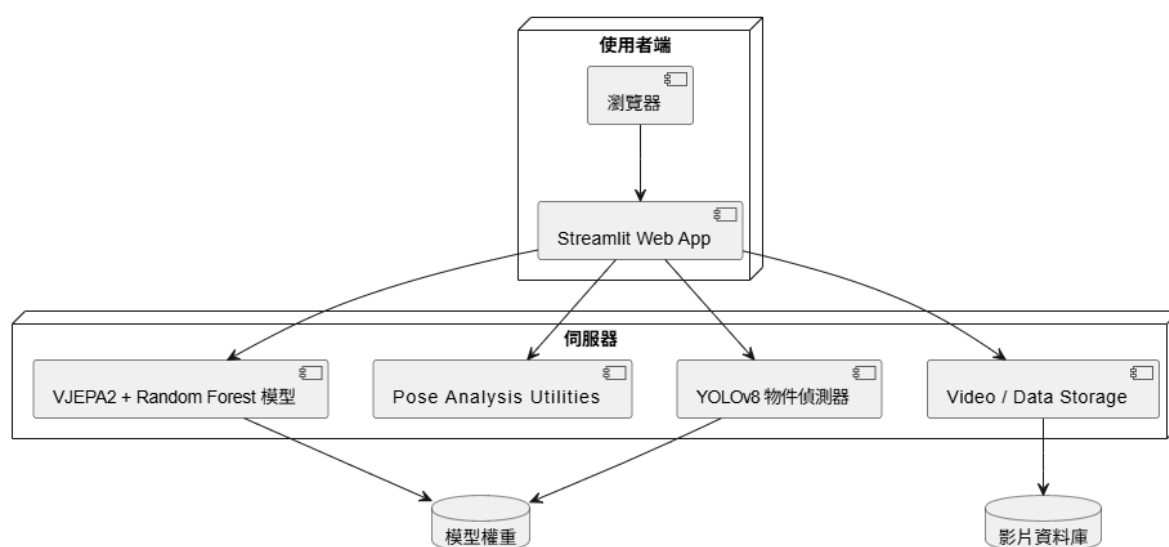


圖7-1-2-佈署圖-Deployment diagram

7-2 套件圖(Package diagram)

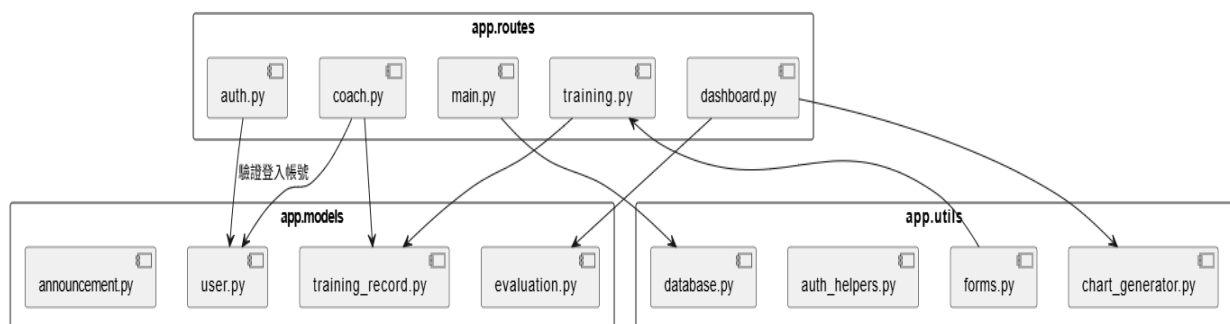


圖7-2-1-套件圖-Package diagram



圖7-2-2-套件圖-Package diagram

7-3 元件圖(Component diagram)

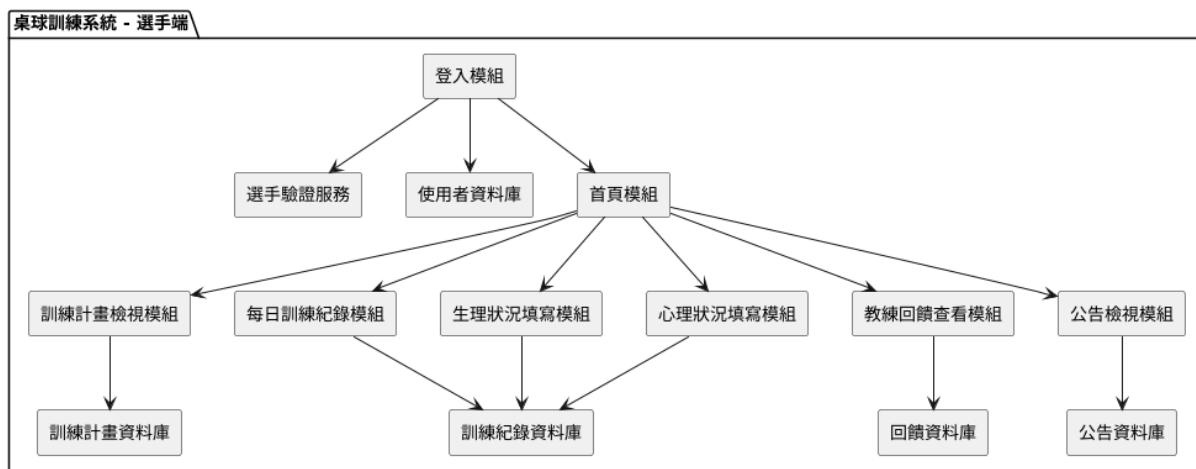


圖7-3-1-元件圖-Component diagram-選手端

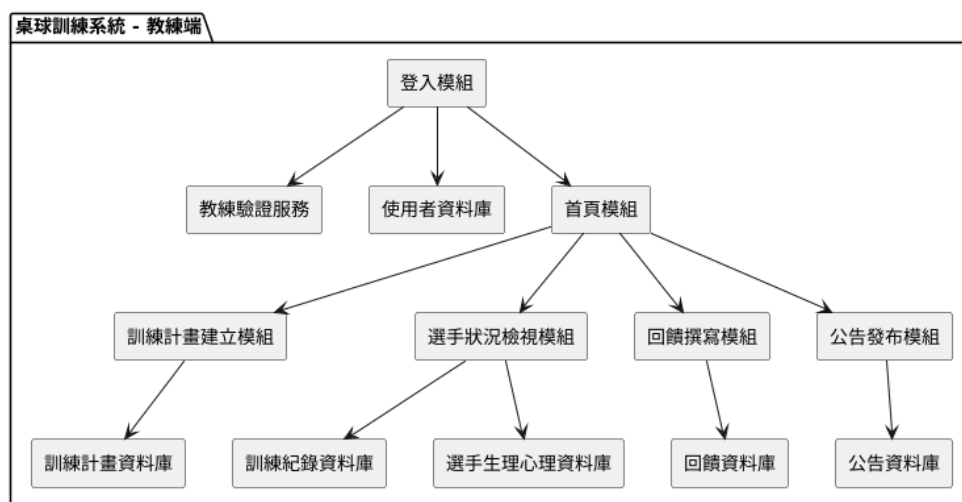


圖7-3-2-元件圖-Component diagram-教練端

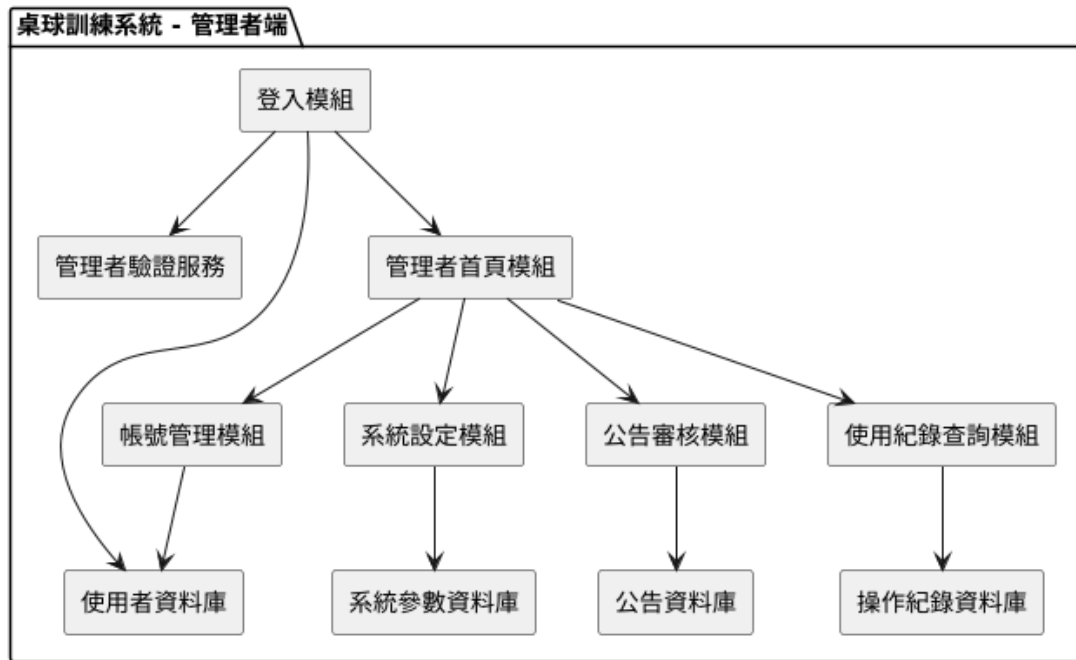


圖7-3-3- 元件圖-Component diagram-管理者端

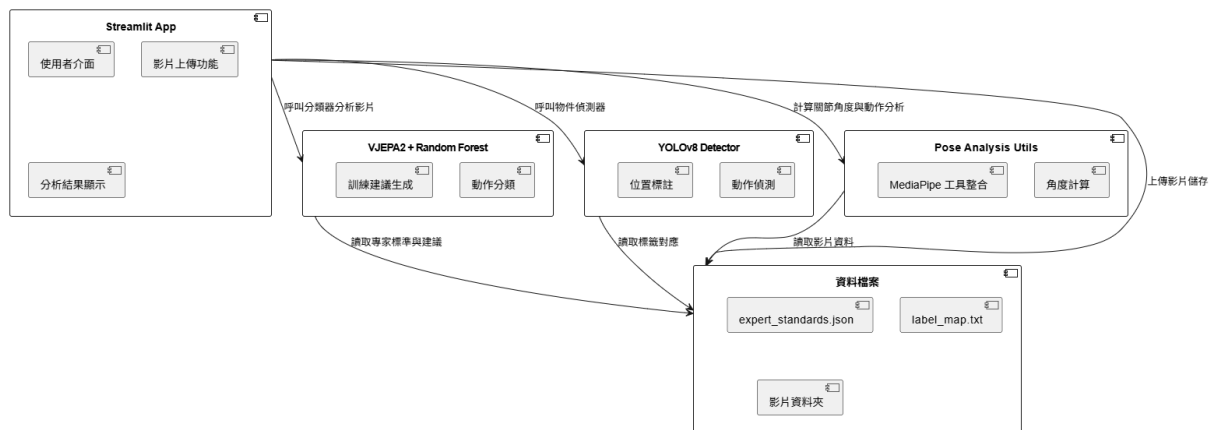


圖7-3-4- 元件圖-Component diagram-AI動作分析模組

7-4 狀態機 (State machine)

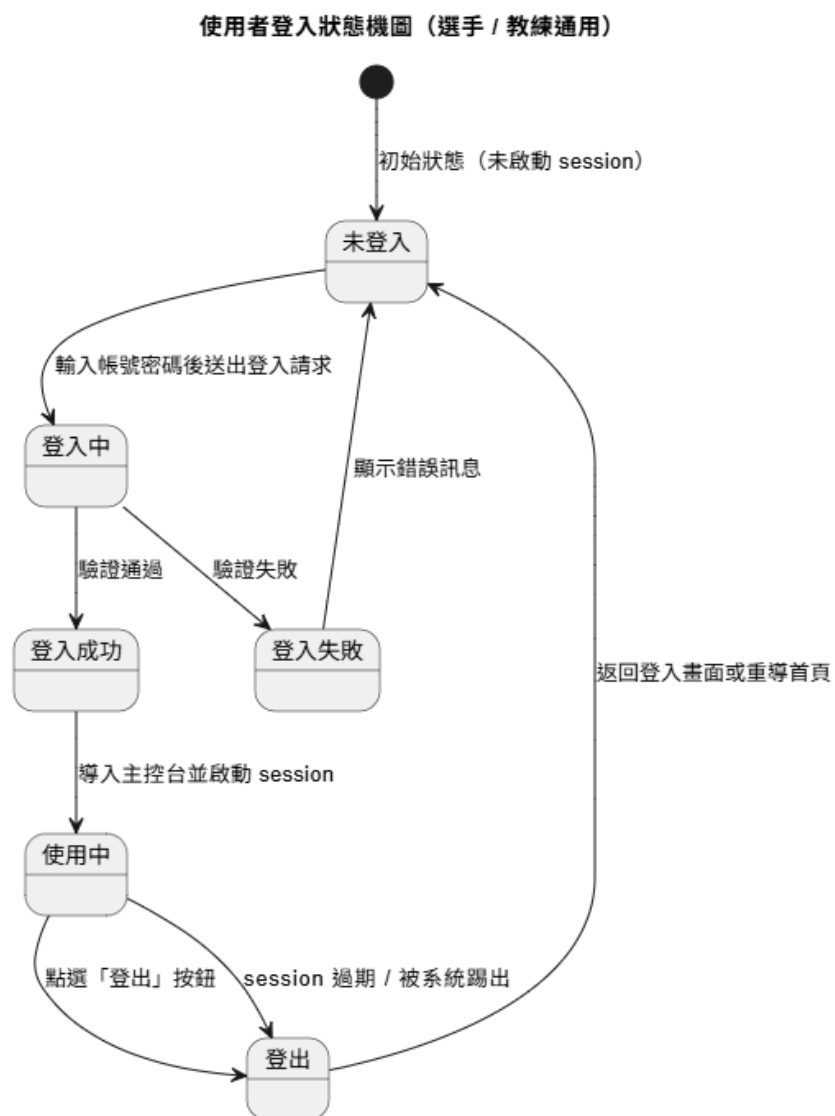


圖7-4-1- 狀態機-State machine-選手教練登入登出

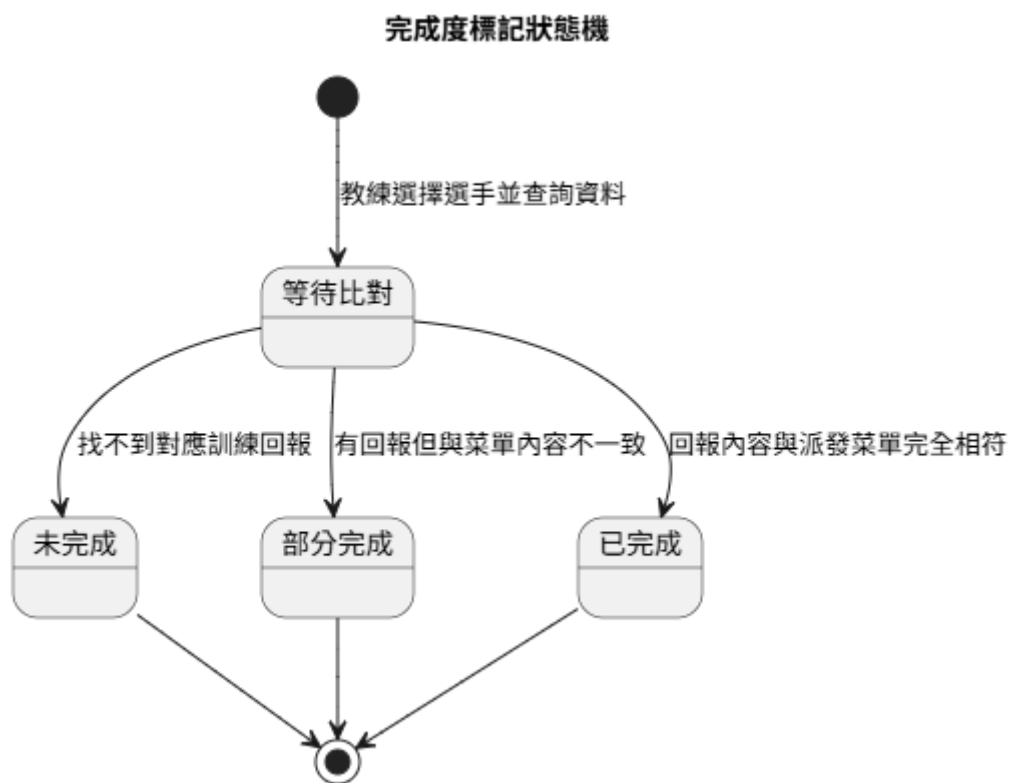
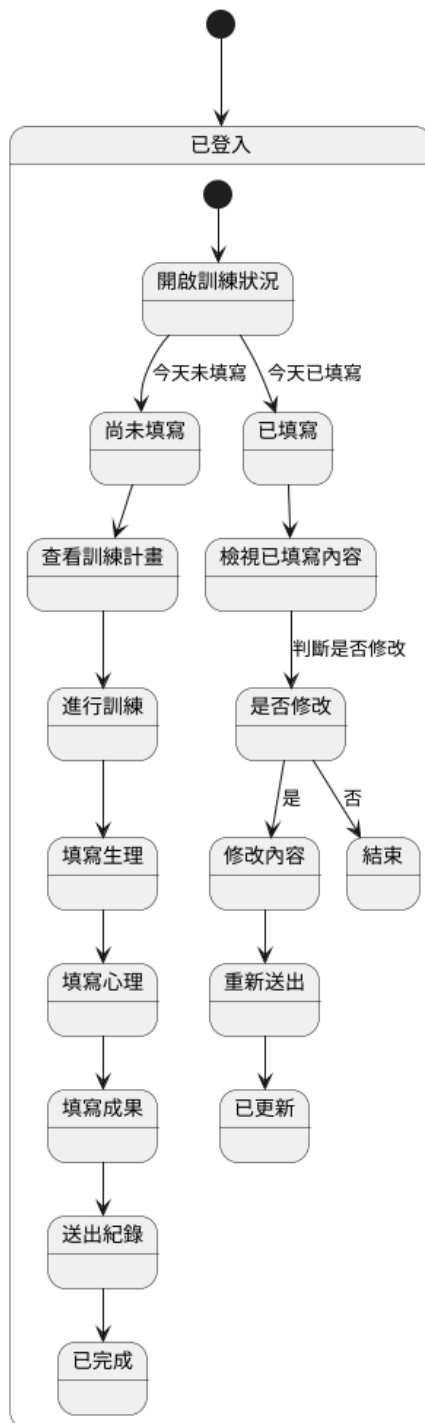
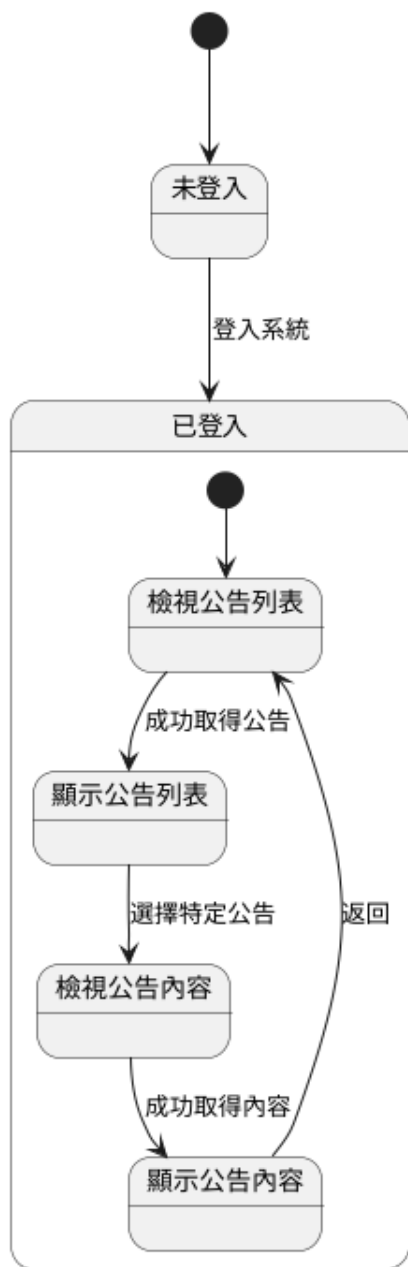


圖7-4-2- 狀態機-State machine-選手訓練完成度



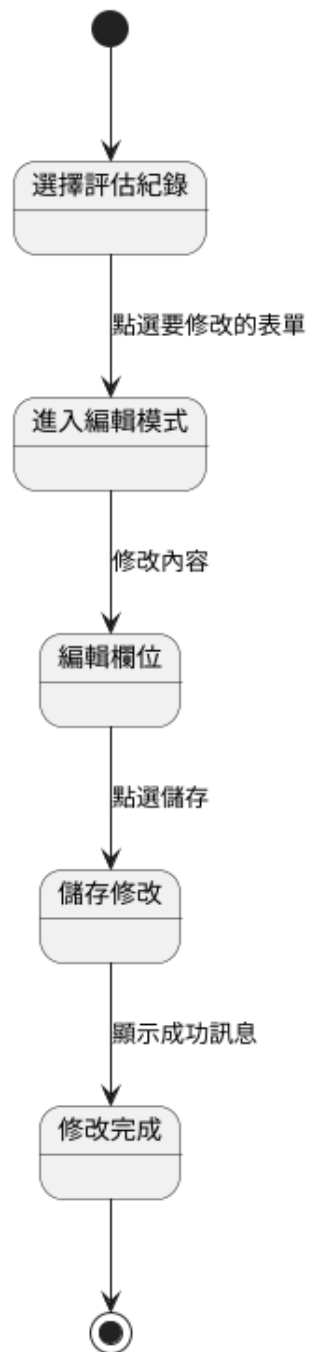


圖7-4-5- 狀態機-State machine-選手編輯狀態表單

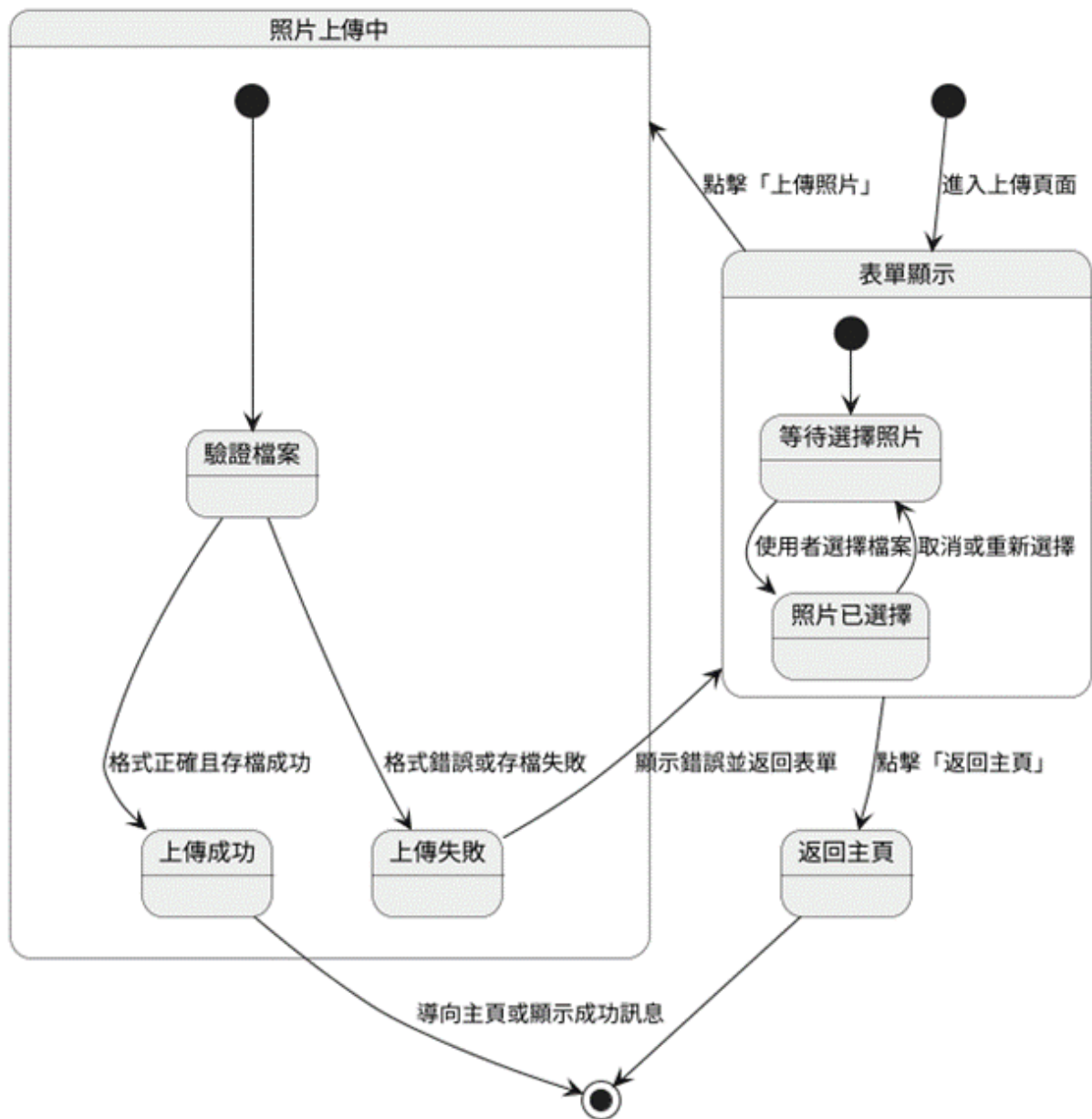


圖7-4-6- 狀態機-State machine- 選手上傳飲食照片

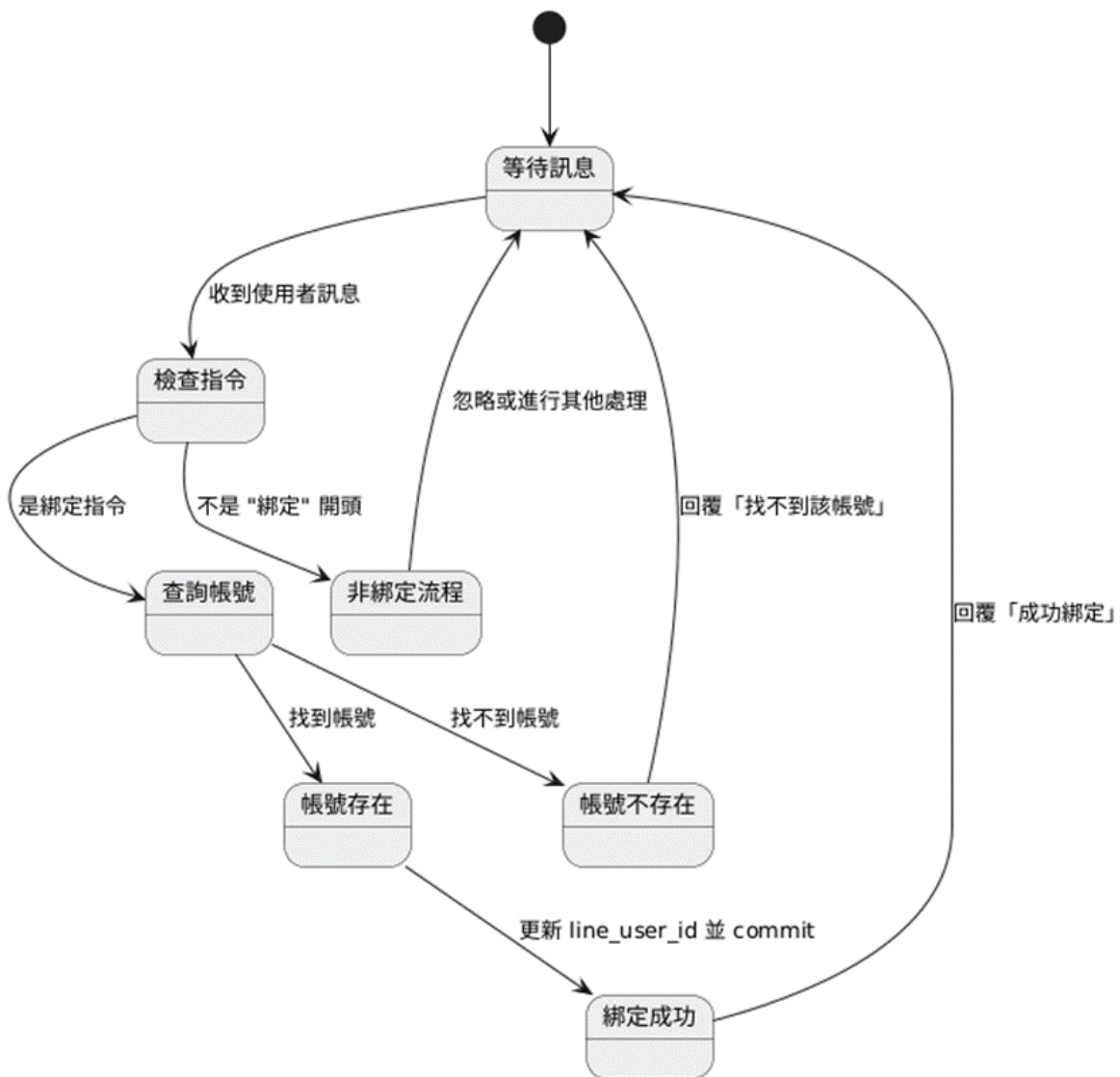


圖7-4-7- 狀態機-State machine- 選手LINE Bot 綁定帳號

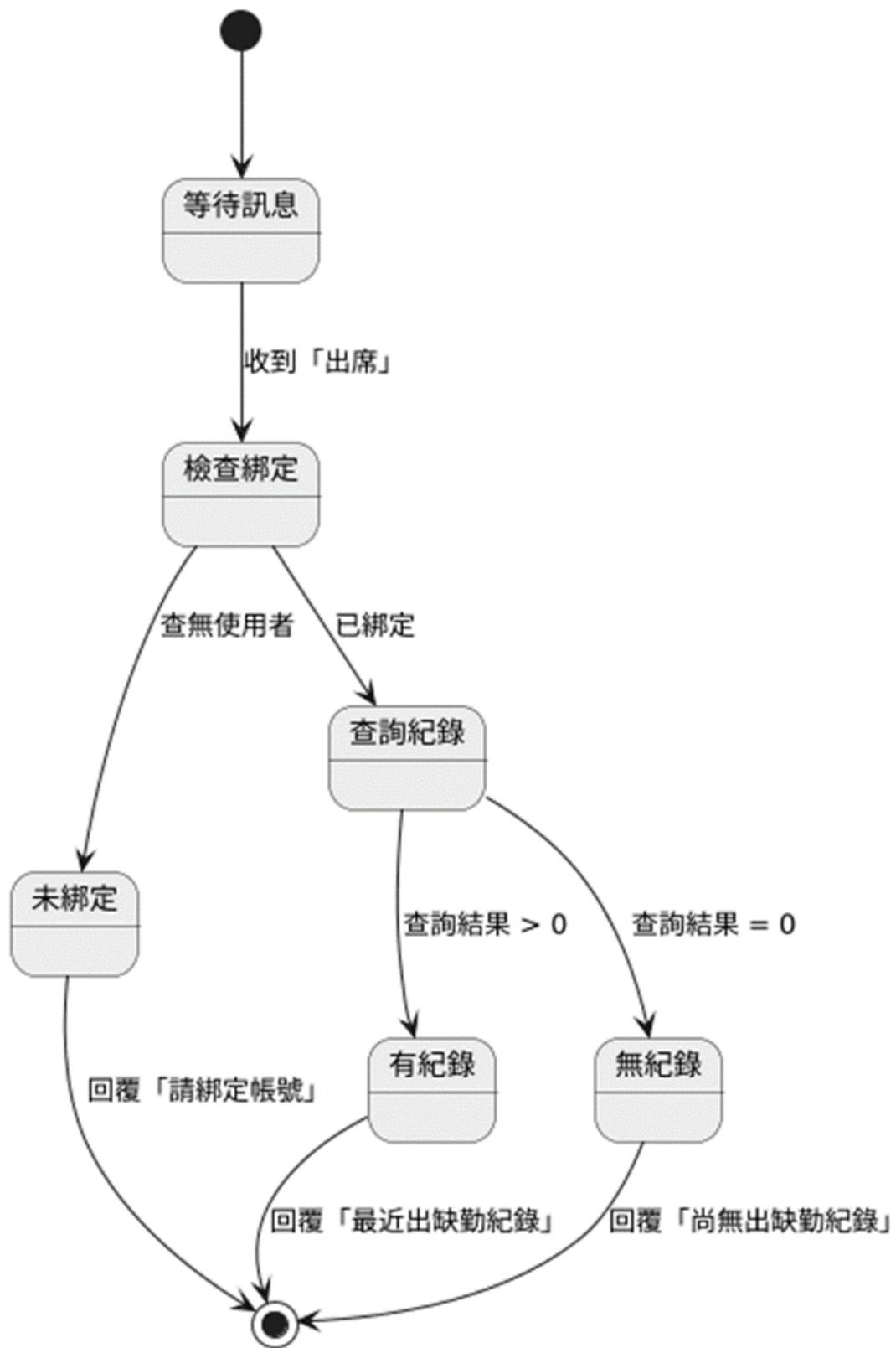


圖7-4-8- 狀態機-State machine- LINE Bot 選手查詢出缺勤紀錄

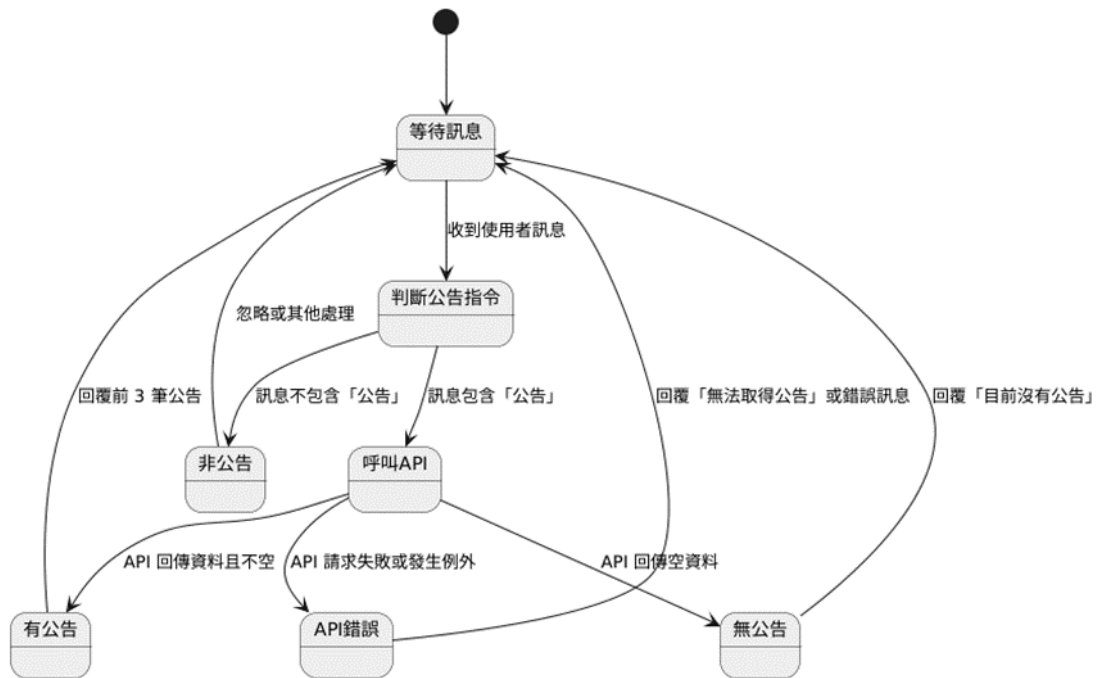


圖7-4-9- 狀態機-State machine- LINE Bot選手公告查詢

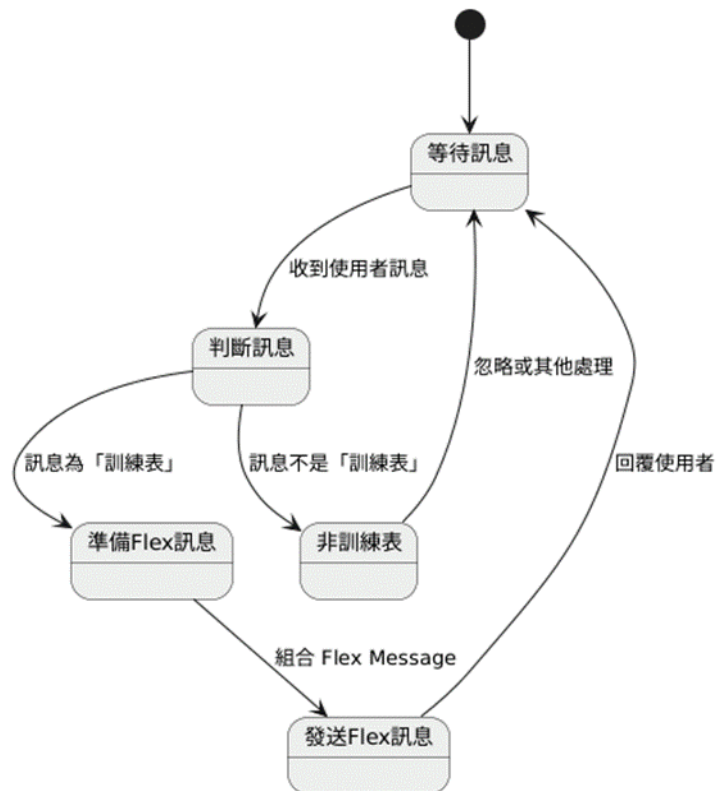


圖7-4-10- 狀態機-State machine- LINE Bot 選手今日訓練狀況

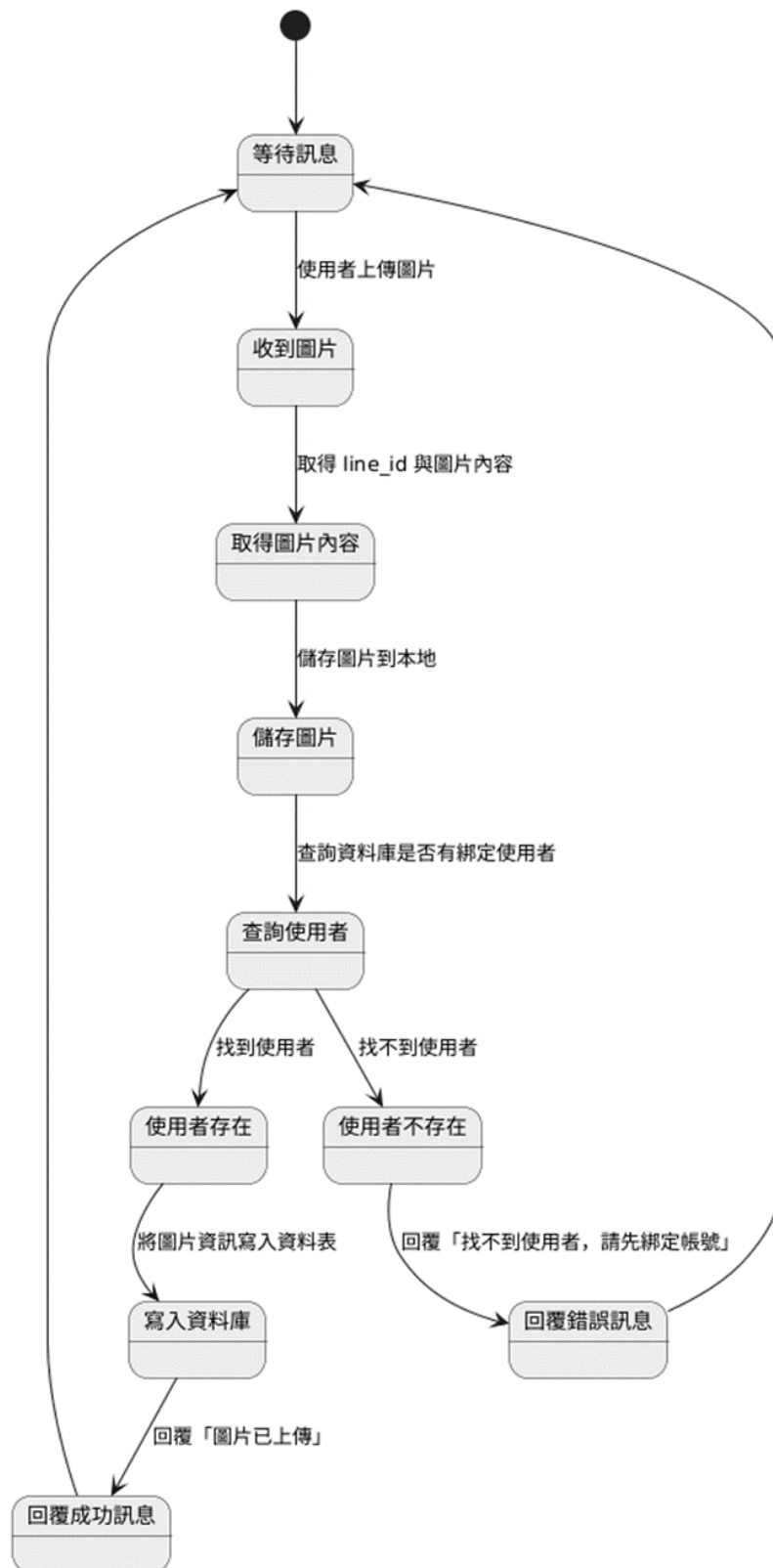


圖7-4-11- 狀態機-State machine- LINE Bot 選手上傳飲食照片

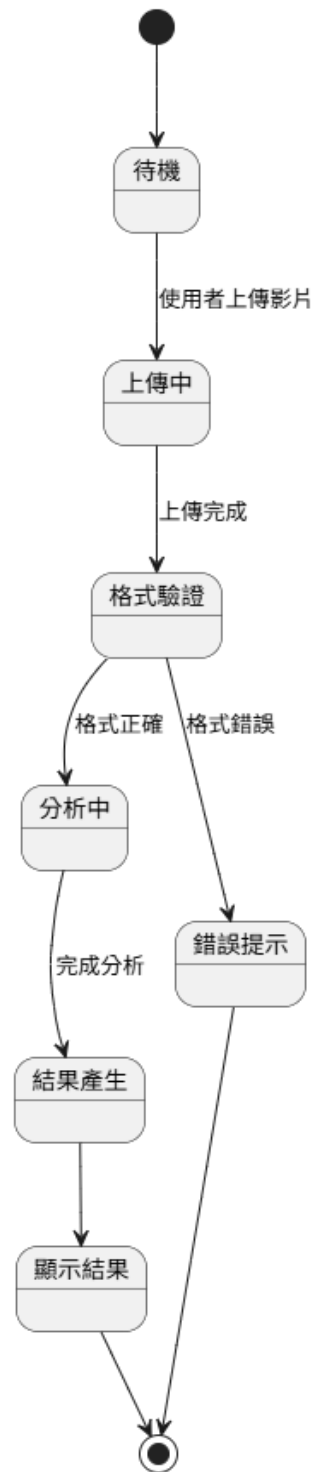


圖7-4-12- 狀態機-State machine- 選手動作分析模組

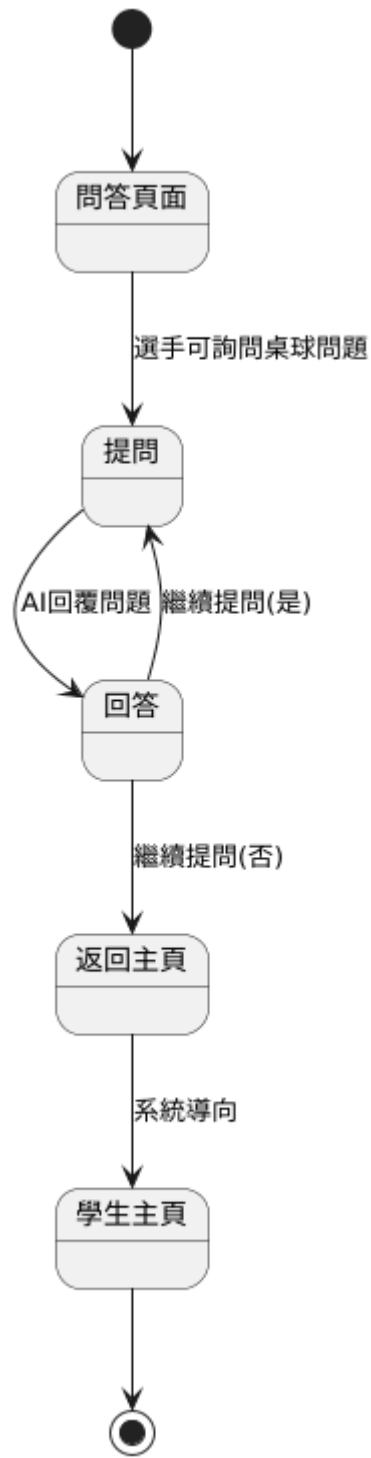


圖7-4-13- 狀態機-State machine- 選手使用智能助手

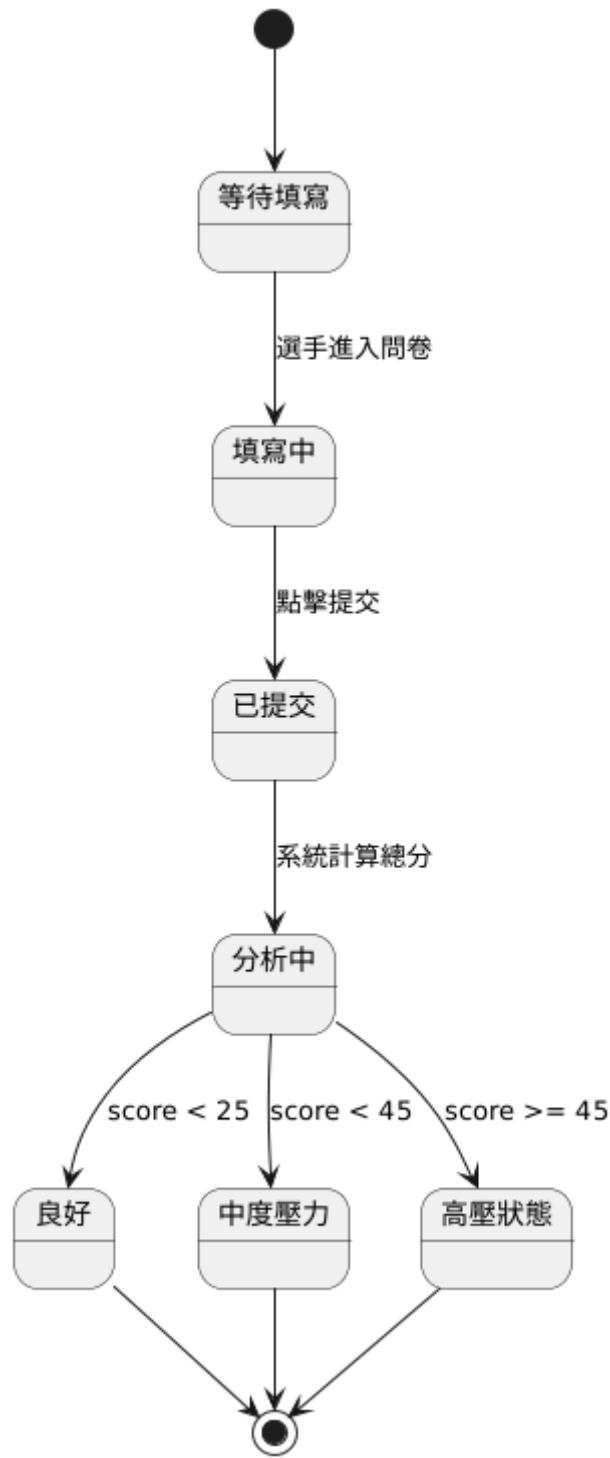


圖7-4-14- 狀態機-State machine- 選手填寫壓力評估

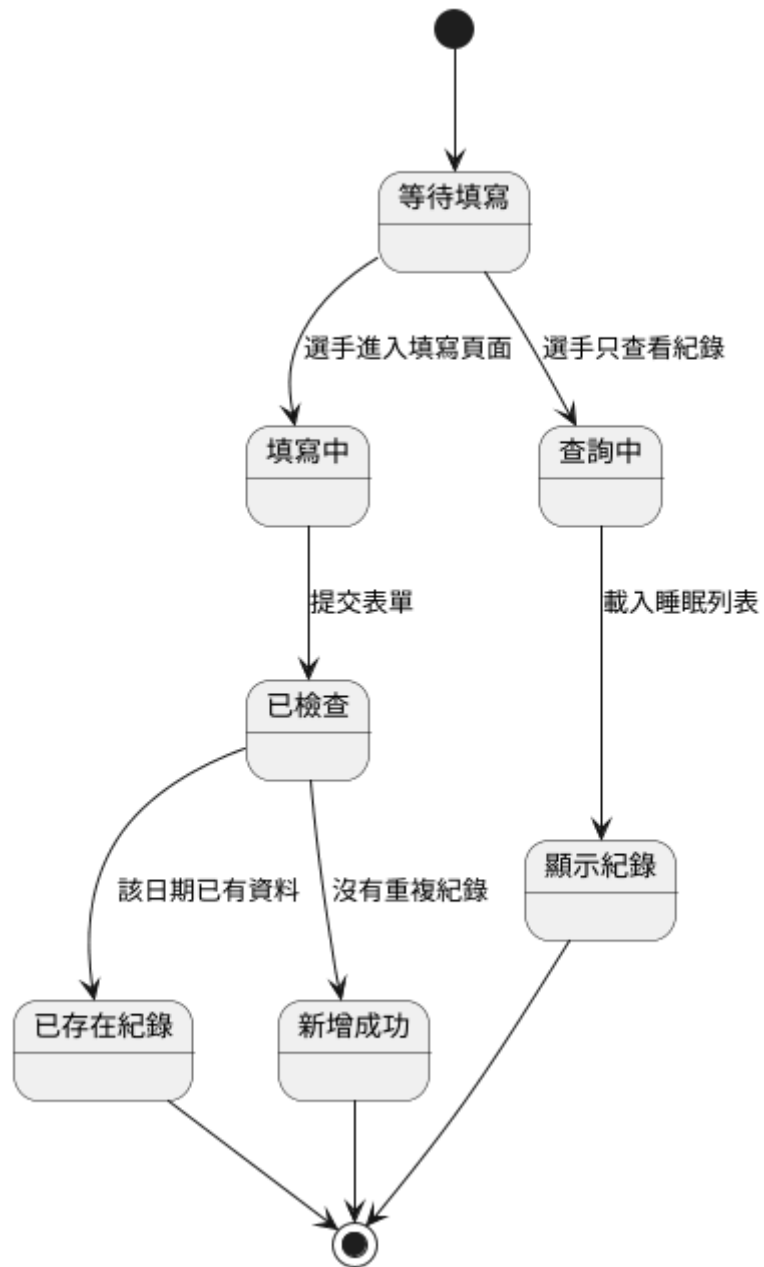


圖7-4-15- 狀態機-State machine- 選手填寫睡眠紀錄

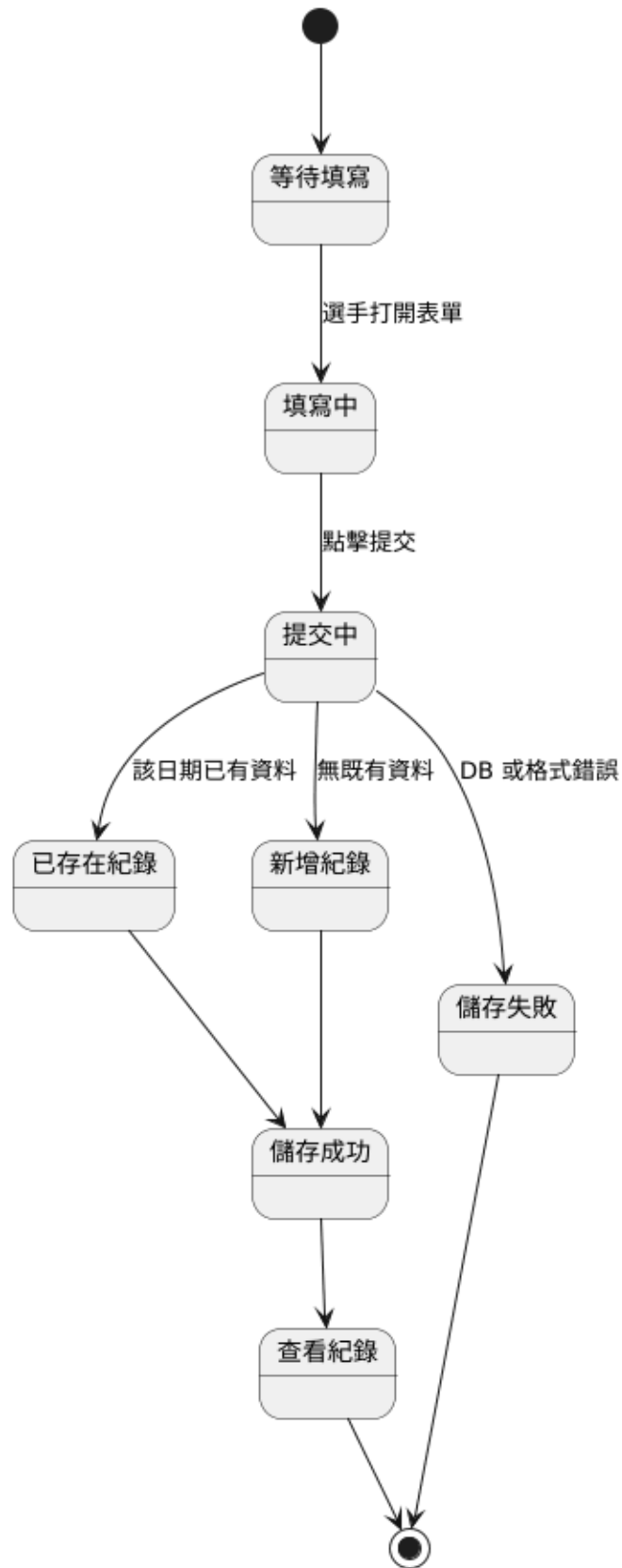


圖7-4-16- 狀態機-State machine- 選手填寫生理評估

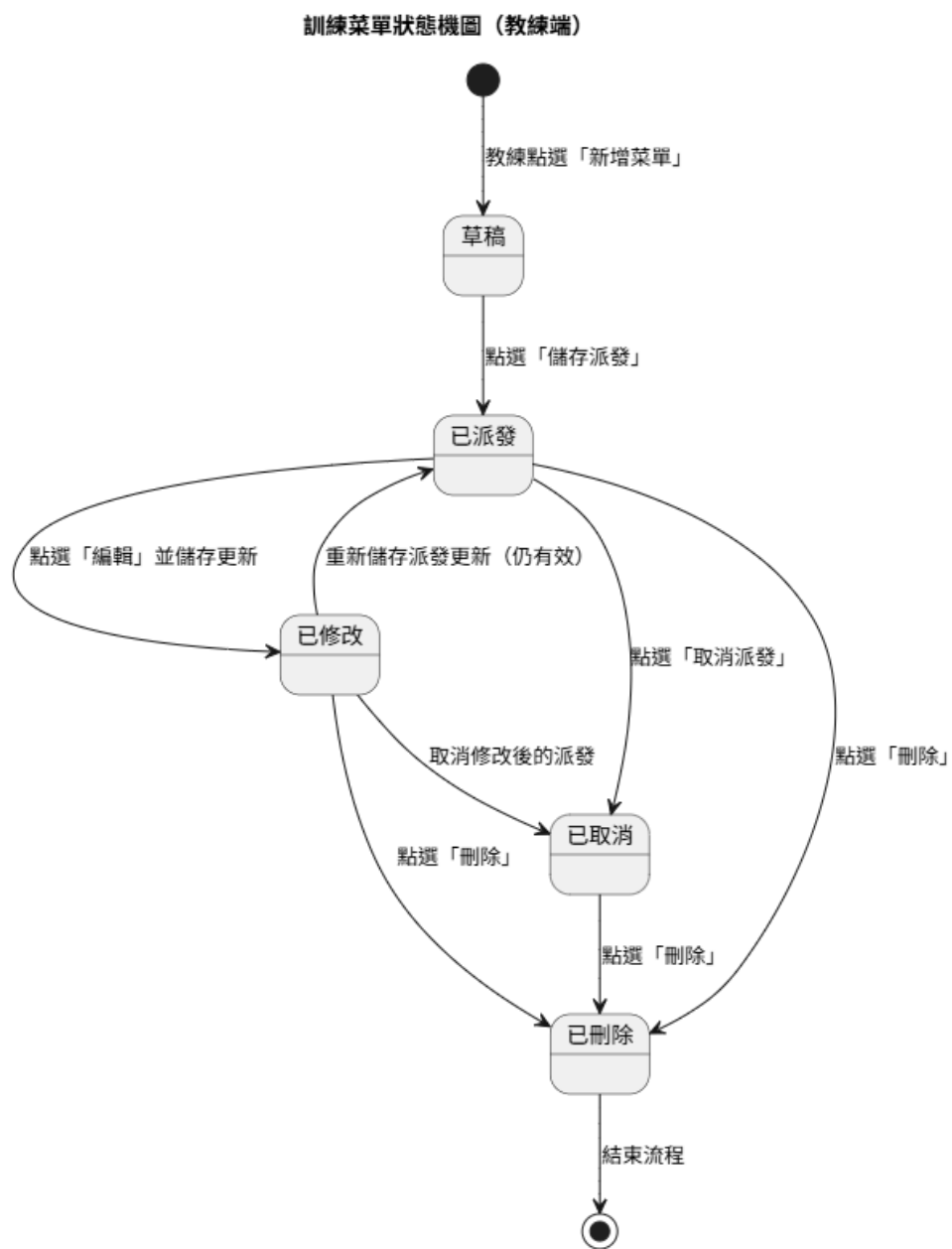


圖7-4-17- 狀態機-State machine-教練派發訓練菜單

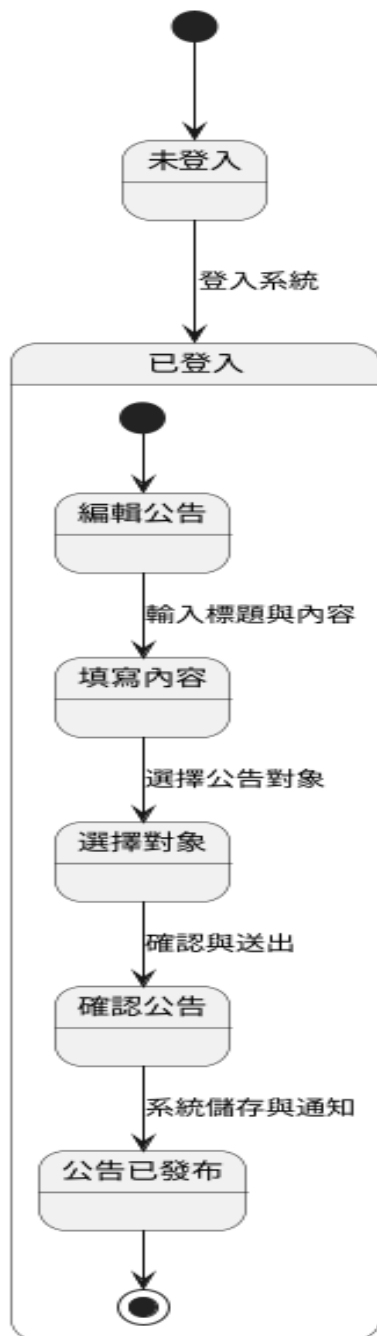


圖7-4-18- 狀態機-State machine- 教練發布公告

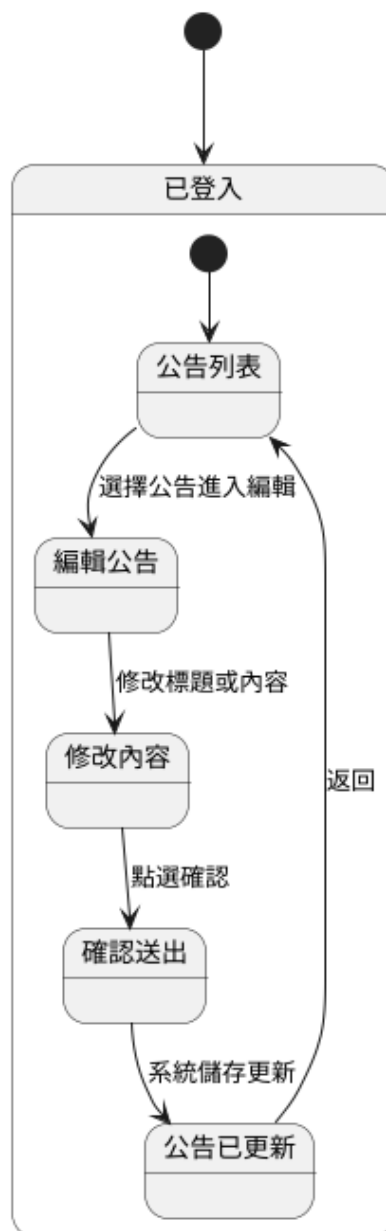


圖7-4-19- 狀態機-State machine- 教練編輯公告

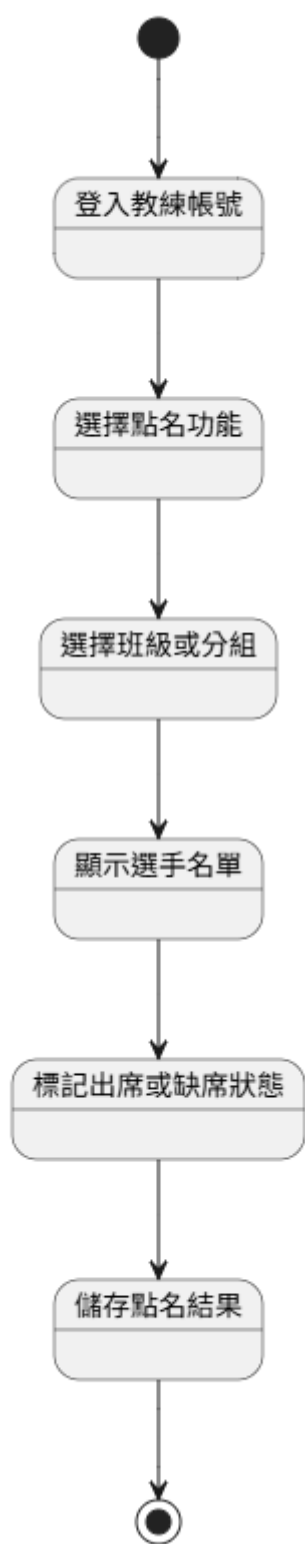


圖7-4-20- 狀態機-State machine-教練點名



圖7-4-21- 狀態機-State machine-教練查看點名紀錄

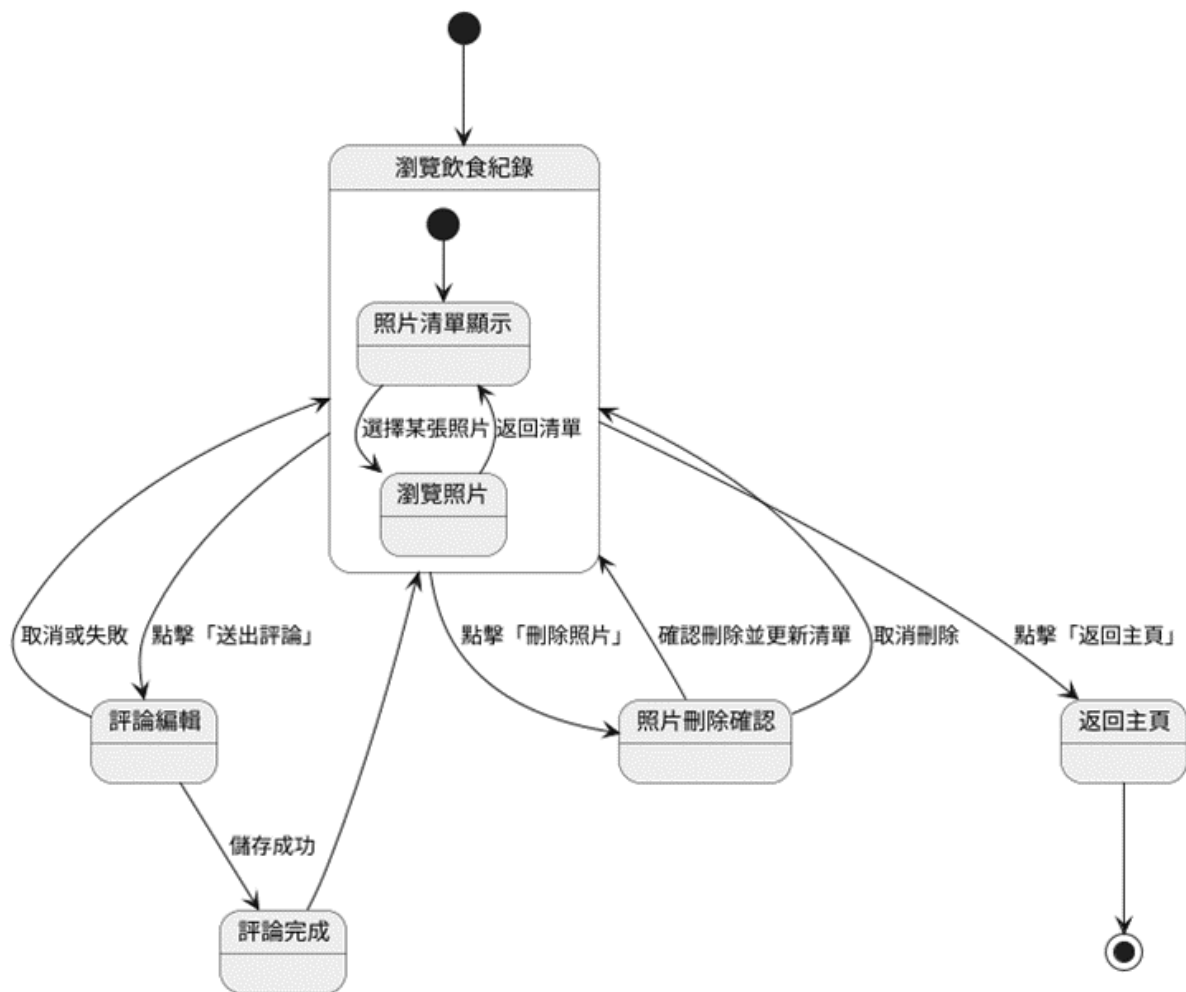


圖7-4-22- 狀態機-State machine- 教練查看飲食紀錄

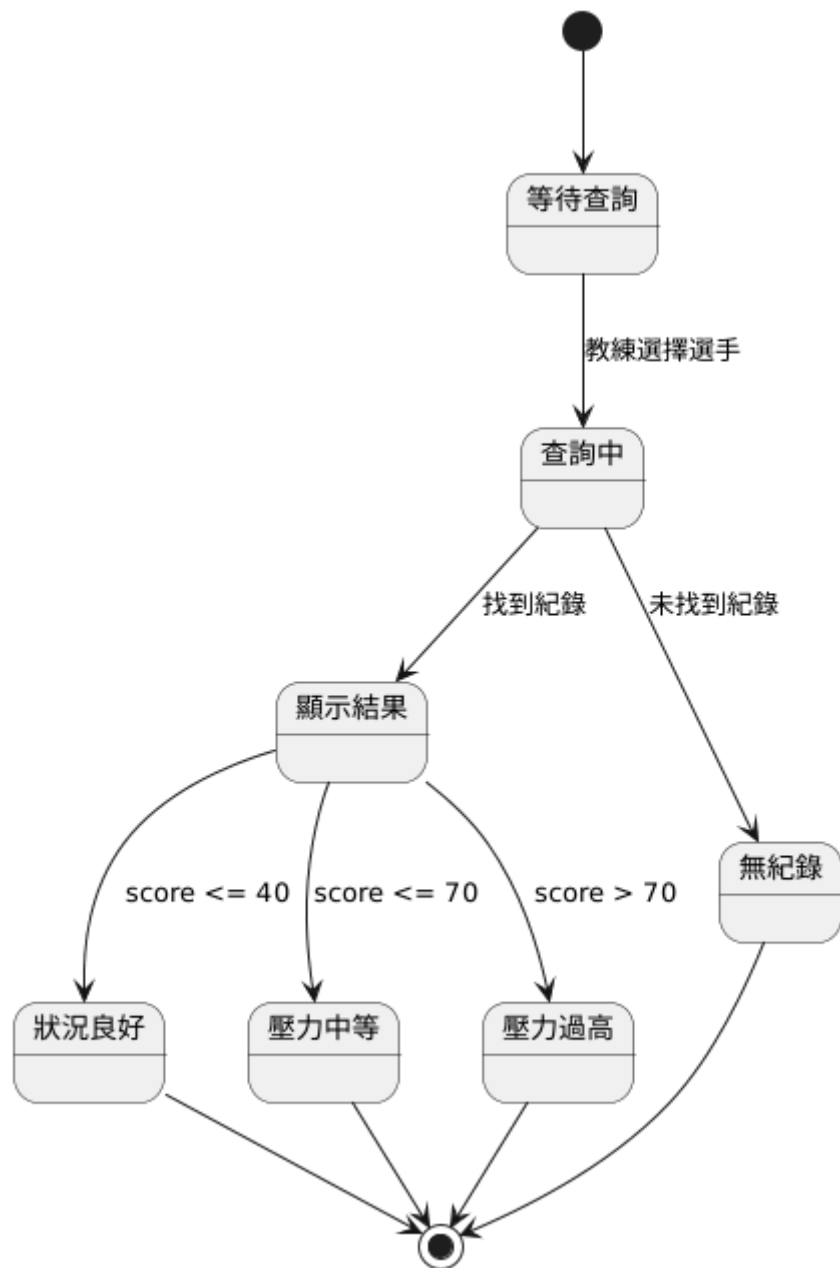


圖7-4-23- 狀態機-State machine- 教練查看壓力評估

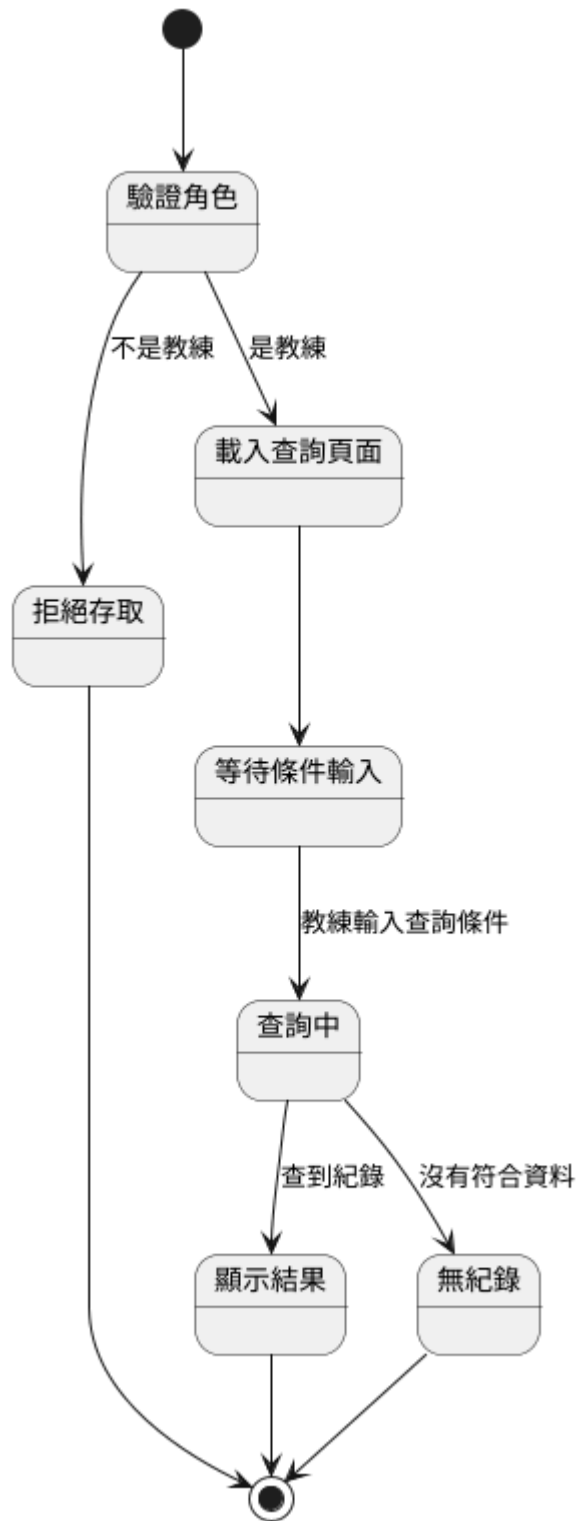


圖7-4-24- 狀態機-State machine- 教練查看睡眠紀錄

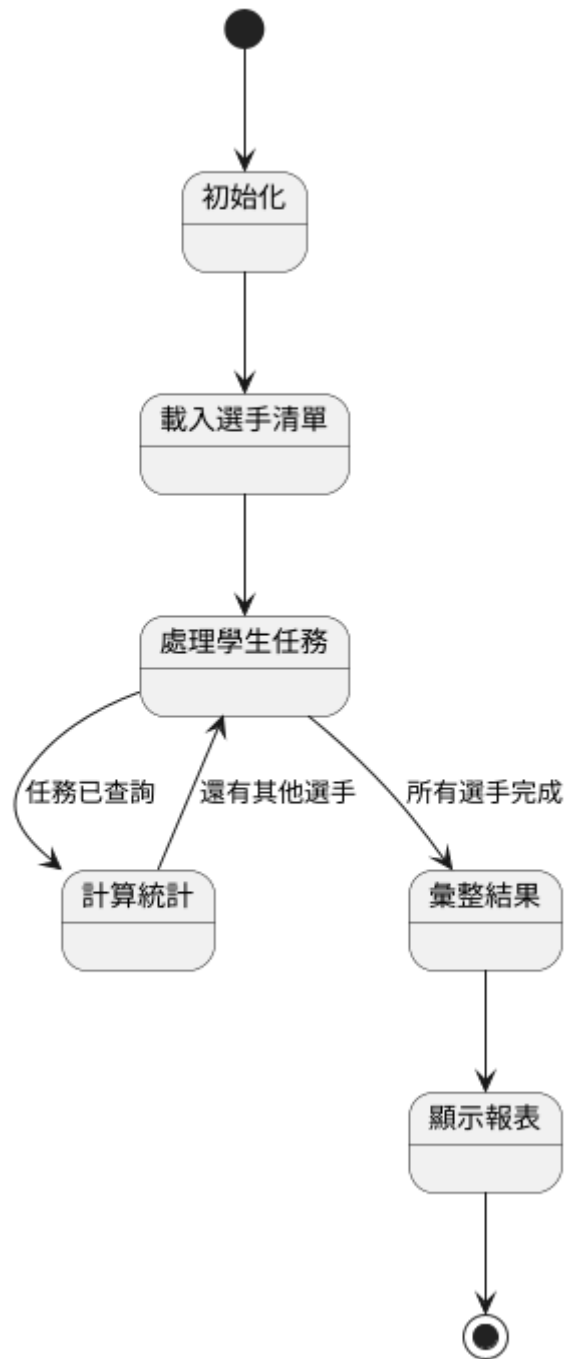


圖7-4-25- 狀態機-State machine- 教練查看生理評估

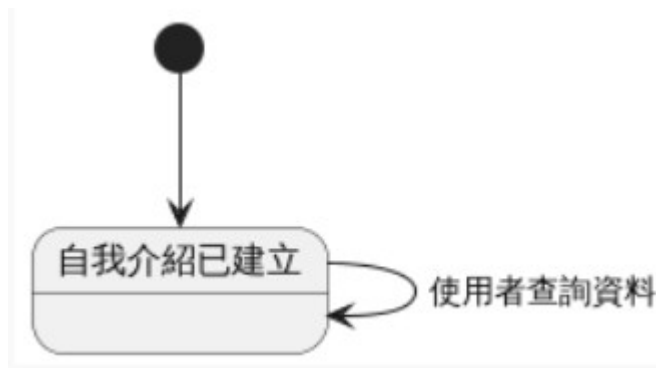


圖7-4-26- 狀態機-State machine- 個人自介查詢

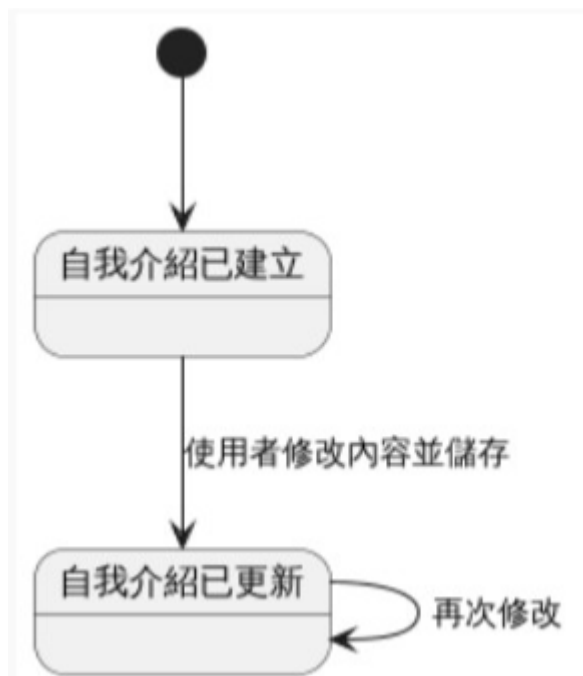


圖7-4-27- 狀態機-State machine- 修改自我介紹

第八章 資料庫設計

8-1 資料庫關聯表

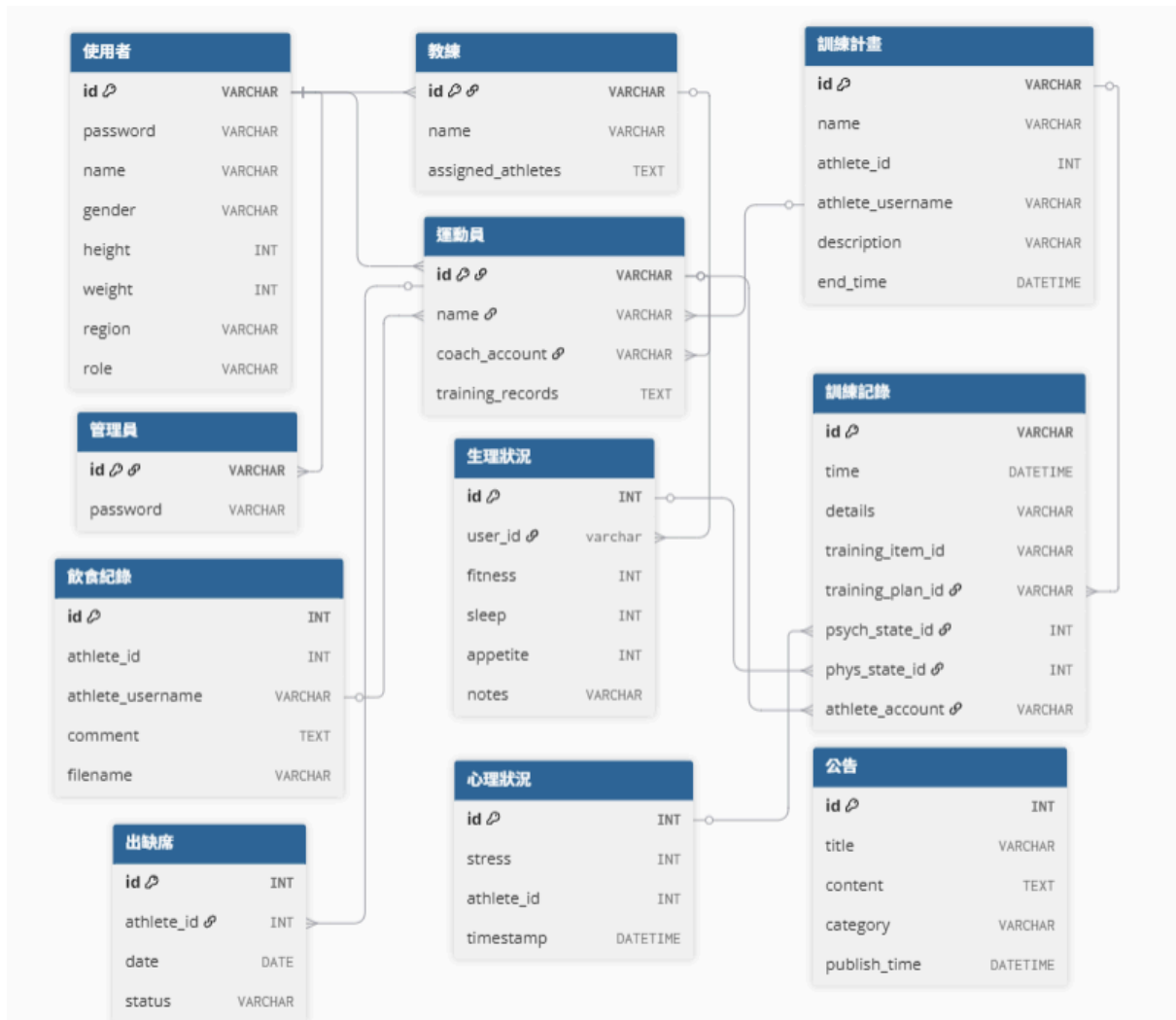


圖8-1-1- 資料庫關聯表

8-2 表格及其Meta data.

資料表	欄位名稱	資料型態	長度	允許空值	鍵
使用者	id	VARCHAR	255	FALSE	PK
使用者	password	VARCHAR	80	FALSE	
使用者	name	VARCHAR	80	FALSE	
使用者	gender	VARCHAR	10	TRUE	
使用者	height	INT		TRUE	
使用者	weight	INT		TRUE	
使用者	region	VARCHAR	80	TRUE	
使用者	role	VARCHAR	20	FALSE	

圖8-2-1- 資料庫描述-使用者

資料表	欄位名稱	資料型態	長度	允許空值	鍵
教練	id	VARCHAR	255	FALSE	PK
教練	name	VARCHAR	80	FALSE	
教練	assigned athlete	TEXT		TRUE	

圖8-2-2- 資料庫描述-教練

資料表	欄位名稱	資料型態	長度	允許空值	鍵
運動員	id	VARCHAR	255	FALSE	PK
運動員	name	VARCHAR	80	FALSE	
運動員	coach_account	VARCHAR	255	TRUE	FK
運動員	training_records	TEXT		TRUE	

圖8-2-3- 資料庫描述-選手

資料表	欄位名稱	資料型態	長度	允許空值	鍵
公告	id	INT		FALSE	PK
公告	title	VARCHAR	100	FALSE	
公告	content	TEXT		FALSE	
公告	category	VARCHAR	20	TRUE	
公告	publish time	DATETIME		FALSE	

圖8-2-4 資料庫描述-公告

資料表	欄位名稱	資料型態	長度	允許空值	鍵
飲食紀錄	id	INT		FALSE	PK
飲食紀錄	athlete_id	INT		FALSE	FK
飲食紀錄	athlete_username	VARCHAR	80	FALSE	
飲食紀錄	filename	VARCHAR	255	FALSE	
飲食紀錄	comment	TEXT		TRUE	

圖8-2-5 資料庫描述-飲食紀錄

資料表	欄位名稱	資料型態	長度	允許空值	鍵
訓練計畫	id	VARCHAR	50	FALSE	PK
訓練計畫	name	VARCHAR	100	FALSE	
訓練計畫	athlete_id	INT		FALSE	FK
訓練計畫	athlete_username	VARCHAR	80	FALSE	
訓練計畫	description	VARCHAR	255	TRUE	
訓練計畫	end_time	DATETIME		TRUE	

圖8-2-6 資料庫描述-訓練計畫

資料表	欄位名稱	資料型態	長度	允許空值	鍵
出缺席	id	INT		FALSE	PK
出缺席	athlete_id	INT		FALSE	FK
出缺席	date	DATETIME		FALSE	
出缺席	status	VARCHAR	255	FALSE	

圖8-2-7 資料庫描述-出缺席

資料表	欄位名稱	資料型態	長度	允許空值	鍵
訓練記錄	id	VARCHAR	50	FALSE	PK
訓練記錄	time	DATETIME		FALSE	
訓練記錄	details	VARCHAR	255	TRUE	
訓練記錄	training_item_id	VARCHAR	20	TRUE	
訓練記錄	training_plan_id	VARCHAR	50	TRUE	FK
訓練記錄	psych_state_id	INT		TRUE	FK
訓練記錄	phys_state_id	INT		TRUE	FK
訓練記錄	athlete_account	VARCHAR	255	FALSE	FK

圖8-2-8 資料庫描述-訓練紀錄

資料表	欄位名稱	資料型態	長度	允許空值	鍵
心理狀況	id	INT		FALSE	PK
心理狀況	stress	INT		TRUE	
心理狀況	athlete_id	INT		FALSE	FK
心理狀況	timestamp	DATETIME		TRUE	

圖8-2-9 資料庫描述-心理狀況

資料表	欄位名稱	資料型態	長度	允許空值	鍵
生理狀況	id	INT		FALSE	PK
生理狀況	user_id	VARCHAR	50	FALSE	FK
生理狀況	fitness	INT		TRUE	
生理狀況	sleep	INT		TRUE	
生理狀況	appetite	INT		TRUE	
生理狀況	notes	VARCHAR	255	TRUE	

圖8-2-10 資料庫描述-生理狀況

資料表	欄位名稱	資料型態	長度	允許空值	鍵
管理員	id	VARCHAR	255	FALSE	PK
管理員	password	VARCHAR	80	FALSE	

圖8-2-11 資料庫描述-管理員

第9章 程式

9-1 元件清單及其規格描述

AI相關：

1. 正反手動作辨識：利用 VJEPa + Random Forest 模型，自動判斷影片中的動作是正手（forehand）還是反手（backhand）。
2. 動作分析與建議：透過 YOLOv8 偵測物件，搭配 MediaPipe 和 OpenCV 進行姿勢分析，並根據自訂義的專家標準給予回饋。
3. 客製化回饋：使用者可以自由定義專家數據與建議內容，讓回饋更符合個人需求。

* 動作分類：VJEPa(特徵提取) + Random Forest(分類)

* 物件偵測：YOLOv8 * 姿勢分析：MediaPipe + OpenCV

* 使用者介面：Streamlit

4. 聊天問答 (QA)

使用者以自然語言提問桌球技術、訓練、策略、規則等。

系統用 ChatGPT API 回覆（含引用 / 建議 / 範例訓練菜單）。

功能：

1. 心理紀錄表單

使用者可定期填寫心理狀態紀錄（壓力、專注力、焦慮程度等），

系統會進行分析，若有異常則提醒教練或心理師。

2. 睡眠紀錄與運動表現分析

紀錄選手的睡眠時長與品質，並分析其對運動表現的影響，

提供改善建議。

3. 教練分派訓練內容

教練可透過系統建立並分派個別化的訓練課表，

選手依照課表進行訓練與操課，達到提升桌球運動表現的目標。

4. 生理表單與狀態趨勢分析

選手可填寫每日的生理紀錄（如心率、疲勞程度、肌肉痠痛等），

系統會自動生成可視化圖表，讓選手能一目了然掌握身體狀態趨勢。

5. 飲食控制紀錄

使用者可上傳每日飲食照片，

系統會將圖片儲存並與使用者帳號綁定，

方便教練或營養師追蹤飲食內容，

並提供建議以協助改善營養攝取與體能表現。

6. Line Bot 即時互動

系統整合 Line Bot，提供自然語言對話功能，

使用者可透過關鍵字（如「公告」、「訓練表」）獲取相關資訊，

包含最新公告、訓練課表、或評估表單填寫連結，

提升使用者操作便利性與即時性。

7. 出缺勤點名管理

教練可透過系統進行選手出缺勤紀錄管理，

系統會自動建立出勤紀錄，

協助教練快速掌握隊員出席情況，作為訓練管理依據。

8. 公告發布與推播

教練或管理員可於系統後台新增公告，

包含訓練時間異動、比賽資訊、注意事項等，

確保選手能即時接收到最新消息，提升資訊傳達效率。

▼表9-1-1 主要檔案清單

模型類別	檔案 / 功能	說明
App 主程式	<code>vjepa_streamlit.py</code>	提供使用者上傳影片入口；整合影片轉檔、動作分類、YOLO 偵測與姿勢分析。
工具 (utils/)	<code>video_converter.py</code>	將上傳影片轉換為標準 MP4 格式。
	<code>data_loader.py</code>	載入標籤對應檔 (<code>label_map.txt</code>) 與專家標準設定檔 (<code>expert_standards.json</code>)。
模型 (models/)	<code>vjepa_classifier.py</code>	整合 V-JEPA 特徵與 Random Forest 進行桌球動作分類。

▼表9-1-2 主要模組清單

模型/檔案	功能說明	依賴	備註
<code>vjepa_streamlit.py</code> (app.py)	Streamlit 前端主程式，負責影片上傳、顯示、呼叫模型進行分類、YOLO 偵測與姿勢分析	<code>streamlit</code> , <code>tempfile</code> , <code>os</code> , 自訂 <code>utils</code> 模組、 <code>models</code>	提供完整使用者介面與互動
<code>utils/video_converter.py</code>	影片格式轉換為 mp4	<code>ffmpeg</code> 或 <code>moviepy</code>	兼容不同影片格式
<code>utils/data_loader.py</code>	讀取 <code>label_map</code> 與專家標準 JSON 配置	<code>json</code>	<code>load_label_map</code> 、 <code>load_all_config</code>
<code>models/vjepa_classifier.py</code>	V-JEPA 特徵提取 + 動作分類器，提供預測功能	<code>torch</code> , <code>transformers</code> , <code>joblib</code> , <code>torchvision</code>	載入 <code>vjepa_classifier_overall_best.pt</code> ，可抽取特徵與預測
<code>models/yolo_v8_detector.py</code>	YOLOv8 物件偵測	<code>ultralytics</code> , <code>torch</code> , <code>cv2</code> , <code>torchvision</code>	支援每 N 幀偵測，回傳帶標註影像

<code>pose_analysis/angle_calculator.py</code>	運動姿勢分析與角度計算	<code>pandas,</code> <code>numpy</code>	搭配專家標準 JSON 分析正 反手姿勢
<code>training/train_classifier.py</code>	訓練分類器模型 (RF / SVM / XGBoost)，並選擇 整體最佳模型	<code>sklearn,</code> <code>xgboost,</code> <code>torch,</code> <code>transformers,</code> <code>joblib</code>	儲存整體最佳 模型 <code>vjepa_classifier_overall_best.pt</code> ，並生成 <code>label_map.txt</code>
<code>Chat.py</code>	使用者以自然語言 提問桌球技術、訓 練、策略、規則 等。	<code>OpenAI</code>	提供使用者問 答
<code>Stress_evaluate.py</code>	使用者可定期填寫 心理狀態紀錄	<code>Traceback</code> <code>flask</code>	
<code>Athlete.py</code>	紀錄選手的睡眠時 長與品質，並分析 其對運動表現的影 響	<code>Datetime</code> <code>Timedelta</code> <code>flask</code>	
<code>Training.py</code>	教練可透過系統建 立並分派個別化的 訓練課表	<code>Datetime</code> <code>flask</code>	

Evaluation.py	選手可填寫每日的生理紀錄（如心率、疲勞程度、肌肉痠痛等），系統會自動生成可視化圖表	Datetime flask	
Line_bot.py	使用者可利用linebot來進行各項操作	LINE Bot SDK LINE Messaging API Flask	多功能的整合 給使用者更多 媒介
Food.py	使用者可定期上傳飲食照片，以便教練追蹤	Datetime Flask MySQL	
coach.py	紀錄選手的出缺席和公告	Datetime Flask MySQL	

9-2 其他附屬之各種元件

▼表9-2-1 主要程式清單

模組/功能	功能說明	主要檔案/技術	備註
使用者介面	提供影片上傳、分析進度提示及結果顯示	Streamlit (vjepa_streamlit.py)	支援 avi, mp4, mov, mkv, flv 格式
影片轉檔	將上傳影片轉為標準 MP4 格式	utils/video_converter.py	保證後續模型可讀取
標籤與配置	標籤映射與專家動作標準設定	data/label_map.txt , data/expert_standards.js on	用於分類與姿勢比對
特徵萃取	從影片抽取 V-JEPA 特徵	models/vjepa_classifier.py	支援幀數填充，GPU 加速
動作分類	使用 Random Forest 或其他 ML 模型進行動作分類	models/vjepa_classifier.py	顯示預測結果與各類別機率
物件偵測	偵測桌球及球員位置	models/yolov8_detector.py (YOLOv8)	每隔固定幀數偵測，輸出影像結果

姿勢分析	計算手臂與身體角度，評估動作標準	<code>pose_analysis/angle_calculator.py</code> , <code>pose_analysis/mediapipe_utils.py</code>	比對專家標準，生成姿勢分析報告
系統效能	暫存檔案管理，GPU 加速，錯誤處理	-	分析完成後自動清理，提供錯誤提示

9-3 程式碼

A.Streamlit 主程式 (影片上傳與整合分析)

功能：使用者上傳影片，顯示影片，並呼叫動作分類模組做預測。

1.影片上傳與整合分析

```

1. uploaded_file = st.file_uploader("請上傳影片檔")
2. type=["avi", "mp4", "mov"])
3.     if uploaded_file is not None:
4.         with
5.             tempfile.NamedTemporaryFile(delete=False, suffix=".mp4") as tmp_file:
6.                 tmp_file.write(uploaded_file.read())
7.                 video_path =
8.                 tmp_file.name
9.                 st.video(video_path)
10.            # 預測動作
11.            predicted_label =
12.            predict_video(video_path, processor, base_model, classifier_model, label_map)
13.            st.success(f"預測動作為:
14.            {predicted_label}")

```

2.V-JEPA + Random Forest 動作分類

功能：從影片中抽取特徵，使用已訓練的分類器做動作辨識。

```
1. def predict_video(video_path, processor, base_model,
2. classifier_model, label_map):
3.     video_tensor, _, _ =
4.     io.read_video(video_path, pts_unit='sec')
5.     clip = video_tensor[:16] # 取前16幀
6.     features = extract_features(clip,
7. processor, base_model)
8.     pred_id =
9.     classifier_model.predict(features)[0]
10.     return label_map.get(pred_id, "未知動作")
```

3.YOLOv8 物件偵測

功能：偵測球與球員位置，回傳標註後影像。

```
1. def detect_objects_yolo(video_path, yolo_model):
2.     video_tensor, _, _ =
3.     io.read_video(video_path, pts_unit='sec')
4.     frame =
5.     video_tensor[0].numpy() # 取第一幀
6.     results = yolo_model(frame)
7.     annotated = results[0].plot()
8.     return annotated
```


4. 姿勢分析 (角度計算)

功能：計算手臂與身體關節角度，對比專家標準。

```
1. def analyze_pose_angles(pose_df, all_config,
2.     predicted_label):
3.     for joint in ['elbow', 'shoulder']:
4.         angles =
5.             pose_df[joint + '_angle']
6.             standard =
7.                 all_config[predicted_label][joint]
8.             print(f"{joint}
9.             平均角度: {angles.mean():.1f}, 專家標準: {standard}")
```

B. AI聊天機器人主程式：

```
1. @chat_bp.route('/chat', methods=['POST'])
2. def chat():
3.     data = request.get_json(force=True)
4.     user_message = data.get('message', '')
5.
6.     try:
7.         completion = client.chat.completions.create(
8.             model="gpt-4o-mini",
9.             messages=[
10.                 {"role": "system", "content": "你是一位桌球教練，請用簡單中文回答"},
11.                 {"role": "user", "content": user_message}
12.             ]
13.         )
14.         reply = completion.choices[0].message.content
15.         return jsonify({'reply': reply})
16.     except Exception as e:
17.         # 印出錯誤，方便除錯
18.         print(f"Error in chat API: {e}")
19.         return jsonify({'reply': '伺服器發生錯誤，請稍後再試。'}), 500
```

C. 心理壓力主程式：

```
1. @stress_bp.route("/evaluate", methods=["POST"])
2. @login_required
3. def submit_form():
4. try:
5.     data = {f"q{i}": int(request.form.get(f"q{i}", 0)) for i in range(1, 18)}
6.     total_score = sum(data.values())
7.
8.     if total_score < 25:
9.         result = "壓力狀況良好"
10.    elif total_score < 45:
11.        result = "中度壓力，建議多休息與自我調適"
12.    else:
13.        result = "高壓狀態，請考慮尋求專業協助"
14.
15.    new_record = StressEvaluate(athlete_id=current_user.id, **data)
16.    with get_db() as db:
17.        db.add(new_record)
18.        db.commit()
```

D. 睡眠分析程式：

```
1. @athlete_bp.route('/sleep_record', methods=['GET', 'POST'])
2. @login_required
3. def sleep_record():
4. if current_user.role != 'athlete':
5.     flash("只有選手可以使用此功能", "danger")
6.     return redirect(url_for('main.index'))
7.
8. with get_db() as db:
9.     # 新增紀錄
10.    if request.method == 'POST':
11.        record_date = request.form['record_date']
12.        sleep_start = request.form['sleep_start']
```

```

13.     sleep_end = request.form['sleep_end']
14.
15.     # 防止重複新增
16.     existing = db.query(SleepRecord).filter_by(
17.         athlete_id=current_user.id,
18.         record_date=record_date
19.     ).first()
20.     if existing:
21.         flash("你已經填過該日期的睡眠紀錄", "warning")
22.     else:
23.         record = SleepRecord(
24.             athlete_id=current_user.id,
25.             record_date=record_date,
26.             sleep_start=sleep_start,
27.             sleep_end=sleep_end,
28.             created_by=current_user.id
29.         )
30.         db.add(record)
31.         db.commit()
32.         flash("睡眠紀錄已新增", "success")
33.
34.     return redirect(url_for('athlete.sleep_record'))
35.
36.     # 查詢自己的紀錄
37.     records = db.query(SleepRecord).filter_by(
38.         athlete_id=current_user.id
39.     ).order_by(SleepRecord.record_date.desc()).all()
40.
41.     # 計算睡眠時長
42.     for r in records:
43.         try:
44.             # r.sleep_start 和 r.sleep_end 已經是 datetime.time
45.             start_dt = datetime.combine(r.record_date, r.sleep_start)
46.             end_dt = datetime.combine(r.record_date, r.sleep_end)
47.
48.             # 跨日處理：睡到隔天
49.             if end_dt <= start_dt:
50.                 end_dt += timedelta(days=1)
51.

```

```
52.         # 計算睡眠時長
53.         r.sleep_duration = end_dt - start_dt # datetime.timedelta
```

E. 教練分派訓練程式碼:

```
1. @training_bp.route('/today/save', methods=['POST'])
2. @login_required
3. def save_today_training():
4.     selected_date = request.form.get("selected_date")
5.     try:
6.         date_obj = datetime.strptime(selected_date, "%Y-%m-%d").date()
7.     except Exception as e:
8.         flash("日期格式錯誤", "danger")
9.         return redirect(url_for("training.training_today"))
10.    try:
11.        with get_db() as db:
12.            record = db.query(Training).filter_by(user_id=current_user.id, date=date_obj).first()
13.            if not record:
14.                record = Training(user_id=current_user.id, date=date_obj)
15.                record.jump_type = request.form.get("jump_type")
16.                record.jump_count = request.form.get("jump_count")
17.                record.run_distance = request.form.get("run_distance")
18.                record.run_time = request.form.get("run_time")
19.                record.weight_sets = request.form.get("weight_sets")
20.                record.agility_type = request.form.get("agility_type")
21.                record.agility_note = request.form.get("agility_note")
22.
23.                record.technical_title = request.form.get("technical_title") or record.technical_title
24.                record.technical_feedback = request.form.get("technical_feedback") or
record.technical_feedback
25.                record.technical_completed = request.form.get("technical_completed") == "on"
26.
27.                categories = request.form.getlist("category[]")
28.                topics = request.form.getlist("topic[]")
29.                durations = request.form.getlist("duration[]")
30.                focuses = request.form.getlist("focus[]")
31.
32.                technical_items = []
```

```

33.     for cat, top, dur, foc in zip(categories, topics, durations, focuses):
34.         if top.strip():
35.             technical_items.append({
36.                 "category": cat,
37.                 "topic": top,
38.                 "duration_or_reps": dur,
39.                 "focus": foc
40.             })
41.         if technical_items:
42.             record.technical_items = json.dumps(technical_items)
43.         else:
44.             record.technical_items = None
45.
46.         record.coach_physical_done = request.form.get("coach_physical_done") == "on"
47.         record.coach_technical_done = request.form.get("coach_technical_done") == "on"
48.         db.add(record)
49.         db.commit()
50.         flash("今日訓練已成功儲存", "success")
51.
52.     except Exception as e:
53.         logging.error(f"儲存錯誤, 日期: {selected_date}, 用戶ID: {current_user.id}, 錯誤: {e}")
54.         flash(f"儲存錯誤: {e}", "danger")

```

F. 評估生理程式碼：

```

1. @evaluation_bp.route('/chart_data', methods=['GET'])
2. @login_required
3. def evaluation_chart_data():
4.     with get_db() as db:
5.         records =
6.         db.query(Evaluation).filter_by(user_id=current_user.id).order_by(Evaluation.eval_date.asc()).all()
7.         data = {
8.             "dates": [r.eval_date.strftime("%Y-%m-%d") for r in records],
9.             "training_status": [r.training_status for r in records],
10.            "fitness": [r.fitness for r in records],
11.            "sleep": [r.sleep for r in records],

```

```

11.     "appetite": [r.appetite for r in records]
12. }
13. return jsonify(data)

```

G. 上傳飲食照片：

```

1. @bp.route('/upload', methods=['GET', 'POST'])
2. @login_required
3. def upload_food():
4.     if request.method == 'POST':
5.         athlete_id = request.form.get('athlete_id', type=int) or 1
6.
7.         if 'photo' not in request.files:
8.             flash('沒有選擇檔案', 'error')
9.             return redirect(request.url)
10.
11.         file = request.files['photo']
12.         if not file or file.filename == '':
13.             flash('沒有選擇檔案', 'error')
14.             return redirect(request.url)
15.
16.         if not allowed_file(file.filename):
17.             flash('檔案類型只允許： png, jpg, jpeg', 'error')
18.             return redirect(request.url)
19.
20.         base_name = secure_filename(file.filename)
21.         timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
22.         unique = uuid4().hex[:6]
23.         filename = f"{timestamp}_{unique}_{base_name}"
24.
25.         upload_folder = get_food_upload_folder()
26.         os.makedirs(upload_folder, exist_ok=True)
27.         save_path = os.path.join(upload_folder, filename)
28.
29.         try:
30.             file.save(save_path)
31.

```

```

32.     with get_db() as db:
33.         if not check_athlete_exists(db, athlete_id):
34.             if os.path.exists(save_path):
35.                 os.remove(save_path)
36.                 flash(f'運動員ID {athlete_id} 不存在或沒有權限', 'error')
37.                 return redirect(request.url)
38.
39.         db.execute(
40.             text(
41.                 "INSERT INTO food_photos (athlete_id, filename, upload_time) "
42.                 "VALUES (:athlete_id, :filename, :upload_time)"
43.             ),
44.             {
45.                 "athlete_id": athlete_id,
46.                 "filename": filename,
47.                 "upload_time": datetime.now()
48.             }
49.         )
50.         db.commit()
51.
52.         flash('上傳成功', 'success')
53.         return redirect(url_for('food.view_food'))
54.
55.     except Exception as e:
56.         current_app.logger.exception(f"上傳失敗: {str(e)}")
57.         if os.path.exists(save_path):
58.             os.remove(save_path)

```

H. 教練建議：

```

1. @bp.route('/comment/<int:photo_id>', methods=['POST'])
2. @login_required
3. def comment_food(photo_id: int):
4.     comment = request.form.get('comment', '').strip()
5.     if not comment:
6.         flash('請輸入評論內容', 'error')
7.         return redirect(url_for('food.view_food'))
8.

```

```

9. with get_db() as db:
10.     photo = db.execute(
11.         text("SELECT * FROM food_photos WHERE id=:photo_id"),
12.         {"photo_id": photo_id}
13.     ).mappings().fetchone()
14.
15. if not photo:
16.     flash("找不到該照片", "error")
17.     return redirect(url_for('food.view_food'))
18.
19. now_time = datetime.now()
20.
21. db.execute(
22.     text(
23.         "UPDATE food_photos "
24.         "SET comment=:comment, comment_time=:comment_time "
25.         "WHERE id=:photo_id"
26.     ),
27.     {"comment": comment, "comment_time": now_time, "photo_id": photo_id}
28. )
29. db.commit()
30.
31. flash('評論已送出', 'success')
32. return redirect(url_for('food.view_food'))

```

I. Webhook 接收與事件處理：

```

1. @line_bp.route("/callback", methods=['POST'])
2. def callback():
3.     signature = request.headers.get('X-Line-Signature')
4.     body = request.get_data(as_text=True)
5.     print("Received body:", body)
6.     print("Signature:", signature)
7.
8.     try:
9.         handler.handle(body, signature)
10.    except InvalidSignatureError:
11.        abort(400)

```



```
12. return 'OK'
```

J. linebot圖片訊息處理（飲食照片上傳）：

```
1. @handler.add(MessageEvent, message=ImageMessage)
2. def handle_image_message(event):
3.     line_id = event.source.user_id
4.
5.     message_content = line_bot_api.get_message_content(event.message.id)
6.     os.makedirs(UPLOAD_FOLDER, exist_ok=True)
7.     filename = f"{datetime.now().strftime('%Y%m%d_%H%M%S')}.jpg"
8.     filepath = os.path.join(UPLOAD_FOLDER, filename)
9.
10.    with open(filepath, "wb") as f:
11.        for chunk in message_content.iter_content():
12.            f.write(chunk)
13.
14.    with get_db() as session:
15.        user = session.query(User).filter_by(line_user_id=line_id).first()
16.        if not user:
17.            line_bot_api.reply_message(
18.                event.reply_token,
19.                TextSendMessage(text="❌ 找不到使用者，請先綁定帳號")
20.            )
21.        return
22.    session.execute(
23.        text(
24.            "INSERT INTO food_photos (athlete_id, athlete_username, filename, upload_time) "
25.            "VALUES (:athlete_id, :athlete_username, :filename, :upload_time)"
26.        ),
27.        {
28.            "athlete_id": user.id,
29.            "athlete_username": user.username,
30.            "filename": filename,
31.            "upload_time": datetime.now()
32.        }
33.    )
34.    session.commit()
```

```

35. line_bot_api.reply_message(
36.     event.reply_token,
37.     TextSendMessage(text="✅ 圖片已上傳! ")
38. )

```

K. Linebot Rich Menu 建立:

```

1. def create_rich_menu():
2.     rich_menu = RichMenu(
3.         size={"width": 2500, "height": 1686},
4.         selected=True,
5.         name="功能選單",
6.         chat_bar_text="選單",
7.         areas=[
8.             RichMenuArea(
9.                 bounds=RichMenuBounds(x=0, y=0, width=833, height=843),
10.                action=MessageAction(label="訓練表", text="訓練表")
11.            ),
12.            RichMenuArea(
13.                bounds=RichMenuBounds(x=834, y=0, width=833, height=843),
14.                action=MessageAction(label="出席", text="出席")
15.            ),
16.            RichMenuArea(
17.                bounds=RichMenuBounds(x=1667, y=0, width=833, height=843),
18.                action=MessageAction(label="公告", text="公告")
19.            ),
20.
21.            RichMenuArea(
22.                bounds=RichMenuBounds(x=0, y=843, width=833, height=843),
23.                action=MessageAction(label="功能", text="功能")
24.            ),
25.            RichMenuArea(bounds=RichMenuBounds(x=834, y=843, width=833, height=843),
26.                action=MessageAction(label="Logo", text="這是 PingPro Logo 🤖")),
27.            RichMenuArea(
28.                bounds=RichMenuBounds(x=1667, y=843, width=833, height=843),
29.                action={"type": "uri", "label": "網站", "uri": "https://dashboard.pingpro.xyz"}
30.            ),
31.        ]

```

```

32. )
33. rich_menu_id = line_bot_api.create_rich_menu(rich_menu=rich_menu)
34. print("✅ Rich Menu 建立成功, ID:", rich_menu_id)
35.
36. image_path = "static/richmenu.png"
37. if os.path.exists(image_path):

```

L. 點名功能:

```

1. @coach_bp.route("/rollcall", methods=["GET", "POST"])
2. @login_required
3. def roll_call():
4.     with get_db() as session:
5.         athletes = session.query(User).filter_by(role='athlete').all()
6.
7.         if request.method == "POST":
8.             roll_date_str = request.form.get("roll_date")
9.             roll_date = datetime.strptime(roll_date_str, "%Y-%m-%d").date()
10.
11.            for athlete in athletes:
12.                status = request.form.get(f"attendance_{athlete.id}")
13.                if status:
14.                    existing = session.query(Attendance).filter_by(
15.                        athlete_id=athlete.id, date=roll_date
16.                    ).first()
17.
18.                    if existing:
19.                        existing.status = status
20.                    else:
21.                        attendance = Attendance(
22.                            athlete_id=athlete.id,
23.                            date=roll_date,
24.                            status=status
25.                        )
26.                        session.add(attendance)
27.                session.commit()
28.                flash("✅ 點名結果已成功儲存! ", "success")
29.                return redirect(url_for("coach.roll_call"))
30.
31.            today = datetime.today().strftime("%Y-%m-%d")

```

```

32.     return render_template(
33.         "coach/rollcall.html",
34.         athletes=athletes,
35.         today=today
36.     )

```

M. 新增與查詢公告：

```

1.  @coach_bp.route("/announcements", methods=["GET"])
2.  @login_required
3.  def announcements():
4.      session = SessionLocal()
5.      try:
6.          announcements = (
7.              session.query(Announcement)
8.              .filter(Announcement.coach_id == current_user.id)
9.              .order_by(Announcement.date.desc())
10.             .all()
11.         )
12.         return render_template("coach/announcements.html", announcements=announcements)
13.     finally:
14.         session.close()
15. @coach_bp.route("/announcements/new", methods=["GET", "POST"])
16. @login_required
17. def new_announcement():
18.     if request.method == "POST":
19.         form = request.form
20.         ann = Announcement(
21.             date=datetime.strptime(form["date"], "%Y-%m-%d").date(),
22.             title=form["title"],
23.             content=form["content"],
24.             category=form["category"],
25.             coach_id=current_user.id,
26.         )
27.         with get_db() as session:
28.             session.add(ann)
29.             session.commit()
30.         return redirect(url_for("coach.announcements"))
31.     return render_template("coach/new_announcement.html")

```

第10章 測試模型

10-1 測試計畫

在測試計畫中，分成AI動作辨識分析系統與桌訓系統，分別對應的測試方法如下：

AI動作辨識分析系統：

本系統主要功能包括：桌球正反手動作分類、辨識選手動作並提供專業姿勢分析與回饋。為了確保正確與穩定性，採用了以下測試方法：

1. 功能測試 (Functional Testing)

目的：驗證系統對各類桌球影片的動作辨識、物件偵測及姿勢分析功能是否正確，方法如下：

- 上傳正手與反手影片，動作分類結果正確。
- YOLOv8 模型能準確偵測選手之動作。
- 姿勢分析能計算選手動作中關節角度正確，並提供姿勢修正建議。

2. 介面測試 (UI Testing)

目的：確認系統使用介面順暢，操作直覺且資訊呈現正確，方法如下：

- 測試影片上傳流程能順利完成。
- 檢查系統回饋呈現，包括影片播放、分析進度、結果回傳並顯示等。

3. 異常測試 (Exception Testing)

目的：確認系統在異常情況下仍能穩定運作並提示使用者，方法如下：

- 上傳非影片檔案或有損毀之影片，確認系統提示正確錯誤訊息。
- 上傳影片幀數不足時，確認系統能即時提示。

4. 性能測試 (Performance Testing)

目的：評估系統對上傳影片的處理效率，方法如下：

- 上傳不同長度影片，測量分析時長，並能在過程中提供進度提示。

桌訓系統模組：

A.LineBot 聊天機器人測試

1. 功能測試 (Functional Testing)

目的：驗證聊天機器人是否能正確引用API並回覆使用者訊息。

- 輸入關鍵字（如「公告」、「出缺勤」），確認回覆正確。
- 回覆訊息正確性、與系統資料同步、聊天流程順暢

2. 介面測試 (UI Testing)

目的：確認聊天介面順暢，訊息呈現清楚。

檢查訊息即時顯示，清楚區分使用者與系統回覆。

- 測試不同訊息長度及格式，確認版面正確。

3. 異常測試 (Exception Testing)

目的：確認系統在異常輸入下仍能穩定運作並提示使用者。

- 輸入不支援的指令或非桌球相關問題，檢查回覆是否合理。
- 測試網路斷線情況，確認提示訊息正確。

4. 性能測試 (Performance Testing)

目的：評估系統在大量訊息下的回應效率。

- 多名使用者同時發送訊息，測量回覆延遲時間。
- 長時間互動，檢查系統穩定性。
- 回覆延遲時間是否合理
- 多用戶同時操作下系統是否順暢

B. AI 桌球聊天機器人測試：

1. 功能測試 (Functional Testing)

目的：驗證聊天機器人能正確回覆選手問題。

- 輸入桌球技術問題（如「正手攻球怎麼練？」），確認系統提供合理建議。

2. 介面測試 (UI Testing)

目的：確認聊天介面操作直覺，訊息呈現清楚。

- 測試輸入文字、送出訊息流程，確認操作順暢。
- 檢查回覆訊息顯示清楚、時間順序正確、使用者與系統訊息區分明確。

3. 異常測試 (Exception Testing)

目的：驗證系統在異常輸入下仍能提示使用者並保持穩定。

- 輸入系統不支援的指令或非桌球相關問題，確認回覆仍合理或提示錯誤。
- 長訊息或空訊息送出，確認系統提示正確並不崩潰。

4. 性能測試 (Performance Testing)

目的：評估系統在大量訊息同時進行時的效率。

- 同時與聊天機器人互動，檢查回覆速度與正確性。
- 訊息回覆速度合理。

C. 教練課表派發訓練菜單與選手查看訓練菜單：

1. 功能測試 (Functional Testing)

目的：驗證課表派發與查看功能能正確運作。

- 教練新增每日訓練課表，確認選手端能即時看到課表。
- 教練修改任務，確認更新內容同步至選手端。
選手查看課表並勾選完成，系統即時更新完成度紀錄。
- 完成度紀錄能顯示於教練端長條圖或報表中。

2. 介面測試 (UI Testing)

目的：確認課表操作直覺，資訊呈現清楚。

- 測試教練端新增、修改課表流程，確認欄位與操作提示正確。
測試選手端查看課表及勾選完成流程，確認操作順暢。
- 檢查課表與完成度圖表呈現清晰易讀。

3. 異常測試 (Exception Testing)

目的：驗證系統在異常操作下仍能提示並保持穩定。

- 教練嘗試新增或修改課表中缺少必填欄位，系統提示錯誤。
- 選手勾選課表前課表不存在，系統提示無法操作。
- 多名選手同時操作課表，確認不會導致資料衝突或錯誤。

4. 性能測試 (Performance Testing)

目的：評估系統在多名教練與選手同時操作課表時的效率。

- 同時查看課表與勾選完成，檢查完成度更新速度。
- 大量課表資料生成圖表，測試渲染效率。

D. 點名系統測試

1. 功能測試 (Functional Testing)

目的：驗證點名系統能正確記錄選手出席狀態並更新資料。

- 教練發起點名，選手回覆出席、缺席或請假，確認系統正確紀錄並更新狀態。
- 出缺勤紀錄正確保存
- 資料即時更新

2. 介面測試 (UI Testing)

目的：確認點名介面清楚，選手操作直覺且結果明確顯示。

- 測試點名回覆頁面，確認出缺勤資訊清楚顯示。
- 確認選手點選回覆後，頁面立即更新。
- 出缺勤資訊可讀性
- 回覆操作流程順暢

3. 異常測試 (Exception Testing)

目的：驗證系統在異常操作下能正確提示並保持穩定。

- 選手重複提交點名狀態，確認系統能防止重複紀錄。
- 教練刪除不存在的點名，確認系統提示正確。
- 錯誤訊息清楚易懂
- 系統穩定不中斷

4. 性能測試 (Performance Testing)

目的：測試系統在多用戶同時點名時的處理效率。

- 同時多名選手回覆點名，確認紀錄即時更新。
- 多名教練同時發起點名，測量系統反應速度。
- 多用戶操作下紀錄更新即時
- 系統處理速度合理

E. 公告系統測試

1. 功能測試 (Functional Testing)

目的：驗證公告系統能正確發布、接收與顯示公告。

- 教練新增公告（重要、普通、閒聊），確認選手端能即時接收訊息。
- 測試公告分類功能，確認公告可依日期與類別正確篩選。
- 公告即時顯示
- 公告分類正確
- 選手端接收無延遲

2. 介面測試 (UI Testing)

目的：確認公告頁面清晰易讀，操作直覺。

- 檢查公告列表頁面，確認標題、日期與內容呈現正確。
- 測試公告點開與關閉操作，確認使用流暢。
- 標題、時間與內容可讀性
- 操作流程順暢

3. 異常測試 (Exception Testing)

目的：驗證系統在異常情況下能提示並穩定運作。

- 刪除不存在的公告，確認系統能提示錯誤而不崩潰。
- 上傳過長或格式錯誤的公告內容，確認系統提示正確。
- 錯誤訊息清楚
- 系統穩定不中斷

4. 性能測試 (Performance Testing)

目的：測試公告系統在大量公告或多用戶操作下的穩定性。

- 同時新增多則公告，確認選手端能即時接收。
- 多名使用者同時查看公告，測量頁面加載速度。
- 多用戶操作下公告更新即時

F. 心理紀錄表單測試

1. 功能測試 (Functional Testing)

目的：驗證心理紀錄表單能正確收集與分析選手心理狀態數據。

- 選手輸入心理狀態數據（如焦慮值高於閾值），確認系統能正確記錄並標記異常。
- 系統能生成歷史心理狀態趨勢圖。

檢查點：

- 心理數據正確儲存
- 異常數據標記正確

2. 介面測試 (UI Testing)

目的：確認心理紀錄表單操作簡單、清晰易懂。

- 檢查欄位填寫規則與必填欄位提示是否正確。
- 表單填寫流程順暢
- 欄位驗證正確

3. 異常測試 (Exception Testing)

目的：驗證系統在異常輸入下仍能穩定運作並提示使用者。

- 輸入超出允許範圍的數值，確認系統提示錯誤。
- 留空必填欄位提交，確認提示正確。
- 錯誤訊息清楚易懂
- 系統穩定不中斷
- 防呆機制完整

4. 性能測試 (Performance Testing)

目的：評估系統在多名選手同時填寫心理表單時的處理效率。

- 多名選手同時提交心理數據，檢查系統儲存與更新速度。

G. 睡眠分析測試

1. 功能測試 (Functional Testing)

目的：驗證系統能正確分析睡眠數據並提供警示。

- 選手輸入不同睡眠時長，系統能計算睡眠品質並顯示分析結果。
- 睡眠數據正確儲存
- 警示訊息正確顯示
- 分析結果與實際數據一致

2. 介面測試 (UI Testing)

目的：確認睡眠紀錄操作簡單且資訊呈現清楚。

- 檢查欄位填寫規則與必填提示是否正確。
- 確認提交後結果與警示顯示清楚。
- 表單填寫流程順暢
- 提示訊息清楚

3. 異常測試 (Exception Testing)

目的：驗證系統在異常輸入下仍能穩定運作並提示使用者。

- 輸入不合理的睡眠時數，確認系統提示錯誤。
- 留空必填欄位提交，確認系統提示正確。
- 錯誤訊息清楚易懂

4. 性能測試 (Performance Testing)

目的：評估系統在多名選手同時提交睡眠數據時的處理效率。

- 同時提交睡眠紀錄，檢查資料儲存與分析速度。

H. 訓練狀況紀錄測試

1. 功能測試 (Functional Testing)

目的：驗證訓練狀況模組能正確收集、分析與顯示選手訓練數據。

- 選手新增訓練狀況紀錄，系統能儲存資料並更新折線圖。
- 系統能編輯與刪除既有紀錄，折線圖同步更新。
- 歷史紀錄折線圖正確顯示選手訓練趨勢。

2. 介面測試 (UI Testing)

目的：確認訓練狀況紀錄操作直覺，資料呈現清楚。

- 測試新增、編輯、刪除紀錄流程，確認操作順暢。
- 折線圖能即時更新並清楚顯示歷史趨勢。

3. 異常測試 (Exception Testing)

目的：驗證系統在異常輸入下能提示並保持穩定。

- 選手提交資料不完整或格式錯誤，系統提示正確。
- 編輯或刪除不存在的紀錄，系統提示錯誤。

4. 性能測試 (Performance Testing)

目的：評估系統在多名選手同時新增或查看訓練紀錄時的效率。

- 同時新增訓練狀況紀錄，檢查資料儲存與折線圖更新速度。
- 查看大量歷史紀錄時，測試折線圖渲染效率。
- 折線圖生成與更新時間合理

I. 飲食控制測試

1. 功能測試 (Functional Testing)

目的：驗證飲食控制模組能正確收集、儲存與呈現飲食資料。

選手上傳飲食照片或手動輸入餐點內容，確認資料能正確儲存。

- 系統能在紀錄頁面即時顯示最新飲食資料，並可追蹤歷史紀錄。
- 教練能查看選手上傳的飲食資料並提供回饋。

2. 介面測試 (UI Testing)

目的：確認飲食控制操作直覺，資料呈現清楚。

- 測試照片上傳流程，確認檔案選擇、上傳進度及成功提示。
- 測試手動輸入餐點內容，確認欄位格式及必填驗證正確。

3. 異常測試 (Exception Testing)

目的：驗證系統在異常輸入下仍能提示使用者並穩定運作。

- 上傳非圖片檔案（如 PDF 或文檔），確認系統提示錯誤。
- 留空必填欄位或上傳過大檔案，確認系統提示正確。

性能測試 (Performance Testing)

目的：評估系統在多名選手同時提交飲食資料時的效率。

方法如下：

- 同時上傳照片或資料，檢查系統儲存速度與頁面更新時間。

10-2 測試個案與測試結果資料

表10-2-1 測試個案與結果

編號	測試功能	測試資料	預期結果	實際結果	備註
01	動作分類與姿勢分析	上傳桌球正手影片(MP4)	系統正確辨「正手」，並計算手臂與身體角度	預期結果正確	代表性案例，其他影片測試結果類似
02	動作分類(反手)	上傳桌球反手影片(MP4)	系統正確辨「反手」，姿勢角度計算完成	預期結果正確	測試多種反手影片確認穩定性
03	異常影片格式	上傳文字檔或損壞影片	系統提示「影片無法讀取」或「格式錯誤」	系統正確提示	
04	YOLO物件偵測	上傳桌球比賽影片	系統標註球員與球，至少偵測到球及人	標註正確	每 10 幀抽檢確認
05	數據回饋	詳細姿勢回饋	取得擊球瞬間姿勢，分析擊球角度數據分析並回饋個數據，給出建議擊改進方法、變化趨勢圖	標註正確	

第11章 操作手冊

11-1 系統元件

表11-1 系統安裝元件資訊

元件名稱	PingPro桌訓系統
版本	第一版
存取方式	透過瀏覽器進入
軟體類別	工具
價格	免費
最低版本需求	Chrome 90 以上 / Firefox 88 以上 / Safari 14 以上
網路需求	需具備 4G/WiFi 無線網路連線

11-2 安裝

表11-2 系統安裝說明表

步驟一	開啟瀏覽器（建議使用 Chrome、Firefox、Edge 或 Safari 最新版本）
步驟二	輸入系統網址:dashboard.pingpro.xyz/

第12章 使用手冊

12-1 系統使用說明



註冊帳號

使用者名稱

密碼

角色

選手

註冊

已有帳號? [登入](#)

圖12-1-1 帳號註冊
任一角色皆須進行帳號註冊

從圖12-1-2到圖12-1-12為**教練**操作說明



The image shows a login interface for athletes. At the top, there is a green running icon followed by the text "運動員登入" (Athlete Login). Below this, there are two input fields: the first is labeled "使用者名稱" (Username) and the second is labeled "密碼" (Password). A green button labeled "登入" (Login) is positioned below the password field. At the bottom, there is a link that says "沒有帳號? 註冊" (No account? Register).

圖12-1-2 登入介面



The image displays the "教練綜合管理系統" (Coach Comprehensive Management System) interface. At the top right, there are links for "常見問題 Q&A", "語音助手", "聊天機器人", and "個人資訊". The main header is "教練綜合管理系統". Below the header, there is a user profile section for "vxnnio" with the role "coach". It includes a profile picture, a progress bar, and the text "上次登入: 2025/04/11". On the right side, there is a section titled "本週隊伍訓練完成度" (This week's team training completion).

圖12-1-3 教練個人資訊顯示
教練登入完成後的介面，顯示教練個人資訊

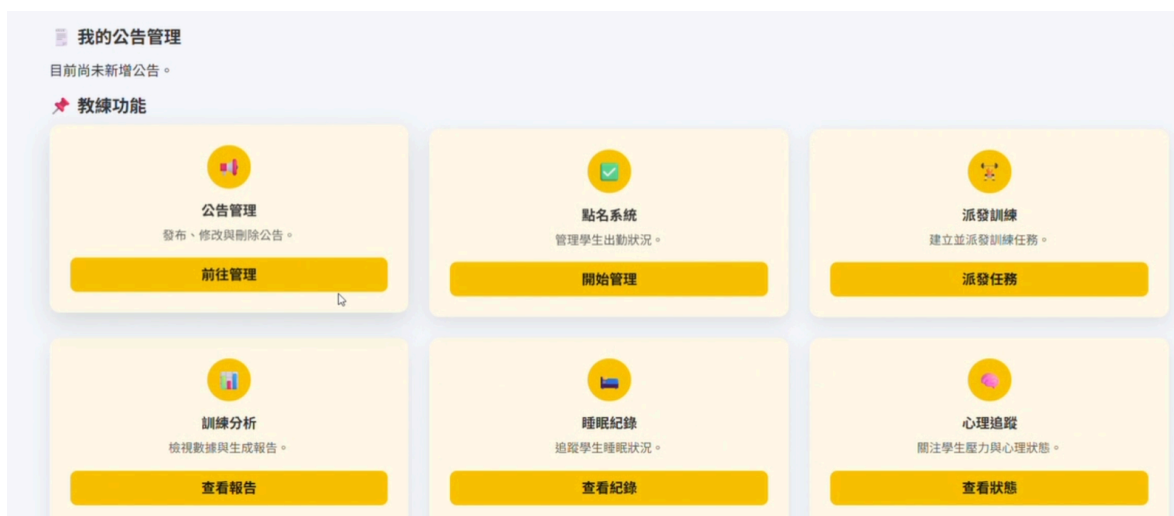


圖12-1-4 教練介面功能顯示

教練登入完成後的介面，顯示教練功能，分為公告管理、點名系統、派發訓練、訓練分析、睡眠紀錄、心理追蹤與飲食紀錄。



圖12-1-5 公告列表

點選教練功能的公告管理可進入此列表查看，允許教練新增、修改或刪除，該公告會讓所有選手看到。

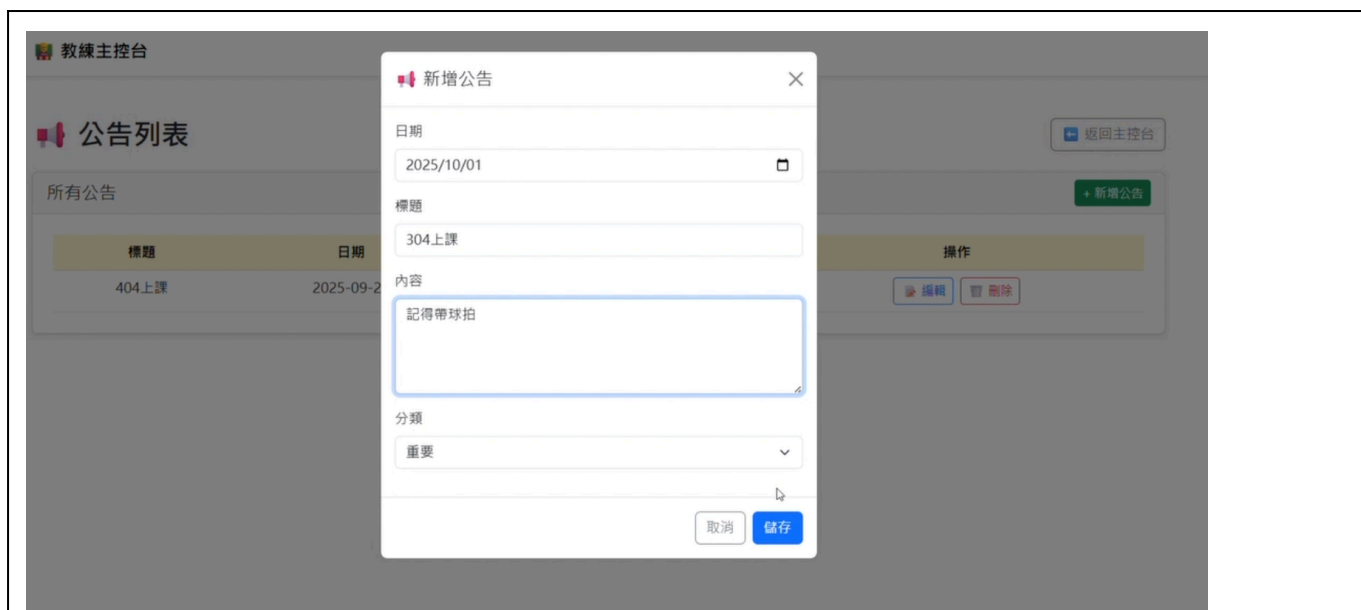


圖12-1-6 新增公告

點選右上角的新增公告讓教練輸入公告內容以新增公告。



圖12-1-7 派發任務

點選教練功能中的派發任務可以顯示及新增訓練任務給選手。



圖12-1-8 訓練分析

點選教練功能中的訓練分析可以讓教練查看選手的訓練分析成果，輔助教練制定訓練策略。



圖12-1-9 睡眠紀錄

點選教練功能中的睡眠紀錄可以讓教練查看選手的睡眠紀錄。

選手壓力評估查詢

選擇選手：

高偉倫



查詢

回到主頁



圖12-1-10 心理追蹤

點選教練功能中的心理追蹤可以讓教練查看選手的心理壓力變化。

Athlete 飲食紀錄

Athlete : gao (ID : 1)

上傳於 2025-10-01 11:11:07



教練評論

吃太好了

送出評論

已評論：吃太好了

時間：2025-10-01 11:25:47

圖12-1-11 飲食紀錄

點選教練功能中的飲食紀錄可以讓教練查看選手近期的飲食紀錄並給予評論。

教練主控台

出缺勤點名

點名日期

2025/10/01

#	姓名	出席
1	gao	☒ 出席 ☐ 缺席 ☐ 遲到

儲存點名結果

返回主控台

圖12-1-12 點名系統

點選教練功能中的點名系統可以讓教練查看選手的每日出勤狀況。

從圖12-1-13到圖12-1-23為選手操作說明



圖12-1-13 選手登入介面



圖12-1-14 選手登入後的介面，含選手資訊與教練發的公告。

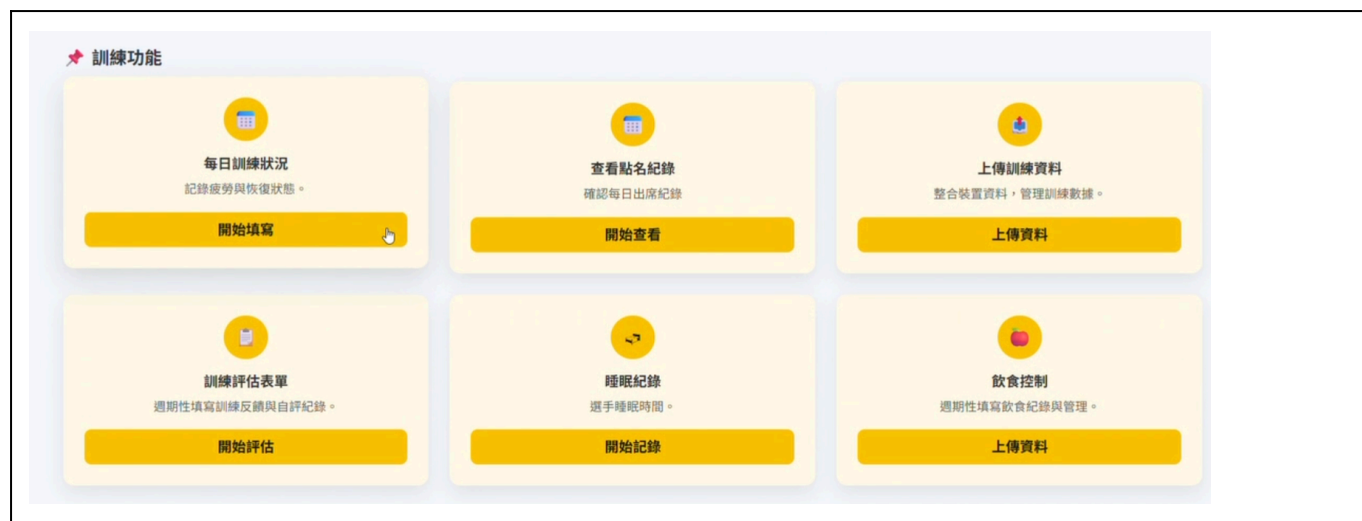
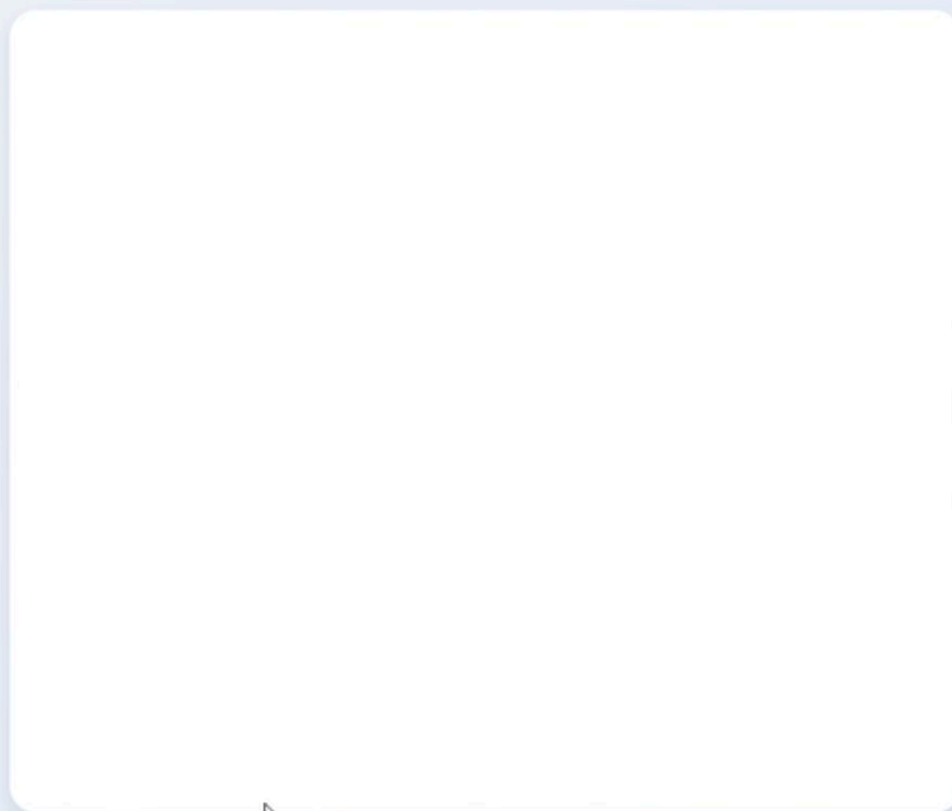


圖12-1-15 選手功能

選手功能包括每日訓練狀況記錄、查看點名紀錄、上船訓練資料、訓練評估表單、睡眠紀錄與飲食控制。

桌球教學聊天機器人



教我怎麼反手

送出

圖12-1-16 聊天機器人

選手可以使用聊天機器人詢問關於桌球相關的問題

3

體能

2

睡眠

3

食慾

3

備註

提交評估

返回歷史紀錄

返回主控台

圖12-1-17 每日訓練狀況

選手可以記錄自己每天的訓練狀況，並提交評估後讓系統給出分析圖表

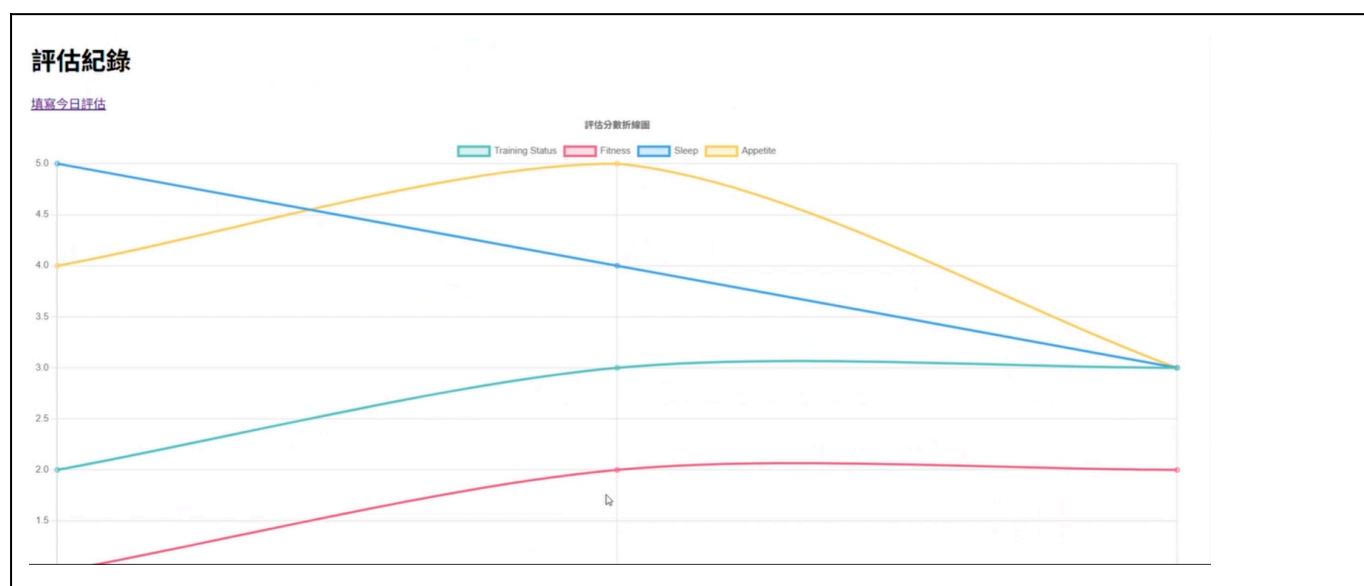
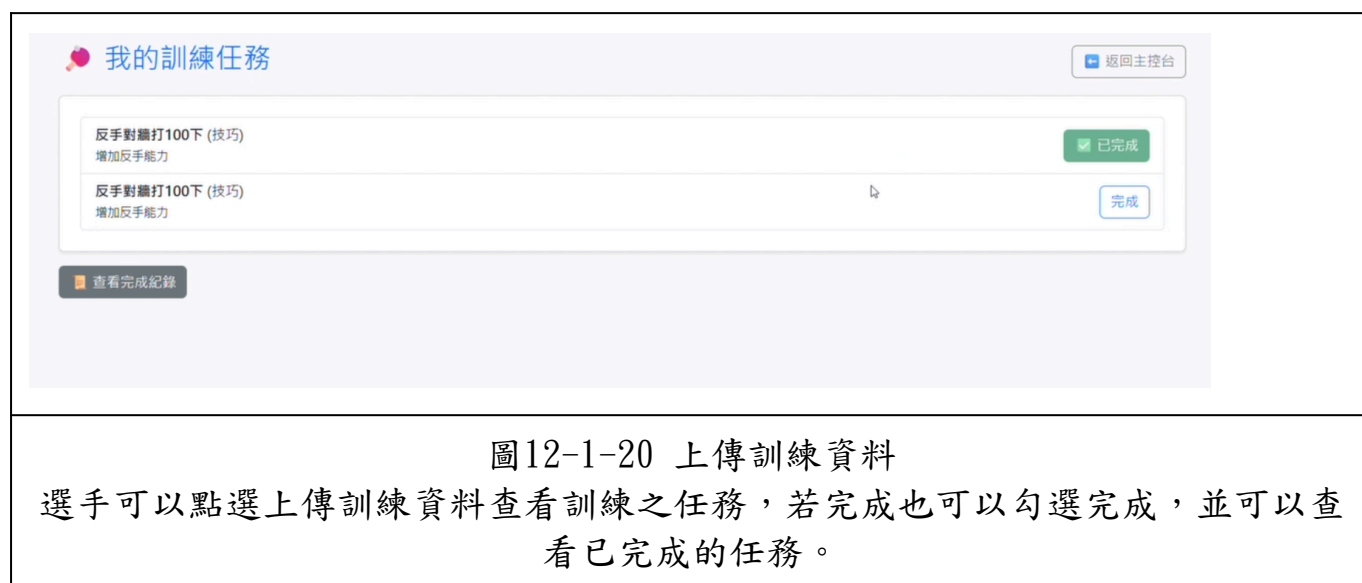


圖12-1-18 訓練狀況圖表
選手可以查看訓練狀況圖表



已完成任務

[返回主控台](#)

分類	項目	描述	完成時間
技巧	反手對牆打100下	增加反手能力	2025-10-01 03:56
體能	仰臥起坐60次	增加核心能力	2025-09-27 09:50

[返回任務列表](#)

圖12-1-21 訓練評估表單

我的睡眠紀錄

日期

2025/10/02

睡覺開始

下午 11:57

睡覺結束

上午 08:59

新增紀錄

返回主頁

歷史紀錄

日期	開始時間	結束時間	睡眠時長
2025-10-01	22:54:00	10:52:00	11 小時 58 分鐘

圖12-1-22 睡眠紀錄

選手可以填寫睡眠紀錄，記錄每日的睡眠時長。



圖12-1-24為**管理員**操作說明



從圖12-1-25到圖12-1-27為**AI動作辨識分析系統**操作說明

桌球動作分析平台

上傳影片，進行正反手專業運動姿勢分析。

✓ 所有配置載入完成！

請上傳影片檔 (avi, mp4, mov, mkv, flv)



Drag and drop file here

Limit 200MB per file • AVI, MP4, MOV, MKV, FLV, MPEG4

Browse files



779779816.093004.mp4 296.7KB



圖12-1-25 上傳影片
點選Browse files上傳影片以進行分析

💪 專業運動姿勢分析

啟動姿勢分析

🕒 正在進行複雜的運動姿勢分析 (此過程可能較長，請耐心等待)...

✳ 啟動運動姿勢分析...

✳ 姿勢分析進度: 14/30 幀

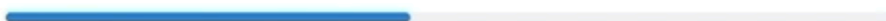


圖12-1-26 啟動運動姿勢分析

點選啟動姿勢分析，讓AI系統評估選手的姿勢是否有需要修正。

Hip Flexion (180.00° / 專家標準: 165.0°±15.0°)

狀態: ✓ 良好

問題: 無

建議: 姿勢良好，符合專家標準。

改進方法: 暫無具體建議或姿勢良好。

Trunk Rotation (175.02° / 專家標準: 170.0°±10.0°)

狀態: ✓ 良好

問題: 無

建議: 姿勢良好，符合專家標準。

改進方法: 暫無具體建議或姿勢良好。

📈 關節角度變化趨勢圖

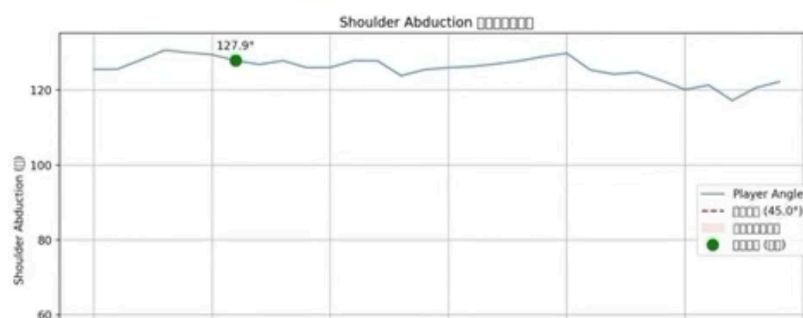
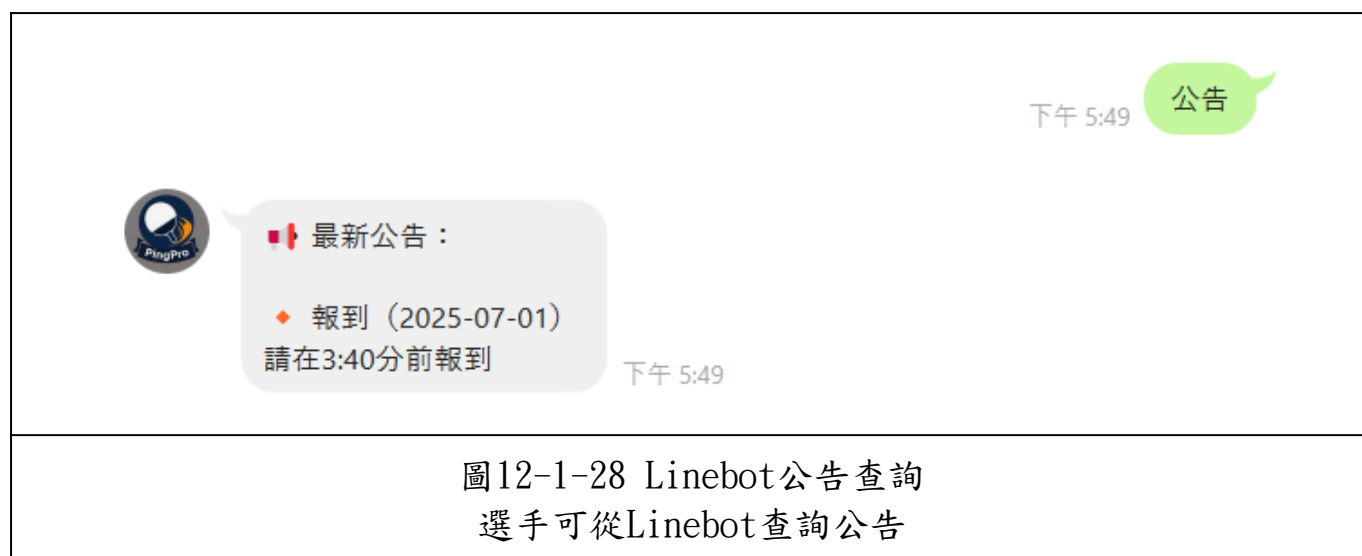


圖12-1-27 動作姿勢分析回饋

系統會給予各關節角度的分析與專家的標準數據比對，並給予適當的回饋。

從圖12-1-28到圖12-1-32為Linebot操作說明



下午 5:49

訓練表



填寫訓練評估表

請填寫最近的訓練評估紀錄

前往填寫

圖12-1-30 Linebot今日訓練狀況
選手可從Linebot直接前往填寫今日訓練狀況



下午 5:54

儲存 | 另存新檔 | 分享 | 傳送至Keep筆記



✓ 圖片已上傳！

下午 5:54

圖12-1-31 Linebot飲食控制
選手可直接從Linebot上傳個人飲食照片



第13章 感想

曾順暉：

在這次專題的過程中，我學到了如何把一個想法從零開始轉換成實際的系統。從資料庫設計、前後端串接，到使用者介面的規劃，每個步驟都讓我更了解系統開發的完整流程。雖然過程中遇到不少困難，例如功能驗證與資料整合的問題，但在跟他交流還有查詢資料後，問題都一一解決了。完成後覺得不只是技術能力的提升，更重要的是培養了面對問題的態度與解決能力。這份專題不僅讓我收穫滿滿，也讓我對未來在資訊領域的學習更有信心。

陳頤謙：

在這為期一年的專題，其實很多事情都是要從零開始學習，從環境建置安裝、前端網頁設計、後端程式架構及資料庫，再到最後架設伺服器，我學習到了很多課程當中沒有學習過的內容，雖然過程很辛苦，但我覺得我更深入了解資訊這個領域。這個專題的主題結合了我的專長去做延伸，透過與組員還有校方的合作，我也意識到，所有的領域其實都能與資訊相互結合，也是因為專題的實作，我掌握了製作一個系統，需要準備的所有前置作業，包含調查使用者需求以及功能之間的互相關聯，在製作文件的過程也會更加了解這個系統的整體性以及欠缺的地方。

張禎益：

在這次的專題中擔任組長的經驗讓我受益良多，許多項目一開始對我來說都很陌生甚至會感到緊張，不論是技術方面或者是行政方面。為了將功能成功寫出來，讓我學會了前端設計、後端架構甚至是資料庫的相關邏輯。組長的部分，其實最初在如何統籌與規劃組上的進度，以及和指導老師的溝通上也遇到多困難。不過後來也逐漸找到適當的方法，讓組員之間的配合及運轉更有效率。最後，在處理大量的文件後也掌握了許多新的技巧。相信在經過這段時間後，我在資訊領域的實力能更上一層樓。

高偉倫：

專題過程中會遇到許多挑戰，在這次專題中，我學習到團隊溝通合作的重要性，當專題範圍龐大時，如何有效地協調、分配工作量，是我們必須培養的能力。同時，也讓我深入了解資料處理方式、前後端的設計邏輯、資料庫 schema 的建立以及 Docker 的應用。專題是一個絕佳的實踐機會，可以勇於嘗試新挑戰，為未來職場累積寶貴經驗與養分。只要保持持續學習與探索，就能不斷成長並精進自我，並在面對困難時培養解決問題的能力，讓自己更有信心迎向未來挑戰。

第14章 參考資料

桌球選手訓練文獻：

Forehand

Biomechanical Comparison between Down-the-Line and
Cross-Court Topspin Backhand in Competitive Table Tennis

(<https://www.mdpi.com/journal/ijerph>)

Backhand

Biomechanical analysis of forehand and backhand strokes in elite
table tennis players: Implications for training and injury
prevention(<https://doi.org/10.62617/mcb1421>)

BIOMECHANICAL ANALYSIS AND MODEL DEVELOPMENT APPLIED TO TABLE
TENNIS FOREHAND STROKES

([NANYANG TECHNOLOGICAL UNIVERSITY](#))

心理量表：

<https://srda.sinica.edu.tw/file/5f6ae87d-545a-451e-b54a-a2a8538abb90>

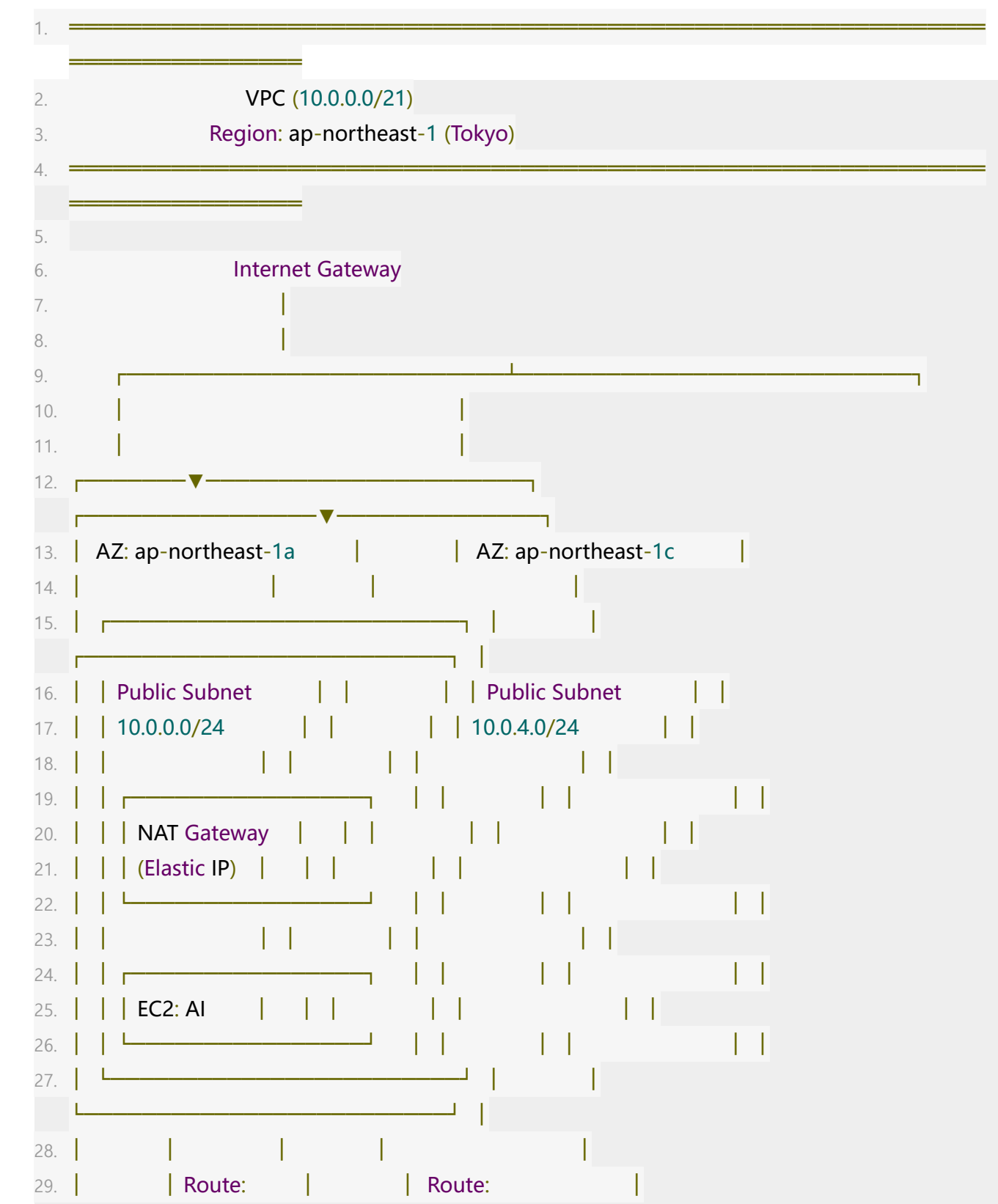
附錄

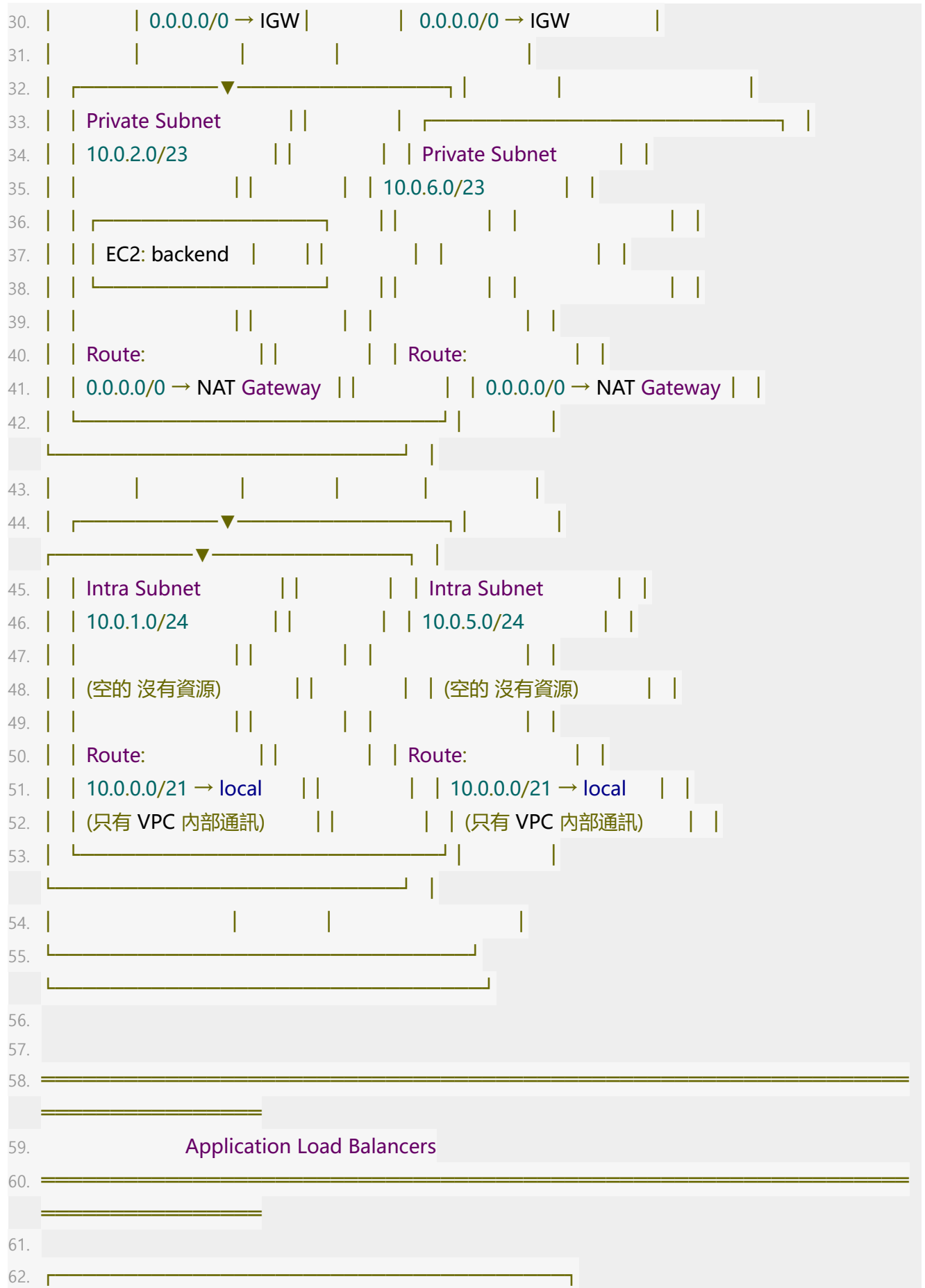
附錄-1 評審建議修改事項

評審建議事項	修正情形
1. 輸入方面： ● 都是自己輸入，如何克服惰性 ● 練習的時候不可能一直用電腦，國小生會使用電腦和了解怎麼填寫嗎 ● line聯絡最方便，如何結合官方line 2. 系統面向： ● 目前涵蓋對象範圍太大，功能太過廣泛但不深入。 3. 系統介面： ● 功能設計陽春，數據化分析跟視覺化有待提升目前沒看到，可否加入飲食控制。 ● 教練及選手輸入介面親切性待加強 4. 系統功能： ● 要能收集紀錄每位學員的訓練狀況 5. 其它： ● 缺GitHub上傳記錄，產學合作對象的須求介紹 6. 提案： ● 能否添加姿勢分析，讓訓練更加科技化	1. 已結合LINE Bot 2. 已修正 3. 已修正 4. 已修正 5. 已添加 6. 已添加

附錄-2 系統架設部署設定：

1. 網路架構：





63.	ALB: backend		ALB: AI
64.			
65.	Subnets:		Subnets:
66.	- 1a-public (10.0.0.0/24)		- 1a-public (10.0.0.0/24)
67.	- 1c-public (10.0.4.0/24)		- 1c-public (10.0.4.0/24)
68.			
69.	Target:		Target:
70.	└─> EC2: backend (1a-private)		└─> EC2: AI (1a-public)

流量路徑說明

【外部使用者 → backend 服務】

Internet → ALB: backend (1a-public, 1c-public) → EC2: backend (1a-private)

【外部使用者 → AI 服務】

Internet → ALB: AI (1a-public, 1c-public) → EC2: AI (1a-public)

【EC2: backend 對外連線】

EC2: backend (1a-private) → NAT Gateway (1a-public) → Internet Gateway → Internet

【EC2: AI 對外連線】

EC2: AI (1a-public) → Internet Gateway → Internet (直接對外)

子網路類型說明

Public Subnet (10.0.0.0/24, 10.0.4.0/24)

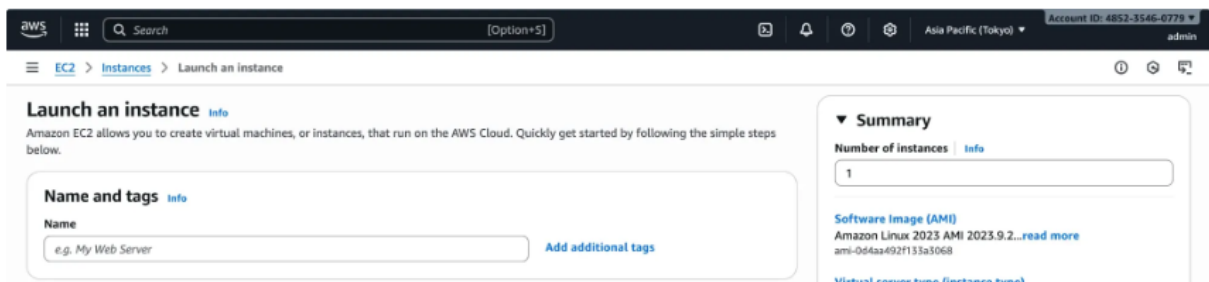
└─ 路由: 0.0.0.0/0 → Internet Gateway

└─ 資源: NAT Gateway, EC2: AI, ALB endpoints

- 97. └─ 用途：對外服務、NAT Gateway
- 98.
- 99. Private Subnet (10.0.2.0/23, 10.0.6.0/23)
- 100. └─ 路由：0.0.0.0/0 → NAT Gateway
- 101. └─ 資源：EC2: backend
- 102. └─ 用途：應用程式伺服器（可透過 NAT 對外連線）
- 103.
- 104. Intra Subnet (10.0.1.0/24, 10.0.5.0/24)
- 105. └─ 路由：只有 local (10.0.0.0/21)
- 106. └─ 資源：無
- 107. └─ 用途：完全隔離的子網路（預留給資料庫等服務）

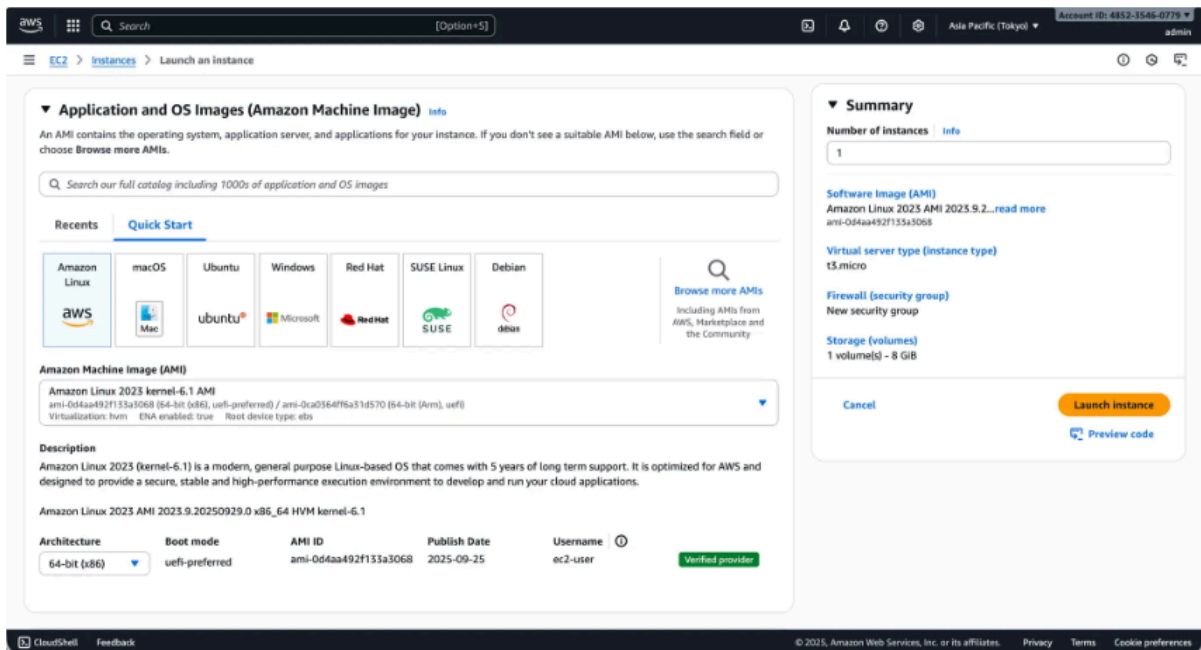
2. EC2 設定 for Linux:

1. 名稱設定：



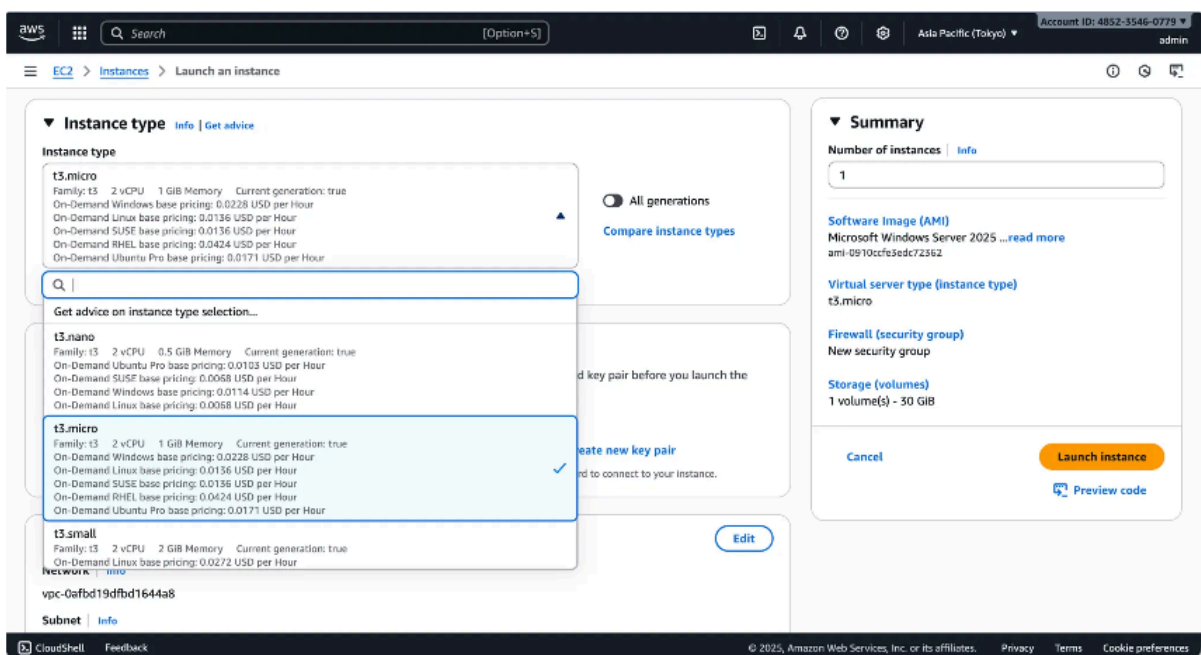
圖A-2-1 名稱設定

2. OS 設定：



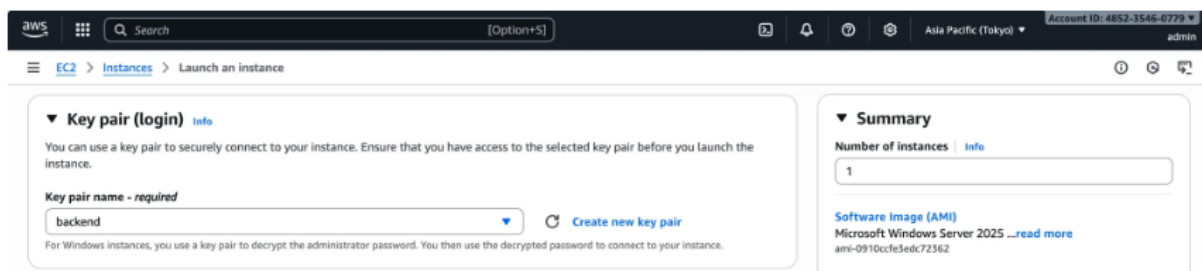
圖A-2-2 OS設定

3. 硬體設定：



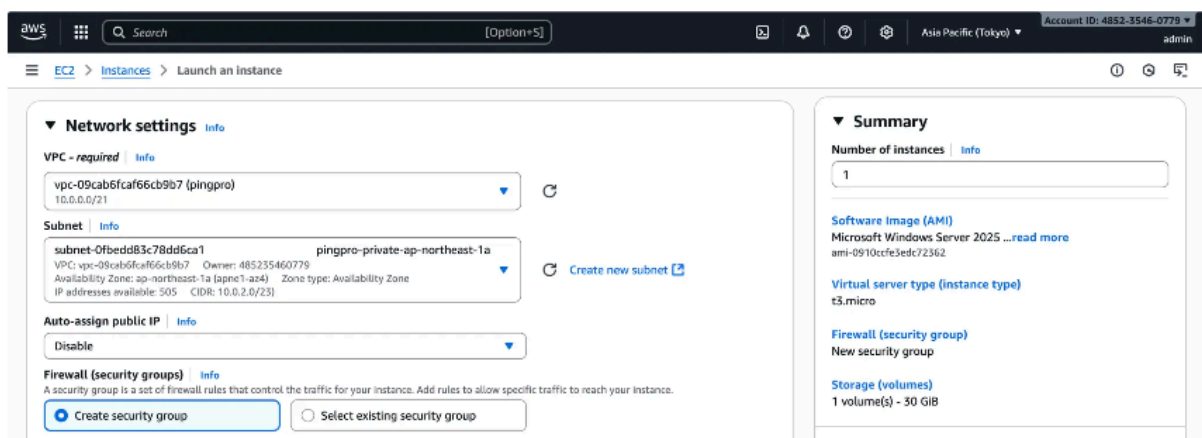
圖A-2-3 硬體設定

4. 金鑰設定：



圖A-2-4 金鑰設定

5. 網路設定：



圖A-2-5 網路設定

6. 網路設定(Security Group)：

The screenshot shows the AWS Management Console interface for launching an instance. The 'Configure storage' tab is active, displaying the 'Inbound Security Group Rules' section. Two rules are listed:

- Security group rule 1 (TCP, 22, 0.0.0.0/0)**: Type: ssh, Protocol: TCP, Port range: 22, Source type: Anywhere, Source: 0.0.0.0/0, Description: e.g. SSH for admin desktop.
- Security group rule 2 (TCP, 5000, sg-03f1bc3bad010ecd5, Allow 5000 port access from ALB SG)**: Type: Custom TCP, Protocol: TCP, Port range: 5000, Source type: Custom, Source: sg-03f1bc3bad010ecd5, Description: Allow 5000 port access from ALB SG.

The 'Summary' panel on the right indicates the following configuration:

- Number of instances: 1
- Software Image (AMI): Amazon Linux 2023 AMI 2023.9.2
- Virtual server type (instance type): t3.micro
- Firewall (security group): New security group
- Storage (volumes): 1 volume(s) - 8 GiB

圖A-2-6 網路設定(Security Group)

7. 硬碟設定：

The screenshot shows the AWS Management Console interface for launching an instance. The 'Configure storage' tab is active, displaying the 'Configure storage' section. A single volume is listed:

- 1x 8 GiB gp3**: Root volume, 3000 IOPS, Not encrypted.

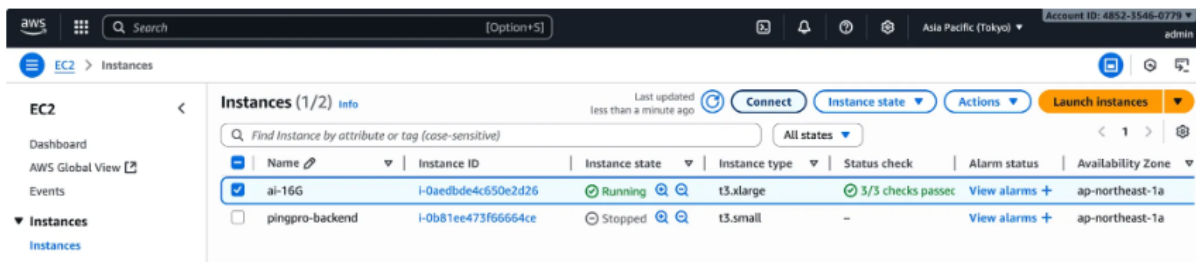
The 'Summary' panel on the right indicates the following configuration:

- Number of instances: 1
- Software Image (AMI): Amazon Linux 2023 AMI 2023.9.2
- Virtual server type (instance type): t3.micro
- Firewall (security group): New security group
- Storage (volumes): 1 volume(s) - 8 GiB

圖A-2-7 硬碟設定

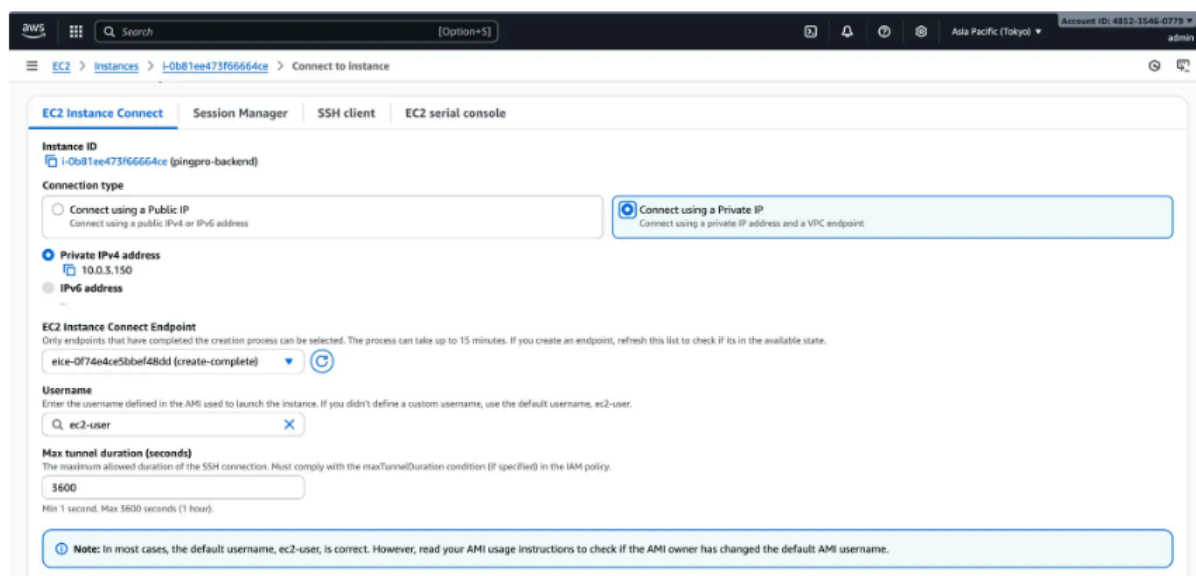
Connect to EC2:

1. 不論哪種OS的EC2，都可以透過以下介面點選Connect來對機器連線：



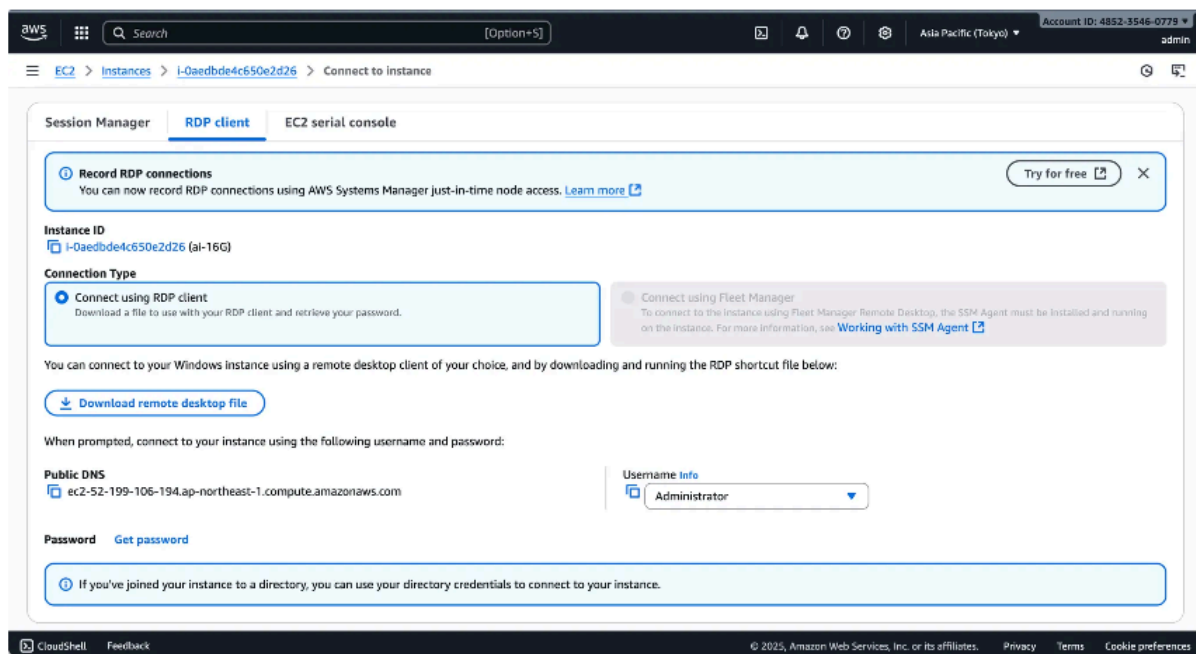
圖A-3-1 EC2 Connect

2. Linux 連線方式：



圖A-3-2 Linux連線方式

3. Windows 連線方式：



圖A-3-3 Windows連線方式

4. Dashboard 部署過程:

1. 安裝 git:

```
sudo dnf install -y git
```

2. python3.11:

```
sudo dnf install -y python3.11
```

3. 安裝 uv:

```
curl -Lsf https://astral.sh/uv/install.sh | sh # 下載 uv 並執行安裝檔
```

```
export PATH="$HOME/.cargo/bin:$PATH" # 註冊 uv 指令
```

```
source ~/.bashrc # reload bashrc 設定檔
```

```
uv --version # 確認有安裝成功
```

4. 安裝 docker(for DB):

```
sudo dnf install -y docker
```

```
sudo systemctl enable docker # 允許背景執行 docker daemon
```

```
sudo systemctl start docker # 執行 docker daemon
```

```
docker # 確認有正確安裝並開啟 docker daemon
```

5. 安裝 docker compose:

```
DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
```

```
mkdir -p $DOCKER_CONFIG/cli-plugins
```

```
curl -SL
```

```
https://github.com/docker/compose/releases/download/v2.29.7/docker-compose-linux-x86_64 -o  
$DOCKER_CONFIG/cli-plugins/docker-compose
```

```
# 以上皆為下載指令
```

```
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose # 給 binary 加上可被執行權限
sudo usermod -aG docker $USER # 讓現在這個 user (ec2-user) 也可以執行這個指令、不需要在前面加上
sudo
exit # 重開 ssh 連線才會生效
```

6. Git clone repo:

```
git clone https://github.com/vxnnio/sport_analytics.git
uv sync # 直接安裝所有相關套件
```

7. 寫好 .env 設定檔:

```
vim .env # 使用 vim 編輯器來改設定檔內容
```

8. 開啟 docker compose stack，讓 DB 正確執行:

```
docker compose up --build -d
docker ps # 確認容器真的有在背景跑起來
```

9. 將 server 設定為背景執行程式:

```
sudo vi /etc/systemd/system/flask-app.service # 寫背景執行 service 的設定檔，service name 叫
flask-app
sudo systemctl daemon-reload # reload 以讀取剛寫好的 service
sudo systemctl enable flask-app # 允許背景執行 flask-app service (like docker)
sudo systemctl start flask-app # 背景執行 flask-app
sudo systemctl status flask-app # 確認 flask-app 跑起來之後輸出的 logs
```

10. flask-app.service 設定檔內容，一樣用 vi 或 vim 寫入:

```
[Unit]
Description=Flask Application in venv
After=network-online.target
Wants=network-online.target
```

[Service]

User=ec2-user

Group=ec2-user

WorkingDirectory=/home/ec2-user/sport_analytics

Environment="PATH=/home/ec2-user/sport_analytics/.venv/bin"

Environment="PYTHONUNBUFFERED=1"

ExecStart=/home/ec2-user/sport_analytics/.venv/bin/python
/home/ec2-user/sport_analytics/main.py

Restart=always

RestartSec=2

StandardOutput=journal

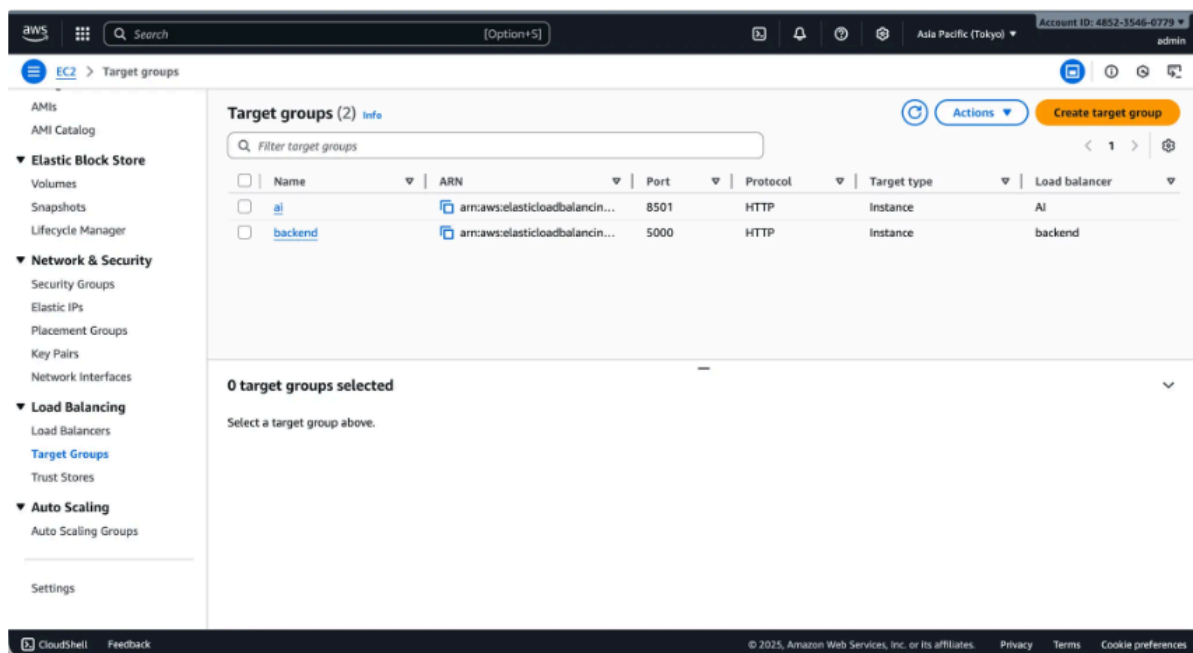
StandardError=journal

[Install]

WantedBy=multi-user.target

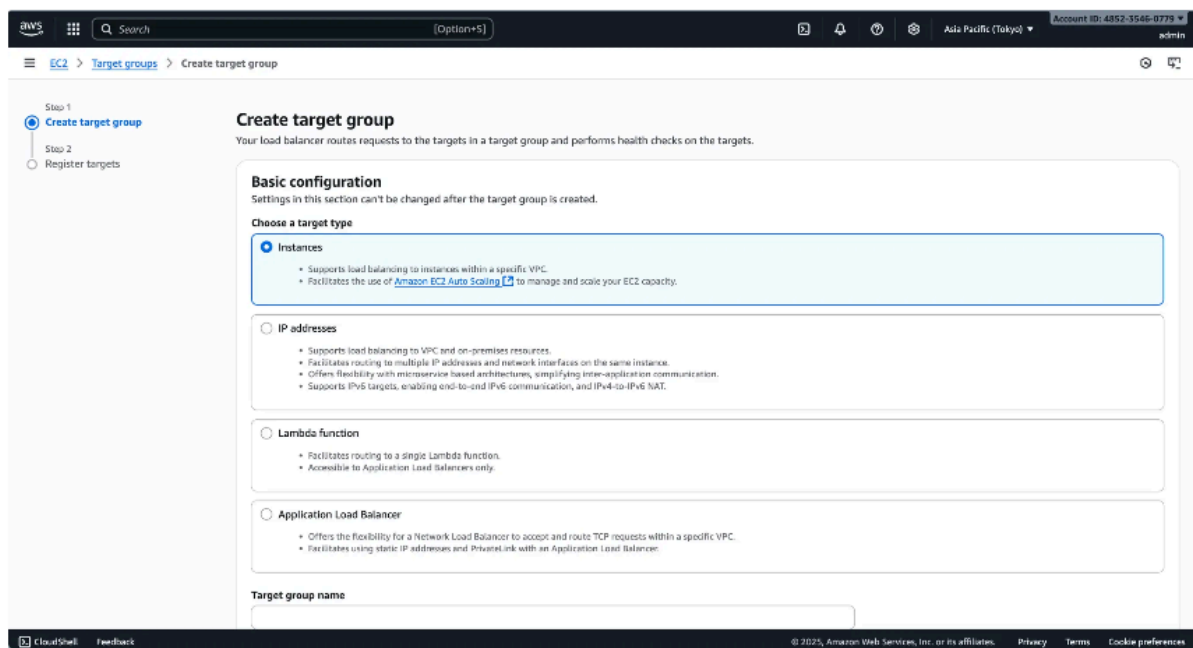
5. ALB 設定：

1. 在介面側邊，有一項Target Group，選Create Target Group:



圖A-5-1 Linux連線方式

2. 選取instance，因為目標都是EC2裡面的Servers:



圖A-5-2 選取instance

3. 選擇EC2所在的VPC及應用程式所能接受的port&HTTP協定代數：

The screenshot shows the 'Create target group' page in the AWS Management Console. The 'Target group name' is 'backend'. The 'Protocol' is 'HTTP' and the 'Port' is '5000'. The 'IP address type' is 'IPv4'. The 'VPC' is 'vpc-09cab6fca66cb9b7 (pingpro)'. The 'Protocol version' is 'HTTP1'.

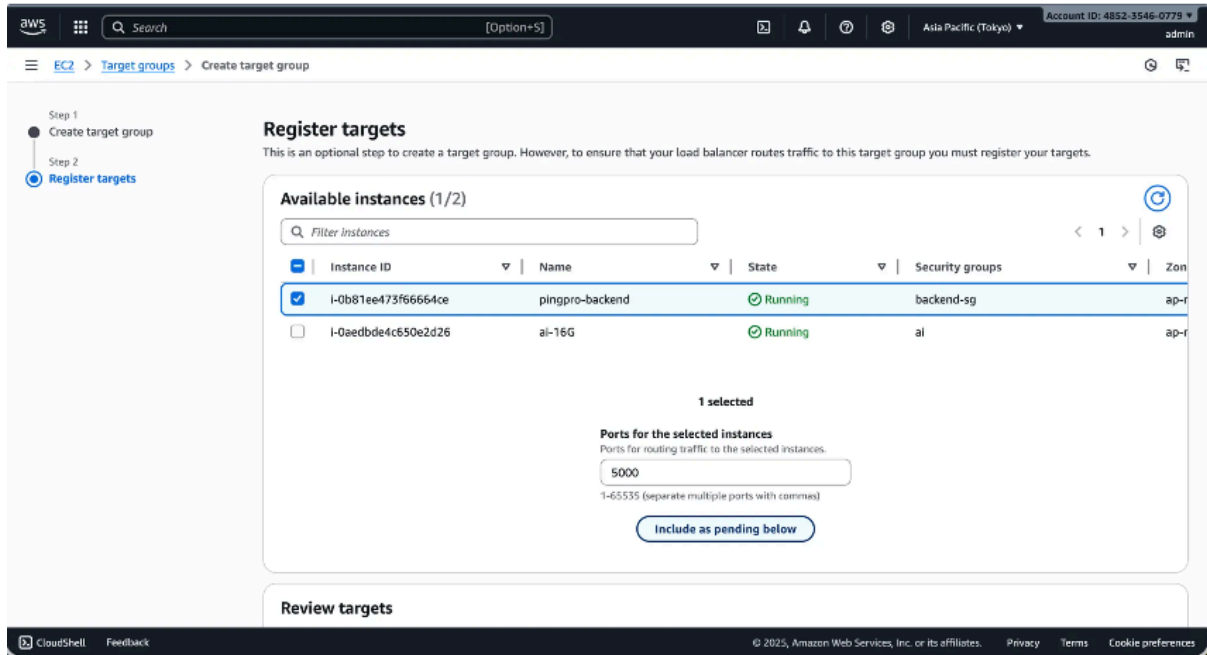
圖A-5-3 選擇EC2所在的VPC

4. 設定health checks的路徑：

The screenshot shows the 'Create target group' page in the AWS Management Console, specifically the 'Health checks' section. The 'Health check protocol' is 'HTTP' and the 'Health check path' is '/'. The 'Attributes' section shows a message about default attributes.

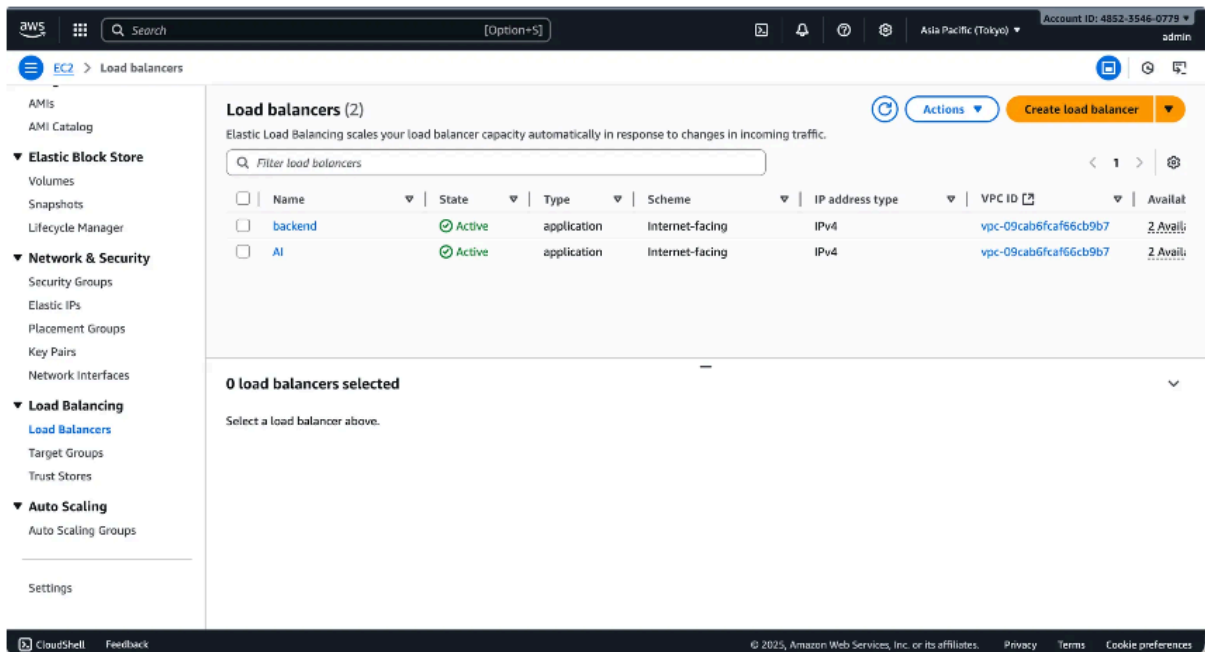
圖A-5-4 設定health checks

5. 選擇 instance 作為 target:



圖A-5-5 選擇instance為target

6. 側邊欄有Load Balancers，選reate load balancer、接著選ALB:



圖A-5-6 選ALB

7. 填入基本資訊、選擇要公開 ALB:

The screenshot shows the 'Create Application Load Balancer' page in the AWS Management Console. The 'Basic configuration' tab is active. The 'Load balancer name' field contains 'Test'. The 'Scheme' is set to 'Internet-facing'. The 'Load balancer IP address type' is set to 'IPv4'.

How Application Load Balancers work

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.
Test
A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**
• Serves internet-facing traffic.
• Has public IP addresses.
• DNS name resolves to public IPs.
• Requires a public subnet.

☐ **Internal**
• Serves internal traffic.
• Has private IP addresses.
• DNS name resolves to private IPs.
• Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type [Info](#)
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**
Includes only IPv4 addresses.

☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

☐ **Dualstack without public IPv4**
Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with internet-facing load balancers only.

圖A-5-7 填入基本資訊、選擇要公開 ALB

8. 網路設定選EC2所在的VPC，並讓ALB同時部署在兩個AZ以上:

The screenshot shows the 'Create Application Load Balancer' page in the AWS Management Console, with the 'Network mapping' tab active. The 'VPC' is set to 'vpc-09cab6fca66cb9b7 (pingpro)'. The 'IP pools' section has 'Use IPAM pool for public IPv4 addresses' unchecked. Two subnets are selected for availability zones 'ap-northeast-1a' and 'ap-northeast-1c'.

Network mapping [Info](#)
The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC [Info](#)
The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-09cab6fca66cb9b7 (pingpro)
10.0.0.0/21

[Create VPC](#)

IP pools [Info](#)
You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the [Amazon VPC IP Address Manager console](#).

☐ **Use IPAM pool for public IPv4 addresses**
The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

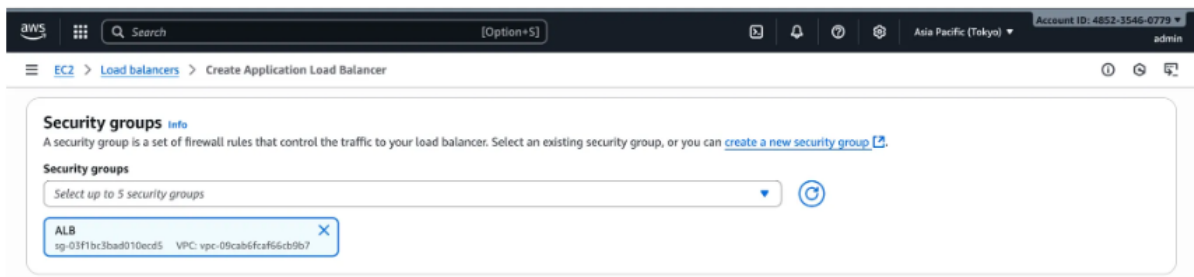
Availability Zones and subnets [Info](#)
Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ **ap-northeast-1a (apne1-az4)**
Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.
subnet-09d0ed14b515f6115
IPv4 subnet CIDR: 10.0.0.0/24
pingpro-public-ap-northeast-1a

☒ **ap-northeast-1c (apne1-az1)**
Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.
subnet-0e9805da9224a0ead
IPv4 subnet CIDR: 10.0.4.0/24
pingpro-public-ap-northeast-1c

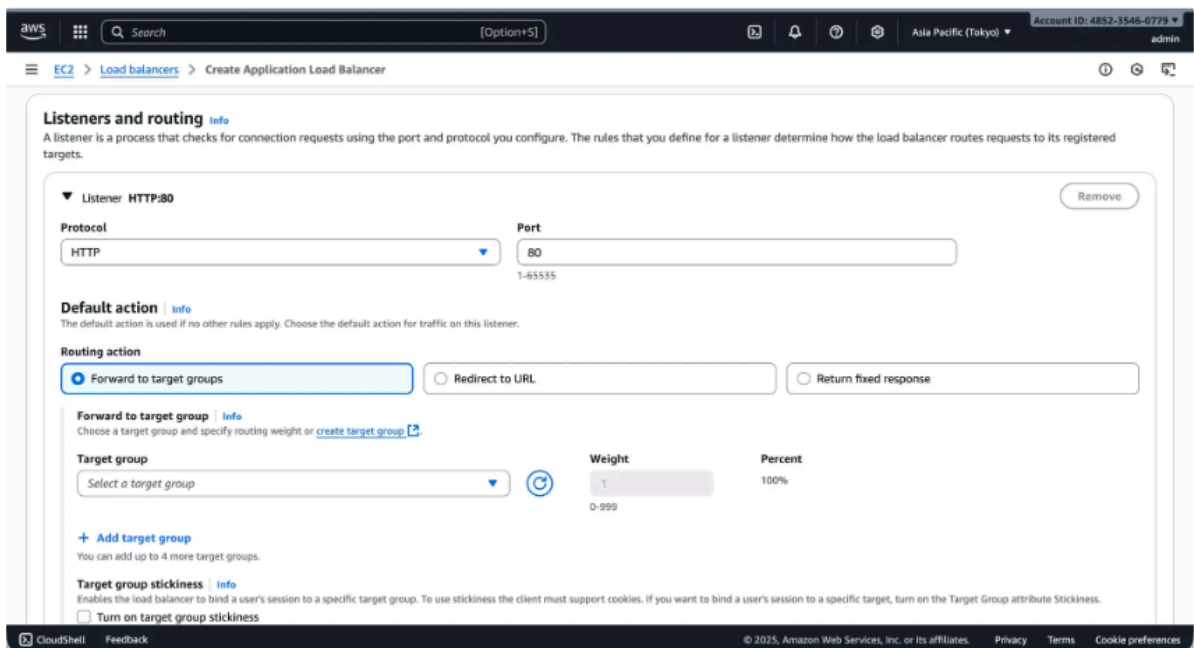
圖A-5-8 讓ALB同時部署在兩個AZ以上

9. 在ALB身上掛 Security Group:



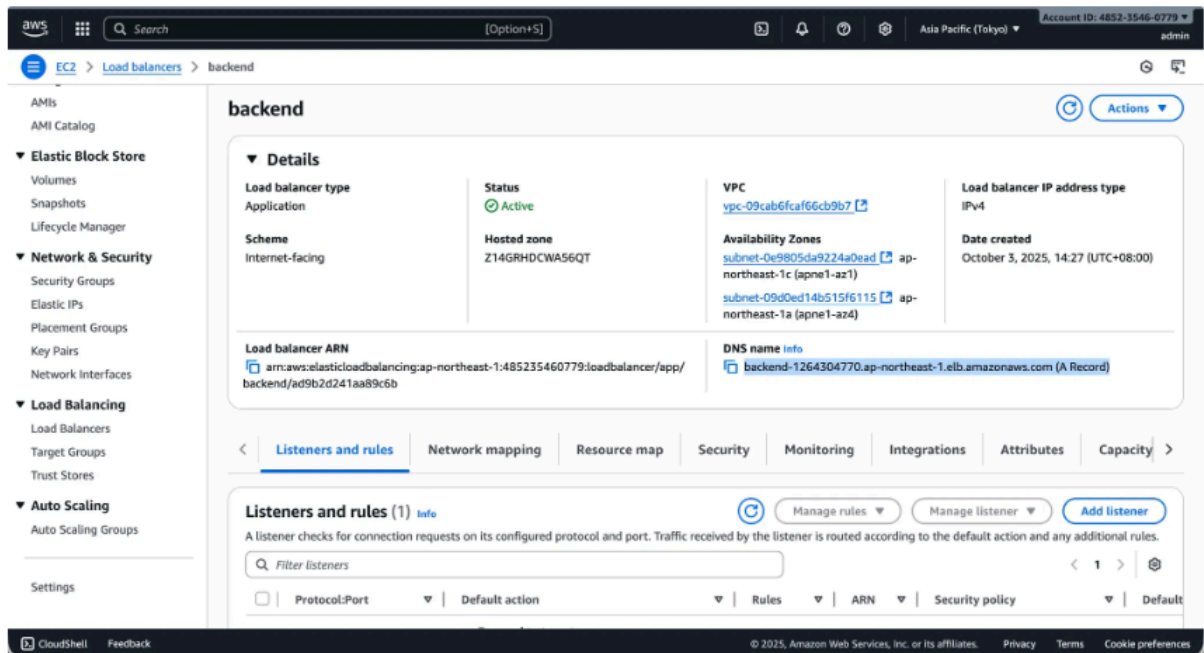
圖A-5-9 在ALB身上掛 Security Group

10. Listener 設定:



圖A-5-10 Listener 設定

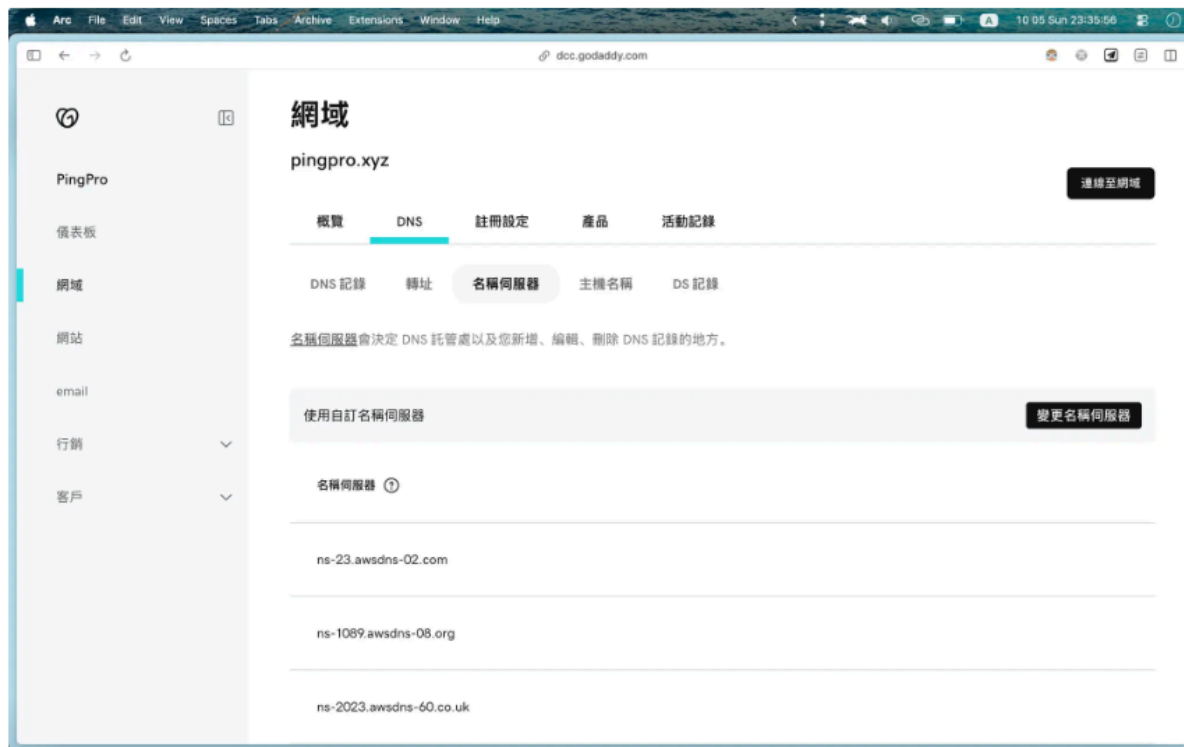
11. 透過詳細資訊頁上的 DNS 對 ALB 發出請求：



圖A-5-11 透過DNS對ALB發出請求

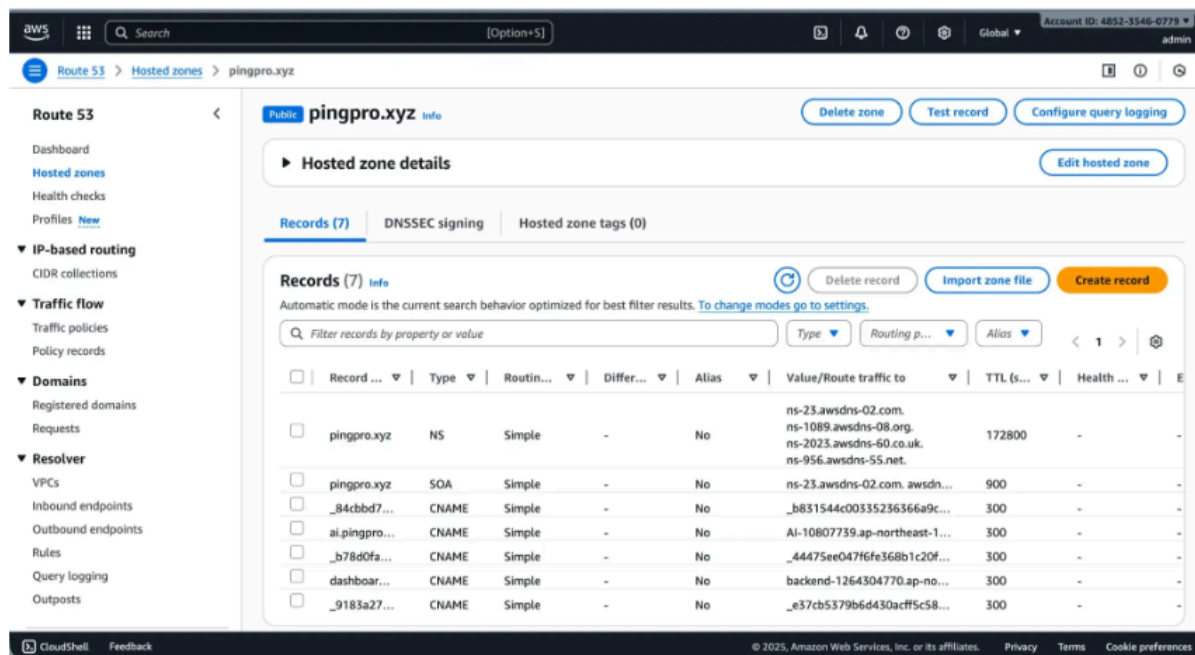
6. DNS 設定：

1. GoDaddy的介面上更新Name Server records成AWS hosted zone中所提供的紀錄：



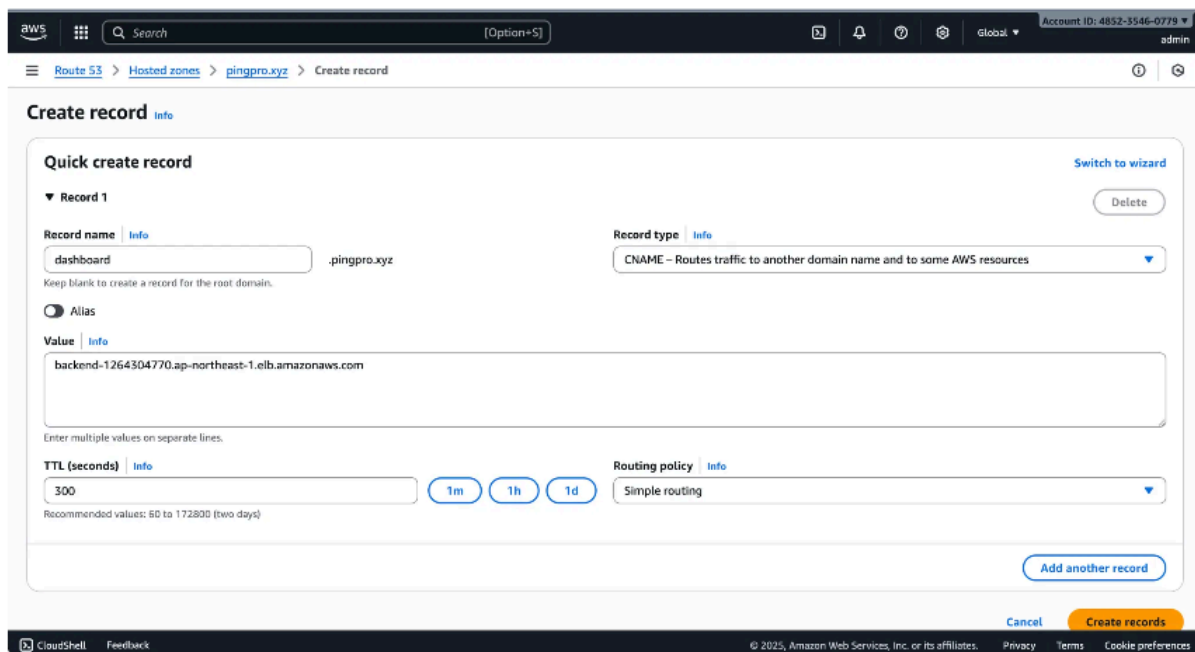
圖A-6-1 更新Name Server records成AWS hosted zone

2. 在 Route53 裡管理 DNS records:



圖A-6-2 在Route53裡管理DNS records

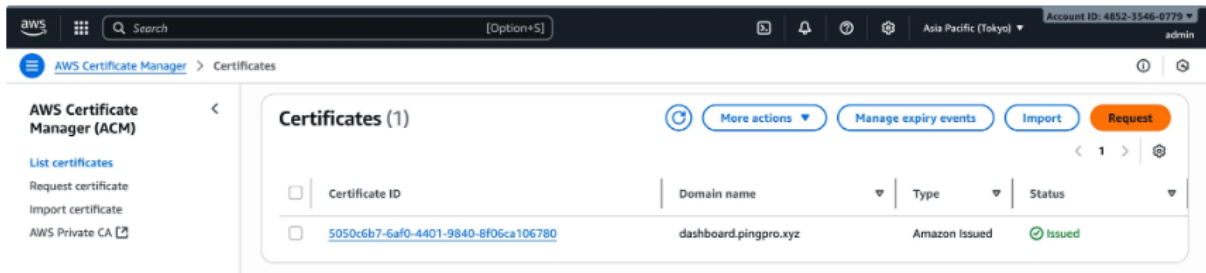
3. 新增一個CNAME record將dashboard.pingpro.xyz指向backend的ALB:



圖A-6-3 新增CNAME record

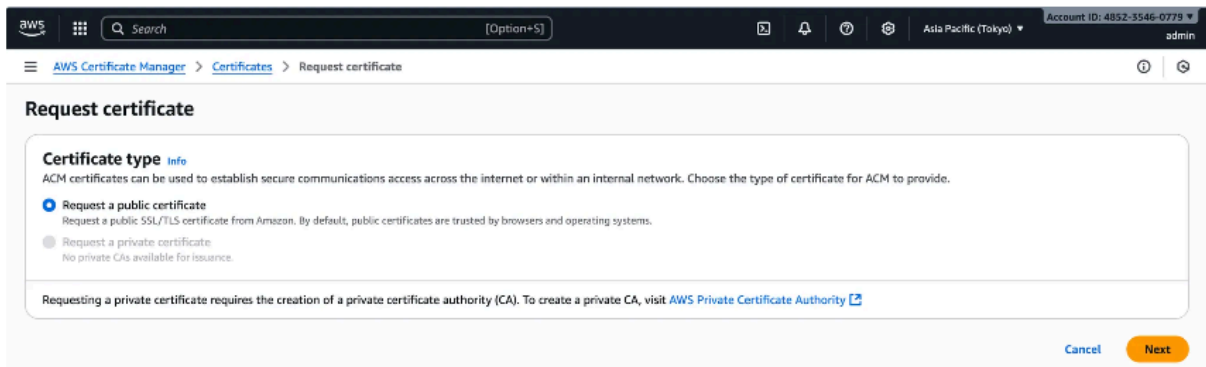
7. ACM 設定：

1. 上 ACM 請求憑證：



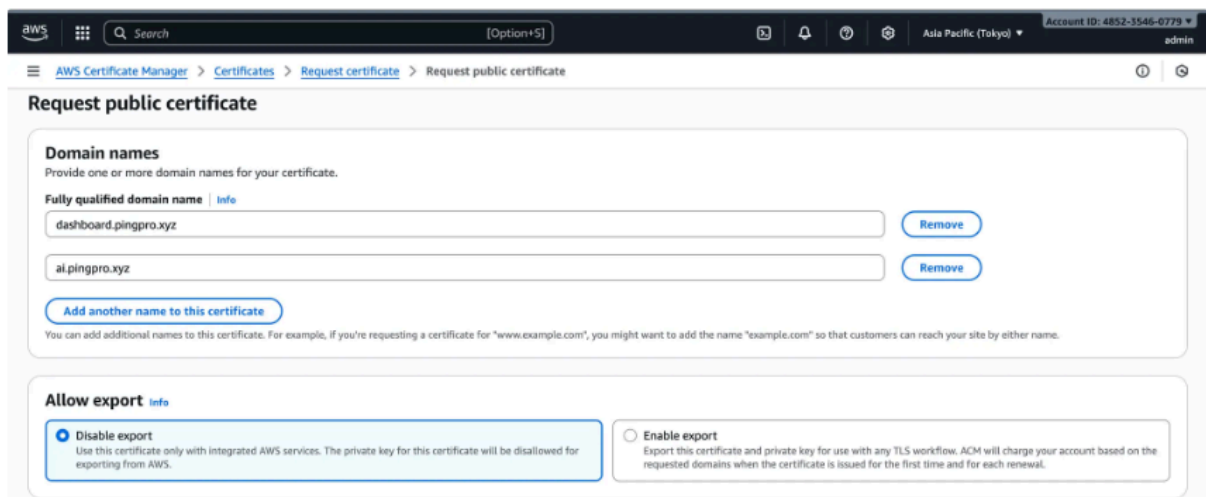
圖A-7-1 上ACM請求憑證

2. 選擇公開憑證：



圖A-7-2 選擇公開憑證

3. 輸入所有需要驗證的 domain names：



圖A-7-3 輸入所有需要驗證的domain names

4. Validation method 選擇 DNS:

The screenshot displays the AWS Certificate Manager console interface for requesting a public certificate. The breadcrumb trail indicates the path: AWS Certificate Manager > Certificates > Request certificate > Request public certificate. The page is divided into three main sections: Validation method, Key algorithm, and Tags. In the Validation method section, 'DNS validation - recommended' is selected with a radio button, accompanied by a note about domain ownership. The Key algorithm section shows 'RSA 2048' as the selected option, with notes about its widespread use and cryptographic strength compared to ECDSA options. The Tags section indicates no tags are currently associated with the resource, with an 'Add new tag' button and a limit of 50 tags. At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Request'. The footer of the console shows 'CloudShell', 'Feedback', copyright information for 2025, and links for 'Privacy', 'Terms', and 'Cookie preferences'.

Validation method [Info](#)
Select a method for validating domain ownership.

☒ **DNS validation - recommended**
Choose this option if you are authorized to modify the DNS configuration for the domains in your certificate request.

☐ **Email validation**
Choose this option if you do not have permission or cannot obtain permission to modify the DNS configuration for the domains in your certificate request.

Key algorithm [Info](#)
Select an encryption algorithm. Some algorithms may not be supported by all AWS services.

☒ **RSA 2048**
RSA is the most widely used key type.

☐ **ECDSA P 256**
Equivalent in cryptographic strength to RSA 3072.

☐ **ECDSA P 384**
Equivalent in cryptographic strength to RSA 7680.

Tags [Info](#)
No tags associated with the resource.

[Add new tag](#)
You can add up to 50 tags.

[Cancel](#) [Previous](#) [Request](#)

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

圖A-7-4 Validation method 選擇 DNS